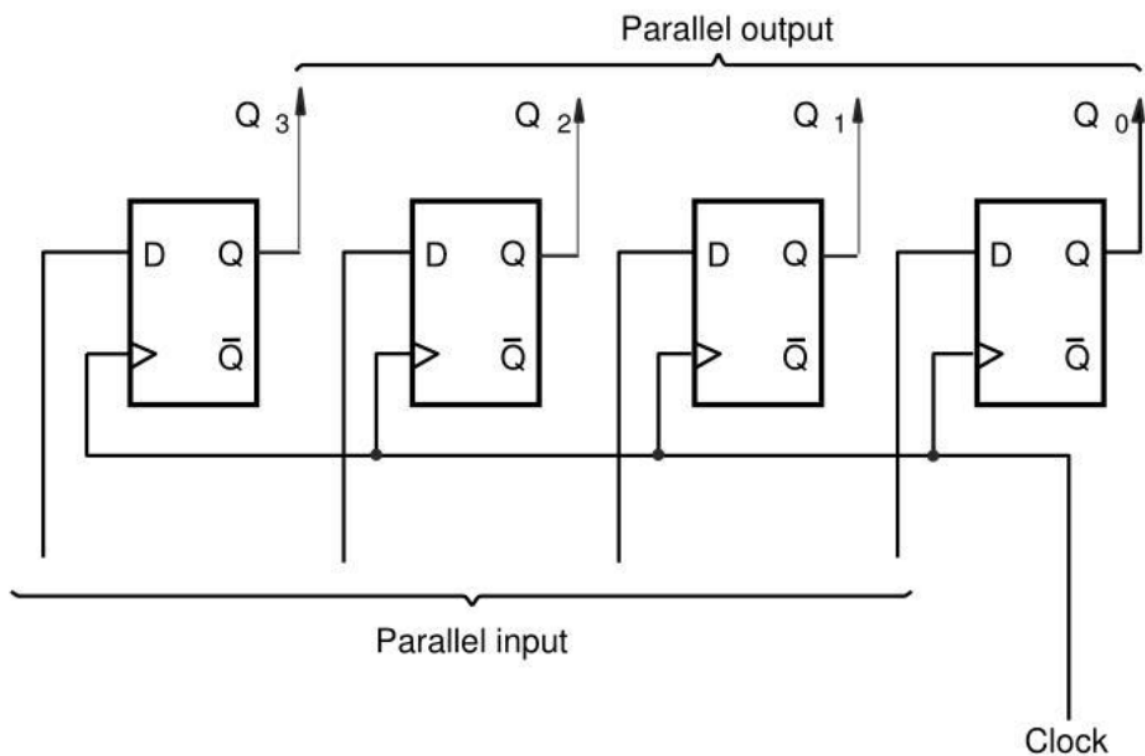




数字电路与 VHDL 设计教程

围绕 23 个重点电路构建的完整知识体系

GitHub 仓库: <https://github.com/myp-super/SXU-CK-Learning-materials>

myp

2026 年 6 月 27 日

目录

前言	15
第一部分 组合逻辑电路体系	18
第一章 8-3 编码器	19
1.1 第一步：解决什么问题	19
1.2 第二步：输入输出关系	19
1.3 第三步：真值表	20
1.4 第四步：逻辑推导	20
1.5 第五步：结构化设计	20
1.6 第六步：为什么这样设计	21
1.7 第七步：考试常见变式	22
1.8 第八步：VHDL 实现	22
1.8.1 普通 8-3 编码器	22
1.8.2 优先级编码器	23
1.9 第九步：Testbench 验证	24
1.10 第十步：如何快速解题	25
1.11 变式详解	25
第二章 8-3 编码器——5 大变式详解	26
2.1 变式 1：16-4 编码器	26
2.2 变式 2：优先级编码器（必考！）	28
2.3 变式 3：带使能端的编码器	30
2.4 变式 4：带 GS（Group Select）输出的编码器	31
2.5 变式 5：低电平有效的编码器	33
2.6 编码器变式总结	34
第三章 3-8 译码器	35
3.1 第一步：解决什么问题	35

3.2	第二步：输入输出关系	35
3.3	第三步：真值表	36
3.4	第四步：逻辑推导	36
3.5	第五步：结构化设计	37
3.6	第六步：为什么这样设计	37
3.7	第七步：考试常见变式	37
3.8	第八步：VHDL 实现	38
3.8.1	普通 3-8 译码器（高电平有效）	38
3.8.2	74LS138 风格（低电平有效 + 使能端）	39
3.9	第九步：Testbench 验证	40
3.10	第十步：如何快速解题	41
3.11	变式详解	41
第四章	3-8 译码器——5 大变式详解	42
4.1	变式 1：74LS138 风格译码器（低有效输出 + 三使能端）——必考！	42
4.2	变式 2：用译码器实现任意逻辑函数（必考！）	43
4.3	变式 3：用 3-8 译码器构建 4-16 译码器（片选级联）	45
4.4	变式 4：用译码器做地址译码	46
4.5	变式 5：显示译码器（BCD→7 段）	47
4.6	译码器变式总结	47
第五章	BCD 译码器	49
5.1	第一步：解决什么问题	49
5.2	第二步：输入输出关系	49
5.3	第三步：真值表	50
5.4	第四步：逻辑推导	50
5.5	第五步：结构化设计	50
5.6	第六步：为什么这样设计	51
5.7	第七步：考试常见变式	51
5.8	第八步：VHDL 实现	52
5.9	第九步：Testbench 验证	53
5.10	第十步：如何快速解题	54
5.11	变式详解	54
第六章	BCD 译码器——5 大变式详解	55
6.1	变式 1：多位 BCD 动态扫描显示	55
6.2	变式 2：带锁存的 BCD 译码器	56
6.3	变式 3：共阳极 vs 共阴极数码管	57

6.4	变式 4: 计数器 + 译码器 + 数码管——综合题	59
6.5	变式 5: 16 进制显示译码器 (0-F)	60
第七章	4 选 1 数据选择器 (MUX)	62
7.1	第一步: 解决什么问题	62
7.2	第二步: 输入输出关系	62
7.3	第三步: 真值表	62
7.4	第四步: 逻辑推导	63
7.5	第五步: 结构化设计	63
7.6	第六步: 为什么这样设计	63
7.7	第七步: 考试常见变式	64
7.8	第八步: VHDL 实现	65
7.9	第九步: Testbench 验证	66
7.10	第十步: 如何快速解题	67
7.11	变式详解	67
第八章	4 选 1 MUX——4 大变式详解	68
8.1	变式 1: 用 MUX 实现任意逻辑函数 (必考!)	68
8.2	变式 2: 双 4 选 1 MUX 构建 8 选 1 MUX	69
8.3	变式 3: 带使能端的 MUX	70
8.4	变式 4: MUX 扩展——16 选 1	71
第九章	4 位数据比较器	72
9.1	第一步: 解决什么问题	72
9.2	第二步: 输入输出关系	72
9.3	第三步: 比较逻辑	72
9.4	第四步: 逻辑推导	73
9.5	第五步: 结构化设计	73
9.6	第六步: 为什么这样设计	74
9.7	第七步: 考试常见变式	74
9.8	第八步: VHDL 实现	74
9.9	第九步: Testbench 验证	75
9.10	第十步: 如何快速解题	76
9.11	变式详解	77
第十章	4 位比较器——4 大变式详解	78
10.1	变式 1: 8 位比较器 (两片 4 位级联)	78
10.2	变式 2: 只输出“相等”的比较器	80
10.3	变式 3: 比较器 + MUX 实现取最大值	81

10.4 变式 4: 带使能的比较器	81
第十一章 4 位加法器	83
11.1 第一步: 解决什么问题	83
11.2 第二步: 输入输出关系	83
11.3 第三步: 一位全加器——最基本的加法单元	83
11.4 第四步: 行波进位加法器 (Ripple Carry Adder)	84
11.5 第五步: 结构化设计	84
11.6 第六步: 为什么这样设计	85
11.7 第七步: 考试常见变式	85
11.8 第八步: VHDL 实现	86
11.9 第九步: Testbench 验证	87
11.10 第十步: 如何快速解题	88
11.11 变式详解	88
第十二章 4 位加法器——5 大变式详解	89
12.1 变式 1: 8 位/16 位加法器 (级联)	89
12.2 变式 2: 减法器 (用加法器实现 $A-B$)	90
12.3 变式 3: 加法器 + 寄存器 = 累加器	91
12.4 变式 4: 带溢出检测的加法器	92
12.5 变式 5: BCD 加法器	93
第十三章 级联二选一数据选择器	95
13.1 第一步: 解决什么问题	95
13.2 第二步: 用 2 选 1 MUX 构建 4 选 1 MUX	95
13.3 第三步: 真值表	96
13.4 第四步: 逻辑推导	96
13.5 第五步: 结构化设计——通用级联规律	96
13.6 第六步: 为什么这样设计	97
13.7 第七步: 考试常见变式	97
13.8 第八步: VHDL 实现	98
13.9 第九步: Testbench 验证	99
13.10 第十步: 如何快速解题	100
13.11 变式详解	100
第十四章 级联 MUX——4 大变式详解	101
14.1 变式 1: 用 2 选 1 MUX 构建 8 选 1 MUX	101
14.2 变式 2: 用 4 选 1 MUX 构建 16 选 1 MUX	102
14.3 变式 3: 不对称级联 (构建 6 选 1 MUX)	102

14.4 变式 4: MUX 树实现逻辑函数	103
第十五章 组合逻辑电路: 统一设计套路	104
15.1 什么是组合逻辑电路	104
15.2 组合逻辑电路谱系	104
15.3 统一设计套路	105
15.4 考试中最高频的组合逻辑考点	105
15.5 从组合逻辑到时序逻辑	106
 第二部分 D 触发器体系	 107
第十六章 D 触发器深度解析	108
16.1 为什么组合逻辑不能存储信息	108
16.2 为什么需要存储	108
16.3 什么叫状态? 什么叫记忆?	109
16.4 D 触发器到底保存了什么	109
16.4.1 D 触发器的输入输出	109
16.5 时钟上升沿发生什么	109
16.5.1 逐拍分析: D 触发器的一次工作过程	110
16.6 $Q(n)$ 和 $Q(n+1)$ 到底是什么意思	110
16.7 FPGA 中的寄存器资源与 D 触发器关系	111
16.8 D 触发器的 VHDL 实现	111
16.8.1 VHDL 中的关键点	112
16.8.2 综合成什么硬件	112
16.9 D 触发器 + 组合逻辑 = 一切时序电路	112
16.10 Testbench 验证	113
16.11 D 触发器体系总结	114
16.12 变式详解	114
 第十七章 D 触发器——核心变式详解	 115
17.1 变式 1: 带异步复位的 D 触发器 vs 带同步复位的 D 触发器	115
17.2 变式 2: 带时钟使能的 D 触发器	117
17.3 变式 3: 多位 D 触发器 = 寄存器	117
17.4 变式 4: D 触发器构成 2 分频器	118
17.5 变式 5: 建立/保持时间与最大工作频率	119

第三部分 计数器体系	120
第十八章 计数器统一框架：从状态机视角理解计数器	121
18.1 最重要的一句话	121
18.2 什么是状态	121
18.3 什么是状态转移	122
18.4 为什么计数器本质是状态机	122
18.5 为什么计数器本质是“当前状态 +1”	123
18.6 通用计数器设计流程	123
18.7 为什么所有计数器本质上是一个问题	124
18.8 接下来的学习顺序	124
第十九章 4 位加法计数器——模 2^N 二进制计数器	126
19.1 应用统一框架	126
19.2 逐拍状态分析	126
19.2.1 初始状态	126
19.2.2 时钟第 1 拍到第 16 拍	127
19.3 为什么 Q0 翻转最快	127
19.4 VHDL 实现	128
19.5 内部寄存器变化过程	128
19.6 Testbench	129
第二十章 模 12 加法计数器	131
20.1 应用统一框架	131
20.2 逐拍状态分析	131
20.2.1 关键问题：为什么模 12 不能自然溢出	131
20.3 模 12 计数器内部硬件结构	132
20.4 VHDL 实现	132
20.5 常见错误分析	133
20.6 Testbench	134
第二十一章 模 24 计数器	135
21.1 应用统一框架	135
21.2 关键状态分析	135
21.3 为什么是 5 位	135
21.4 VHDL 实现	136
21.5 与模 12 的比较	137

第二十二章 模 60 计数器	138
22.1 应用统一框架	138
22.2 模 60 的实际意义	138
22.3 VHDL 实现	138
22.4 三种模值计数器的统一对比	139
第二十三章 模为任意值的二进制计数器	140
23.1 通用设计公式	140
23.2 为什么这个模板适用于任意 N	140
23.3 实例：模 7 计数器	141
23.4 实例：模 5 计数器	141
23.5 实例：模 100 计数器	141
23.6 边界情况分析	142
23.6.1 $N = 2^k$ 的情况	142
23.6.2 $N = 1$ 的情况	142
23.7 考试常见变式	142
23.8 统一思考题	143
23.9 变式详解	143
第二十四章 计数器——统一框架变式详解	144
24.1 变式 1：带使能的模 N 计数器	144
24.2 变式 2：带同步加载的模 N 计数器	145
24.3 变式 3：递增/递减双向计数器	146
24.4 变式 4：倒计时计数器（预置 $\rightarrow 0 \rightarrow$ 预置）	147
24.5 变式 5：级联计数器——总模值 = 乘积	148
24.6 变式 6：格雷码计数器	148
第二十五章 模 10 BCD 计数器	149
25.1 BCD 计数器与二进制计数器的本质区别	149
25.2 一位 BCD 计数器 = 模 10 计数器	149
25.3 逐拍状态分析	150
25.4 VHDL 实现	150
25.5 BCD 模 10 vs 二进制模 10	151
25.6 BCD 模 10 vs 多位 BCD 计数器	151
第二十六章 模 24 BCD 计数器与模 60 BCD 计数器	152
26.1 BCD 级联的基本思想	152
26.2 模 24 BCD 计数器	152
26.2.1 两种实现方案	152

26.2.2 方案 A: BCD 级联实现	152
26.2.3 方案 B: 统一计数	154
26.3 模 60 BCD 计数器	154
26.4 BCD 级联的通用模板	156
26.5 数字钟表的完整结构	156
26.6 计数器部分总结	157
第四部分 分频器体系	158
第二十七章 用计数器设计分频器	159
27.1 什么是分频	159
27.2 为什么计数器能够分频: 从状态变化过程理解	159
27.2.1 回顾 4 位计数器的状态序列	159
27.2.2 关键观察	159
27.3 逐拍分析: 为什么 Q0 天然二分频	160
27.4 逐拍分析: 为什么 Q1 天然四分频	160
27.5 通用规律	161
27.6 偶数分频	161
27.6.1 6 分频器设计	161
27.7 奇数分频	162
27.7.1 3 分频器设计	162
27.8 任意整数分频的统一设计流程	164
27.9 考试常见变式	164
27.10 分频器与计数器的关系总结	165
27.11 变式详解	165
第二十八章 分频器——全部变式详解	166
28.1 变式 1: 直接取 Q_n 实现 2^n 分频	166
28.2 变式 2: 偶数分频 (任意 M)	167
28.3 变式 3: 奇数分频 (双沿法)	168
28.4 变式 4: 占空比可调分频器	168
28.5 变式 5: 多级分频器 (分频比相乘)	169
28.6 变式 6: 秒脉冲发生器	169
第五部分 序列发生器体系	170
第二十九章 用计数器设计序列发生器	171

29.1 什么是序列	171
29.2 为什么计数器可以产生序列	171
29.3 方案 1: 计数器 + 组合逻辑映射	171
29.3.1 设计序列: 0110 0110 0110	171
29.4 方案 2: 状态机型序列发生器	173
29.5 方案 3: 移位寄存器型序列发生器	173
29.6 方案 4: ROM/查找表型序列发生器	173
29.7 方案比较	174
29.8 考试常见变式	174
29.9 变式详解	174
第三十章 序列发生器——全部变式详解	175
30.1 变式 1: 任意周期序列 (计数器 + 映射表)	175
30.2 变式 2: 状态机型序列发生器	176
30.3 变式 3: 移位寄存器 + LFSR 发生器	176
30.4 变式 4: ROM/查找表型序列发生器	176
第六部分 移位寄存器体系	178
第三十一章 移位寄存器	179
31.1 先建立数据流动的思维模型	179
31.2 逐拍分析: 数据如何移动	179
31.2.1 初始条件	179
31.2.2 时钟第 0 拍 (初始状态)	179
31.2.3 时钟第 1 拍上升沿	180
31.2.4 时钟第 2 拍上升沿	180
31.2.5 时钟第 3 拍上升沿	180
31.2.6 时钟第 4 拍上升沿	181
31.2.7 完整数据移动轨迹	181
31.3 核心概念总结	181
31.4 单向移位寄存器: VHDL 实现	181
31.5 双向移位寄存器	182
31.6 带并行装载的移位寄存器	183
31.7 四种输入输出组合	184
31.8 移位寄存器实现分频器	184
31.8.1 基本原理	184
31.8.2 逐拍分析: 4 位环形计数器做 4 分频	185

31.8.3 Johnson 计数器做分频器	185
31.8.4 VHDL 实现：移位寄存器型分频器	185
31.8.5 与计数器分频的对比	186
31.9 移位寄存器实现序列发生器	186
31.9.1 方法一：循环移位型（只能循环输出初值，不是真正的序列发生器）	186
31.9.2 方法二：查表反馈型（通用序列发生器）	187
31.9.3 例题：用 3 位移位寄存器产生序列 00011101 00011101...	188
31.9.4 核心问题：CASE 表是怎么得到的？	188
31.9.5 关键技巧：需要几个 D 触发器？——试 3 个不行就换 4 个	189
31.9.6 完整例题：你需要几个寄存器？	190
31.9.7 速查：多少位寄存器够用？	191
31.9.8 VHDL 实现：5 位寄存器产生序列 0011001	192
31.9.9 两种方案对比	195
31.10 移位寄存器实现串行加法器	195
31.10.1 原理	195
31.10.2 逐拍工作过程	195
31.11 移位寄存器实现串行累加器	197
31.11.1 原理	197
31.12 移位寄存器实现乘法器	197
31.12.1 原理——移位相加法	197
31.13 考试常见变式	199
31.14 Testbench	199
31.15 变式详解	200
第三十二章 移位寄存器——全部变式详解	201
32.1 变式 1：串并转换（SIPO）	201
32.2 变式 2：并串转换（PISO）	201
32.3 变式 3：双向移位	202
32.4 变式 4：循环移位	203
32.5 变式 5：算术移位（保持符号位）	203
第七部分 环形计数器体系	204
第三十三章 移位寄存器构建环形计数器	205
33.1 从移位寄存器出发	205
33.2 逐拍分析：环形计数器如何工作	205

33.2.1 时钟第 0 拍（初始/复位后）	205
33.2.2 时钟第 1 拍上升沿	206
33.2.3 时钟第 2 拍上升沿	206
33.2.4 时钟第 3 拍上升沿	207
33.2.5 时钟第 4 拍上升沿	207
33.2.6 完整的循环过程	207
33.3 环形计数器 vs 普通计数器	208
33.4 为什么反馈后形成循环状态	208
33.5 约翰逊计数器（扭环形计数器）	208
33.6 VHDL 实现	209
33.6.1 约翰逊计数器 VHDL	209
33.7 考试常见变式	210
33.8 Testbench	211
33.9 变式详解	212
第三十四章 环形计数器——全部变式详解	213
34.1 变式 1: N 位环形计数器	213
34.2 变式 2: 约翰逊计数器（2N 个状态）	214
34.3 变式 3: 自启动环形计数器	215
34.4 变式 4: 用环形计数器实现时序控制器	216
第八部分 序列检测器体系	218
第三十五章 序列检测器——状态机的核心应用	219
35.1 什么是序列检测器	219
35.2 为什么序列检测器一定是状态机	219
35.3 1011 序列检测器——Moore 型（可重叠检测）	220
35.3.1 状态定义（5 状态严格 Moore）	220
35.3.2 状态转移图	220
35.3.3 状态转移分析	220
35.3.4 状态转移表	221
35.3.5 VHDL 实现（两段式，异步高电平复位）	221
35.4 1101 序列检测器（Moore 型）	222
35.5 重叠检测 vs 非重叠检测	223
35.6 Moore 型 vs Mealy 型——考试必考核心对比	223
35.6.1 两种状态机模型的根本区别	223
35.6.2 Moore 型 1011 序列检测器（5 状态）	224

35.6.3 Mealy 型 1011 序列检测器 (4 状态)	224
35.6.4 完整对比表	225
35.6.5 VHDL 实现对比 (两段式, 异步高电平复位)	225
35.6.6 输出时序对比	228
35.6.7 考试如何选择	228
35.6.8 硬件综合结果对比	229
35.7 考试常见变式	229
35.8 Testbench	229
35.9 状态机的其他应用——不止序列检测器	231
35.9.1 计数器 = 状态机	231
35.9.2 分频器 = 状态机	233
35.9.3 移位寄存器 = 状态机	234
35.9.4 序列发生器 = 状态机	235
35.9.5 统一视角	237
35.10 变式详解	237
第三十六章 序列检测器——全部变式详解	238
36.1 变式 1: 检测 1011 (重叠 vs 非重叠)	238
36.2 变式 2: 检测 1101	239
36.3 变式 3: 检测 1111 (连续 4 个 1)	241
36.4 变式 4: 检测 0101	242
36.5 变式 5: 用移位寄存器 + 比较器替代状态机	244
第九部分 VHDL 硬件综合专题	246
第三十七章 VHDL 硬件综合专题	247
37.1 学习目标	247
37.2 if-else 综合成什么	247
37.2.1 if-else 级联	247
37.3 case 综合成什么	248
37.4 process 综合成什么	248
37.5 signal 对应什么硬件	249
37.6 variable 对应什么硬件	249
37.7 常见 VHDL 模式的硬件对应	250
37.7.1 计数器代码 → 什么结构	250
37.7.2 移位寄存器代码 → 什么结构	250
37.7.3 状态机代码 → 什么结构	251

37.8 VHDL 编码风格对硬件的影响	251
37.9 关键综合规则	251
37.10 变式详解	252
第三十八章 VHDL 综合——全部变式详解	253
38.1 变式 1: if-else 的综合结果	253
38.2 变式 2: case 的综合结果	255
38.3 变式 3: signal vs variable	256
38.4 变式 4: process 的综合结果	257
38.5 变式 5: 常见 VHDL 模式 → 硬件对照表	259
 第十部分 考试训练体系	 260
第三十九章 计数器考试变式题	261
第四十章 分频器考试变式题	281
第四十一章 移位寄存器考试变式题	286
第四十二章 环形计数器考试变式题	291
第四十三章 序列检测器考试变式题	296
43.1 全书总结	301

前言

本书的目标

这本书不是一本传统的数字电路教材。

传统教材的写法是：数制 → 逻辑门 → 布尔代数 → 卡诺图 → 组合逻辑 → 时序逻辑 → ...

这本书的写法是：

以电路为主线，反向展开其所涉及的数字电路知识。

电路 → 原理 → 状态变化 → 设计思想 → 设计模板 → VHDL 实现 → 考试变式

本书的结构

本书分为十个部分：

第一部分 组合逻辑电路体系——编码器、译码器、MUX、比较器、加法器

第二部分 D 触发器体系——从“为什么需要存储”开始，建立状态与记忆的概念

第三部分 计数器体系——用统一框架串起所有模值计数器

第四部分 分频器体系——从状态变化解释频率减半的物理过程

第五部分 序列发生器体系——多种实现方案比较

第六部分 移位寄存器体系——数据流动的逐拍分析

第七部分 环形计数器体系——从移位寄存器推导环形计数器

第八部分 序列检测器体系——状态机的核心应用

第九部分 VHDL 专题——每种 VHDL 写法综合成什么硬件

第十部分 考试训练体系——每类电路 20+ 道变式题

如何使用本书

1. 如果你时间充裕：从头到尾，按顺序阅读。
2. 如果你时间紧张：直接跳到对应电路的章节，每章是自包含的。
3. 如果你只需要刷题：直接看第十部分。

关于“为什么”

本书最重要的一个原则是：

不讲“是什么”，只讲“为什么”。

每一个设计决策都有原因。

每一个公式都有推导过程。

每一个状态变化都有逐拍分析。

关于代码

不要在理解电路之前看代码。

如果你看到一个电路，第一反应是“它的 VHDL 怎么写”，那你就学反了。

正确的顺序是：

1. 理解这个电路要解决什么问题
2. 理解它的输入输出关系
3. 理解它的内部工作原理
4. 理解为什么这样设计
5. 然后，VHDL 只是这些理解的翻译

如果你理解了设计逻辑，任何 VHDL 变式你都能自己写出来。

写在前面

这套书可能和你在学校里见过的任何一本数字电路教材都不一样。

它不讲数制，不讲卡诺图，不从逻辑代数开始。(它默认你已经学过这些了，并且已经能够利用这些工具分析电路了。)

它从数字电路中典型的 23 个电路开始。

每一个电路，我们不讲废话，只讲三件事：

1. 它为什么存在（解决了什么问题）
2. 它是怎么工作的（原理和状态变化）
3. 怎么用它解题（考试变式和解题模板）

祝学习顺利。

——myp

第一部分

组合逻辑电路体系

第一章 8-3 编码器

1.1 第一步：解决什么问题

想象一个场景：你的 FPGA 有 8 个按键，编号 0 到 7。用户按下其中一个按键。

- 如果用 8 根信号线传给处理器，需要 8 个引脚——浪费资源
- 更好的方案：用 3 根信号线传递“哪个按键被按下”的信息
- 因为 $2^3 = 8$ ，3 根线恰好能表示 8 种状态

编码器（Encoder）的本质：将 2^n 个输入信号，压缩成 n 位二进制输出。

编码器的本质：编码器 = 信号压缩器： 2^n 个输入 $\rightarrow n$ 位输出
8-3 编码器：8 个输入 $\rightarrow 3$ 位输出

1.2 第二步：输入输出关系

- **输入：** $I_0, I_1, I_2, I_3, I_4, I_5, I_6, I_7$ （8 个输入信号，高电平有效）
- **输出：** Y_2, Y_1, Y_0 （3 位二进制编码输出）
- **约束：**任意时刻最多只有一个输入为 1

当 $I_k = 1$ （且其他输入为 0）时，输出 $(Y_2 Y_1 Y_0)$ 应该等于 k 的二进制表示。

1.3 第三步：真值表

有效输入	Y_2	Y_1	Y_0
$I_0 = 1$	0	0	0
$I_1 = 1$	0	0	1
$I_2 = 1$	0	1	0
$I_3 = 1$	0	1	1
$I_4 = 1$	1	0	0
$I_5 = 1$	1	0	1
$I_6 = 1$	1	1	0
$I_7 = 1$	1	1	1

1.4 第四步：逻辑推导

观察真值表，找出每个输出位什么时候为 1：

- $Y_2 = 1$ 当 $I_4 = 1$ 或 $I_5 = 1$ 或 $I_6 = 1$ 或 $I_7 = 1$
- $Y_1 = 1$ 当 $I_2 = 1$ 或 $I_3 = 1$ 或 $I_6 = 1$ 或 $I_7 = 1$
- $Y_0 = 1$ 当 $I_1 = 1$ 或 $I_3 = 1$ 或 $I_5 = 1$ 或 $I_7 = 1$

因此，逻辑表达式为：

$$Y_2 = I_4 + I_5 + I_6 + I_7 \quad (1.1)$$

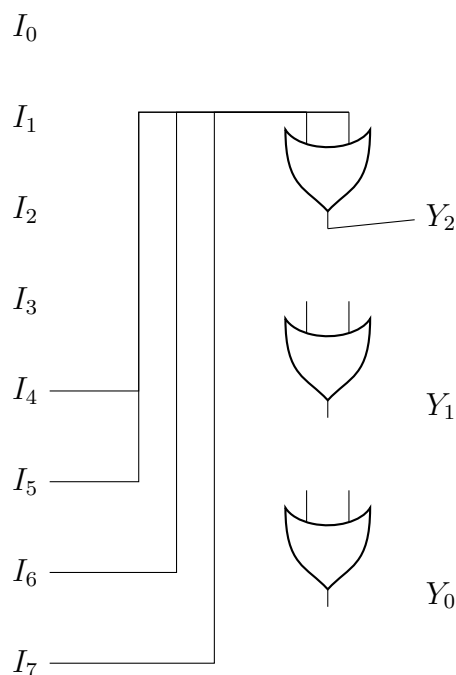
$$Y_1 = I_2 + I_3 + I_6 + I_7 \quad (1.2)$$

$$Y_0 = I_1 + I_3 + I_5 + I_7 \quad (1.3)$$

每个输出位都是若干输入的**逻辑或（OR）**。

1.5 第五步：结构化设计

8-3 编码器内部结构：



设计规律： Y_k 的输出 = 所有编号第 k 位为 1 的输入的 OR。

- Y_2 (bit 2 = 4 的位)：连接 I_4, I_5, I_6, I_7 (这些编号的二进制第 2 位都是 1)
- Y_1 (bit 1 = 2 的位)：连接 I_2, I_3, I_6, I_7
- Y_0 (bit 0 = 1 的位)：连接 I_1, I_3, I_5, I_7

1.6 第六步：为什么这样设计

1. 为什么用 OR 门？

观察真值表：对于 Y_k ，任何使 $Y_k = 1$ 的输入条件之间是“或”的关系——只要其中任何一个条件满足， Y_k 就是 1。

2. 为什么输入之间存在约束？

如果两个输入同时为 1，OR 门的输出仍然是 1，但编码结果就错了（对应了另一个不需要的编码）。所以编码器假设同一时刻只有一个输入有效。

3. 优先级编码器的改进动机？

当多个输入同时有效时，普通编码器产生错误输出。优先级编码器增加了优先级逻辑——一般编号大的输入优先级高。这需要用 if-else 级联逻辑而非简单 OR 门。

1.7 第七步：考试常见变式

1. 变式 1：16-4 编码器

将 16 个输入编码为 4 位输出。原理完全相同，只是规模扩大。 $Y_3 = I_8 + I_9 + \cdots + I_{15}$ ，以此类推。

2. 变式 2：优先级编码器（Priority Encoder）

允许同时多个输入为 1，输出最高优先级输入的编号。这是考试的热门考点。

优先级编码器的真值表（优先级： $I_7 > I_6 > \cdots > I_0$ ）：

I_7	I_6	I_5	...	$(Y_2Y_1Y_0)$
1	x	x	...	111
0	1	x	...	110
0	0	1	...	101
<i>dots</i>	<i>dots</i>	<i>dots</i>	$\ddots$	<i>dots</i> height

3. 变式 3：带使能端的编码器

增加一个 Enable 输入，E=1 时编码器正常工作，E=0 时所有输出为 0。

4. 变式 4：输出有效的编码器

增加一个 GS（Group Select）输出，当任何输入有效时 GS=1，否则 GS=0。用于区分”选择了 I_0 ”（输出 000）和”没有选择”（输出也是 000）。

5. 变式 5：低电平有效的编码器

输入和输出都是低电平有效（即 0 表示有效，1 表示无效）。逻辑门类型可能需要相应改变。

1.8 第八步：VHDL 实现

1.8.1 普通 8-3 编码器

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity encoder_8to3 is
5     port (
6         I : in  std_logic_vector(7 downto 0);
7         Y : out std_logic_vector(2 downto 0)
8     );
```

```

9  end encoder_8to3;
10
11 architecture behavioral of encoder_8to3 is
12 begin
13     process(I)
14     begin
15         case I is
16             when "00000001" => Y <= "000";
17             when "00000010" => Y <= "001";
18             when "00000100" => Y <= "010";
19             when "00001000" => Y <= "011";
20             when "00010000" => Y <= "100";
21             when "00100000" => Y <= "101";
22             when "01000000" => Y <= "110";
23             when "10000000" => Y <= "111";
24             when others      => Y <= "000";
25         end case;
26     end process;
27 end behavioral;

```

Listing 1.1: 8-3 编码器 VHDL 实现 (if-else 版本)

1.8.2 优先级编码器

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity priority_encoder_8to3 is
5      port (
6          I : in  std_logic_vector(7 downto 0);
7          Y : out std_logic_vector(2 downto 0);
8          GS: out std_logic
9      );
10 end priority_encoder_8to3;
11
12 architecture behavioral of priority_encoder_8to3 is
13 begin
14     process(I)
15     begin
16         GS <= '1';
17         if I(7) = '1' then Y <= "111";
18         elsif I(6) = '1' then Y <= "110";
19         elsif I(5) = '1' then Y <= "101";
20         elsif I(4) = '1' then Y <= "100";

```

```

21     elsif I(3) = '1' then Y <= "011";
22     elsif I(2) = '1' then Y <= "010";
23     elsif I(1) = '1' then Y <= "001";
24     elsif I(0) = '1' then Y <= "000";
25     else
26         Y <= "000";
27         GS <= '0'; -- 无有效输入
28     end if;
29 end process;
30 end behavioral;

```

Listing 1.2: 8-3 优先级编码器（使用 if-else 级联）

1.9 第九步：Testbench 验证

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity encoder_8to3_tb is
5  end encoder_8to3_tb;
6
7  architecture test of encoder_8to3_tb is
8      signal I : std_logic_vector(7 downto 0);
9      signal Y : std_logic_vector(2 downto 0);
10
11     component encoder_8to3 is
12     port (
13         I : in  std_logic_vector(7 downto 0);
14         Y : out std_logic_vector(2 downto 0)
15     );
16 end component;
17 begin
18     uut: encoder_8to3 port map (I, Y);
19
20     stimulus: process
21     begin
22         -- 测试每个输入
23         I <= "00000001"; wait for 10 ns;
24         I <= "00000010"; wait for 10 ns;
25         I <= "00000100"; wait for 10 ns;
26         I <= "00001000"; wait for 10 ns;
27         I <= "00010000"; wait for 10 ns;
28         I <= "00100000"; wait for 10 ns;

```



```

29     I <= "01000000"; wait for 10 ns;
30     I <= "10000000"; wait for 10 ns;
31     -- 无输入
32     I <= "00000000"; wait for 10 ns;
33     wait;
34 end process;
35 end test;

```

Listing 1.3: 8-3 编码器 Testbench

验证要点:

- I_0 有效时, $Y = 000$
- I_7 有效时, $Y = 111$
- 中间值按二进制递增

1.10 第十步：如何快速解题

编码器快速解题法:

1. 确定输入输出数量: 2^n 输入对应 n 位输出
2. 写表达式: 第 k 个输出位 = 所有编号中该位为 1 的输入的 OR
3. 优先级编码器: 用 if-else 级联, 优先级从高到低
4. 常考陷阱: I_0 有效时输出 000, 需要 GS 信号区分“无输入”

秒杀口诀: 每个输出位 = 输入编号该位为 1 的所有输入的 OR。

举例:

- 4-2 编码器: $Y_1 = I_2 + I_3$, $Y_0 = I_1 + I_3$
- 16-4 编码器: $Y_3 = I_8 + I_9 + \cdots + I_{15}$, $Y_2 = I_4 + I_5 + I_6 + I_7 + I_{12} + I_{13} + I_{14} + I_{15}$

规律推广: 对于 2^n - n 编码器, $Y_k = \sum I_i$, 其中 i 的二进制表示中第 k 位为 1。

1.11 变式详解

第二章 8-3 编码器——5 大变式详解

2.1 变式 1: 16-4 编码器

完整教学

【电路升级】8-3 编码器： $2^3 = 8$ 个输入 $\rightarrow$ 3 位输出。16-4 编码器： $2^4 = 16$ 个输入 $\rightarrow$ 4 位输出。

【设计思想】原理完全不变——每个输出位 = 所有编号中该位为 1 的输入的 OR。

对于 16-4 编码器，输出 $Y_3Y_2Y_1Y_0$ ，输入 $I_0 \sim I_{15}$ 。

【逻辑推导】

$$Y_3 = I_8 + I_9 + I_{10} + I_{11} + I_{12} + I_{13} + I_{14} + I_{15} \quad (\text{编号 8-15 的 bit3=1}) \quad (2.1)$$

$$Y_2 = I_4 + I_5 + I_6 + I_7 + I_{12} + I_{13} + I_{14} + I_{15} \quad (\text{bit2=1 的编号}) \quad (2.2)$$

$$Y_1 = I_2 + I_3 + I_6 + I_7 + I_{10} + I_{11} + I_{14} + I_{15} \quad (2.3)$$

$$Y_0 = I_1 + I_3 + I_5 + I_7 + I_9 + I_{11} + I_{13} + I_{15} \quad (2.4)$$

【通用公式】对于 2^n - n 编码器：

$$Y_k = \sum_{i=0}^{2^n-1} I_i \quad \text{当 } i \text{ 的第 } k \text{ 位为 1 时}$$

【VHDL——完整 case 版本】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity encoder_16to4 is
5     port (
6         I : in  std_logic_vector(15 downto 0);
7         Y : out std_logic_vector(3  downto 0)
8     );
9 end encoder_16to4;
```

```

11 architecture behavioral of encoder_16to4 is
12 begin
13     process(I)
14     begin
15         case I is
16             when "0000000000000001" => Y <= "0000";
17             when "0000000000000010" => Y <= "0001";
18             when "0000000000000100" => Y <= "0010";
19             when "0000000000001000" => Y <= "0011";
20             when "0000000000100000" => Y <= "0100";
21             when "0000000001000000" => Y <= "0101";
22             when "0000000010000000" => Y <= "0110";
23             when "0000000100000000" => Y <= "0111";
24             when "0000001000000000" => Y <= "1000";
25             when "0000010000000000" => Y <= "1001";
26             when "0000100000000000" => Y <= "1010";
27             when "0001000000000000" => Y <= "1011";
28             when "0010000000000000" => Y <= "1100";
29             when "0010000000000000" => Y <= "1101";
30             when "0100000000000000" => Y <= "1110";
31             when "1000000000000000" => Y <= "1111";
32             when others                => Y <= "0000";
33         end case;
34     end process;
35 end behavioral;

```

Listing 2.1: 16-4 编码器 VHDL 实现 (case 版本)

【更好的写法——用 for 循环查找 (完整 VHDL)】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity encoder_16to4_loop is
6      port (
7          I : in  std_logic_vector(15 downto 0);
8          Y : out std_logic_vector(3  downto 0)
9      );
10 end encoder_16to4_loop;
11
12 architecture behavioral of encoder_16to4_loop is
13 begin
14     process(I)

```

```
15  begin
16      Y <= "0000";
17      for i in 0 to 15 loop
18          if I(i) = '1' then
19              Y <= std_logic_vector(to_unsigned(i, 4));
20          end if;
21      end loop;
22  end process;
23  end behavioral;
```

Listing 2.2: 16-4 编码器 VHDL 实现（for 循环版本）

【考试聚焦】 2^n - n 编码器的通用公式。16-4 编码器需要 16 个 case 分支。

2.2 变式 2：优先级编码器（必考！）

完整教学

【问题】 普通编码器假设同一时刻只有一个输入为 1。但当多个输入同时为 1 时（比如多个按键同时按下），普通编码器产生错误输出。需要**优先级**——编号大的输入优先。

【设计思想】 用 if-else 级联：从高优先级到低优先级依次判断。I7 优先级最高，I0 最低。

【逻辑推导】

I7	I6	I5	I4	I3	I2	I1	I0	Y2	Y1	Y0
1	x	x	x	x	x	x	x	1	1	1
0	1	x	x	x	x	x	x	1	1	0
0	0	1	x	x	x	x	x	1	0	1
0	0	0	1	x	x	x	x	1	0	0
0	0	0	0	1	x	x	x	0	1	1
0	0	0	0	0	1	x	x	0	1	0
0	0	0	0	0	0	1	x	0	0	1
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	0	0	0	0	0

(x= 任意值，不管它是 0 还是 1)

【VHDL——if-else 级联】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity priority_encoder is
5      port (
6          I  : in  std_logic_vector(7 downto 0);
7          Y  : out std_logic_vector(2 downto 0);
8          GS : out std_logic    -- Group Select: 有输入=1
9      );
10 end priority_encoder;
11
12 architecture behavioral of priority_encoder is
13 begin
14     process(I)
15     begin
16         if I(7) = '1' then
17             Y <= "111"; GS <= '1';
18         elsif I(6) = '1' then
19             Y <= "110"; GS <= '1';
20         elsif I(5) = '1' then
21             Y <= "101"; GS <= '1';
22         elsif I(4) = '1' then
23             Y <= "100"; GS <= '1';
24         elsif I(3) = '1' then
25             Y <= "011"; GS <= '1';
26         elsif I(2) = '1' then
27             Y <= "010"; GS <= '1';
28         elsif I(1) = '1' then
29             Y <= "001"; GS <= '1';
30         elsif I(0) = '1' then
31             Y <= "000"; GS <= '1';
32         else
33             Y <= "000"; GS <= '0'; -- 无有效输入
34         end if;
35     end process;
36 end behavioral;

```

Listing 2.3: 8-3 优先级编码器 VHDL 实现 (if-else 级联)

【为什么 if-else 实现优先级】 if-else 的语义：从上到下依次判断，第一个为真的分支被执行。所以 I7 在第一个 if → I7 优先级最高。

【综合成什么硬件】级联 MUX 链（优先级 MUX 树）。I7→I6→...→I0 逐级选择。

【GS 信号的必要性】没有 GS 时，输入全 0（无有效输入）和只有 I0=1 时，输出都是 000——无法区分！GS=1 表示“有有效输入”。

【Testbench——测试优先级】

```
1  -- 同时置 I7 和 I0=1 → 应该输出 111 (I7 优先)
2  I <= "10000001"; wait for 10 ns;
3  -- 输出应为 111, GS=1 (I7 优先级高于 I0)
```

考试聚焦

必考！ 优先级编码器是考试最高频的组合逻辑变式之一。

- if-else 级联 = 优先级递减的硬件
- GS 信号区分“无输入”和“选择 I0”
- 优先级从高到低排列 if 条件

2.3 变式 3：带使能端的编码器

完整教学

【电路】在标准编码器基础上增加 Enable(E) 输入。E=1 时编码器正常工作；E=0 时所有输出为 0（或高阻态）。

【VHDL——完整实现】

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity encoder_with_enable is
5      port (
6          I : in  std_logic_vector(7 downto 0);
7          E : in  std_logic;
8          Y : out std_logic_vector(2 downto 0)
9      );
10 end encoder_with_enable;
```

```
12 architecture behavioral of encoder_with_enable is
13 begin
14     process(I, E)
15     begin
16         if E = '0' then
17             Y <= "000"; -- 禁止输出
18         else
19             -- 正常编码逻辑
20             case I is
21                 when "00000001" => Y <= "000";
22                 when "00000010" => Y <= "001";
23                 when "00000100" => Y <= "010";
24                 when "00001000" => Y <= "011";
25                 when "00010000" => Y <= "100";
26                 when "00100000" => Y <= "101";
27                 when "01000000" => Y <= "110";
28                 when "10000000" => Y <= "111";
29                 when others      => Y <= "000";
30             end case;
31         end if;
32     end process;
33 end behavioral;
```

Listing 2.4: 带使能端的 8-3 编码器 VHDL 实现

【设计思想】使能端是数字电路中常见的控制信号。在不使用时关断输出以节省功耗/避免总线冲突。

2.4 变式 4：带 GS（Group Select）输出的编码器

完整教学

【问题】普通 8-3 编码器：当所有输入为 0 时，输出 000。但当 $I_0 = 1$ 时，输出也是 000。如何区分这两种情况？

【方案】增加 GS（Group Select/Valid）输出：

- 任意输入 = 1 $\rightarrow$ GS = 1（表示输出有效）
- 所有输入 = 0 $\rightarrow$ GS = 0（表示输出无效/无输入选中）

【VHDL——完整实现】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity encoder_with_gs is
5     port (
6         I : in  std_logic_vector(7 downto 0);
7         Y : out std_logic_vector(2 downto 0);
8         GS : out std_logic
9     );
10 end encoder_with_gs;
11
12 architecture behavioral of encoder_with_gs is
13 begin
14     process(I)
15     begin
16         -- GS: 任意输入为1时置1, 否则置0
17         if I = "00000000" then
18             GS <= '0';
19         else
20             GS <= '1';
21         end if;
22
23         -- 编码逻辑
24         case I is
25             when "00000001" => Y <= "000";
26             when "00000010" => Y <= "001";
27             when "00000100" => Y <= "010";
28             when "00001000" => Y <= "011";
29             when "00010000" => Y <= "100";
30             when "00100000" => Y <= "101";
31             when "01000000" => Y <= "110";
32             when "10000000" => Y <= "111";
33             when others      => Y <= "000";
34         end case;
35     end process;
36 end behavioral;
```

Listing 2.5: 带 GS 输出的 8-3 编码器 VHDL 实现

【GS 硬件】 一个 8 输入 OR 门——需要 8 个输入引脚。

【考试常考】 GS 信号的意义: 解决编码器输出的二义性问题。

2.5 变式 5：低电平有效的编码器

完整教学

【电路变化】输入和输出都是低电平有效（0 表示有效/选中，1 表示无效）。这在实际 IC（如 74LS148）中很常见——低电平有效更节省功耗，且与 TTL 电平兼容。

【逻辑推导】低电平有效时：

- 输入端： $\overline{I_0}, \overline{I_1}, \dots, \overline{I_7}$ （上划线表示低电平有效）
- 当 $\overline{I_k} = 0$ 且其他为 1 时 $\rightarrow$ 输出编码为 k
- 用 NAND 门替代 OR 门来实现（因为低有效的或 = 高有效的与非）

【VHDL——完整实现】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity encoder_active_low is
5      port (
6          I_n : in  std_logic_vector(7 downto 0);  -- 低有效
7          Y_n : out std_logic_vector(2 downto 0);  -- 低有效
8          GS_n: out std_logic                      -- 低有效: 0=有输入,
              1=无输入
9      );
10 end encoder_active_low;
11
12 architecture behavioral of encoder_active_low is
13 begin
14     process(I_n)
15     begin
16         GS_n <= '1';  -- 默认: 无有效输入
17         Y_n <= "111";
18
19         if I_n(7) = '0' then
20             Y_n <= "000"; GS_n <= '0';  -- 7的编码=111, 反相=000
21         elsif I_n(6) = '0' then
22             Y_n <= "001"; GS_n <= '0';  -- 6=110, 反相=001
23         elsif I_n(5) = '0' then
24             Y_n <= "010"; GS_n <= '0';
25         elsif I_n(4) = '0' then

```

```
26      Y_n <= "011"; GS_n <= '0';
27      elsif I_n(3) = '0' then
28          Y_n <= "100"; GS_n <= '0';
29      elsif I_n(2) = '0' then
30          Y_n <= "101"; GS_n <= '0';
31      elsif I_n(1) = '0' then
32          Y_n <= "110"; GS_n <= '0';
33      elsif I_n(0) = '0' then
34          Y_n <= "111"; GS_n <= '0';  -- 0的编码=000, 反相=111
35      end if;
36  end process;
37 end behavioral;
```

Listing 2.6: 低电平有效 8-3 优先级编码器 VHDL 实现

【关键理解】低有效编码：输入 0 有效 → 输出也是 0=111(即 7 的二进制反码)。高有效和低有效编码器的输出值互为反码。

【考试提示】看清题目要求：高有效还是低有效？74LS148 是低有效优先级编码器，这是常考器件。

2.6 编码器变式总结

编码器 5 大变式速查

变式	核心改动	VHDL 关键
16-4 编码	规模扩大	case 16 分支/for 循环
优先级编码	允许同时多输入	if-else 级联 (高 → 低)
带使能	增加 E 控制	if E='0' then Y<="000"
带 GS 输出	增加有效标志	GS<=not(I= 全 0)
低电平有效	0 有效 1 无效	判断 I(n)='0', 输出取反

第三章 3-8 译码器

3.1 第一步：解决什么问题

译码器是编码器的逆过程。

编码器：8 个输入 $\rightarrow$ 3 位输出（压缩）

译码器：3 位输入 $\rightarrow$ 8 个输出（展开）

想象一个场景：处理器用 3 根地址线选择 8 个外设中的一个。当处理器输出地址“101”时，第 5 号外设被选中（对应输出线变高），其余外设保持非选中状态。

译码器的本质：译码器 = 信号展开器： n 位输入 $\rightarrow 2^n$ 个输出

3-8 译码器：3 位输入 $\rightarrow$ 8 个输出，恰好一个输出被激活

3.2 第二步：输入输出关系

- **输入：** A_2, A_1, A_0 （3 位地址输入）
- **输出：** $Y_0, Y_1, \dots, Y_7$ （8 个输出，通常高电平有效）
- **功能：**当输入 $(A_2 A_1 A_0) = k$ 时， $Y_k = 1$ ，所有其他输出为 0

3.3 第三步：真值表

A_2 Y_0	A_1	A_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1
0	0	0	0	0	0	0	0	0	0
1									
0	0	1	0	0	0	0	0	0	1
0									
0	1	0	0	0	0	0	0	1	0
0									
0	1	1	0	0	0	0	1	0	0
0									
1	0	0	0	0	0	1	0	0	0
0									
1	0	1	0	0	1	0	0	0	0
0									
1	1	0	0	1	0	0	0	0	0
0									
1	1	1	1	0	0	0	0	0	0
0									

3.4 第四步：逻辑推导

译码器的每个输出对应输入的一个最小项：

$$Y_0 = \overline{A_2} \cdot \overline{A_1} \cdot \overline{A_0} = m_0 \quad (3.1)$$

$$Y_1 = \overline{A_2} \cdot \overline{A_1} \cdot A_0 = m_1 \quad (3.2)$$

$$Y_2 = \overline{A_2} \cdot A_1 \cdot \overline{A_0} = m_2 \quad (3.3)$$

$$Y_3 = \overline{A_2} \cdot A_1 \cdot A_0 = m_3 \quad (3.4)$$

$$Y_4 = A_2 \cdot \overline{A_1} \cdot \overline{A_0} = m_4 \quad (3.5)$$

$$Y_5 = A_2 \cdot \overline{A_1} \cdot A_0 = m_5 \quad (3.6)$$

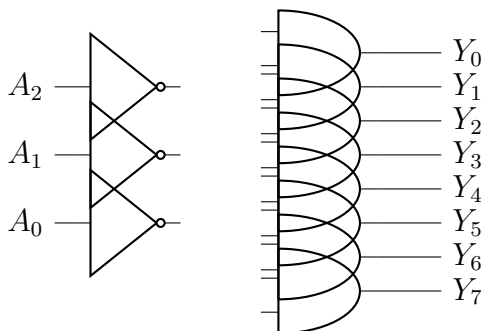
$$Y_6 = A_2 \cdot A_1 \cdot \overline{A_0} = m_6 \quad (3.7)$$

$$Y_7 = A_2 \cdot A_1 \cdot A_0 = m_7 \quad (3.8)$$

关键观察：每个输出是一个 3 输入 AND 门！对于给定输入组合，恰好一个 AND 门的所有输入都为 1（原变量或反变量），所以该 AND 门输出 1。

3.5 第五步：结构化设计

译码器内部结构 = 8 个 3 输入 AND 门 + 3 个反相器：



设计规律：每个 AND 门输入端根据该输出的二进制编号选择原变量或反变量。

- Y_5 (101): A_2 原变量 · $\overline{A_1}$ 反变量 · A_0 原变量
- Y_3 (011): $\overline{A_2}$ 反变量 · A_1 原变量 · A_0 原变量

一般规则：对于 Y_k ，如果 k 的第 i 位是 0，则取 $\overline{A_i}$ ；如果第 i 位是 1，则取 A_i 。

3.6 第六步：为什么这样设计

1. 为什么每个输出恰好是一个 AND 门？

AND 门的特性：**所有**输入必须为 1，输出才为 1。译码器的需求恰好是：对于输入组合 $(A_2A_1A_0)$ 恰好等于某个特定值时，对应的输出线才为 1。AND 门天然实现了这个“完全匹配”检测。

2. 为什么需要反相器？

因为输入信号可能是 0。例如：当 $(A_2A_1A_0) = 000$ 时，要检测“ $A_2 = 0$ 且 $A_1 = 0$ 且 $A_0 = 0$ ”，必须使用 $\overline{A_2} \cdot \overline{A_1} \cdot \overline{A_0}$ —— 需要三个反相器。

3. 译码器与编码器的对偶关系？

编码器：输入线号 → 二进制编码（多对 1 的 OR）译码器：二进制编码 → 输出线号（1 对多的 AND 展开）

这正是“编码”与“译码”的本质区别。

3.7 第七步：考试常见变式

1. 变式 1：低电平有效译码器（如 74LS138）

输入仍是高电平有效，但输出是低电平有效（选中时输出 0，未选中时输出 1）。此时将 AND 门换成 NAND 门即可，有时输出端还有小圆圈标记。

74LS138 还包含三个使能端： G_1 （高有效）， $\overline{G_{2A}}$ 和 $\overline{G_{2B}}$ （低有效）。这几乎是必考题！

2. 变式 2：用译码器实现任意逻辑函数

核心考点：译码器的输出是所有最小项。任何组合逻辑函数都可以表示为若干最小项的 OR。

例如：实现 $F(A, B, C) = \sum m(1, 3, 5, 7)$ （即 $F = C$ ）

解法：将 Y_1, Y_3, Y_5, Y_7 通过一个 OR 门连接即可。

3. 变式 3：用 3-8 译码器构建 4-16 译码器

使用两片 3-8 译码器 + 片选信号。高位地址作为片选，选择使用哪一个 3-8 译码器。

4. 变式 4：用译码器做地址译码

在存储器或外设选择中的应用。 n 位地址输入， 2^n 个片选信号输出。

5. 变式 5：显示译码器（BCD-7 段译码器）

将 4 位 BCD 码译为 7 段数码管的驱动信号。每个输出段也是一个最小项组合的逻辑函数。

3.8 第八步：VHDL 实现

3.8.1 普通 3-8 译码器（高电平有效）

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity decoder_3to8 is
5      port (
6          A : in  std_logic_vector(2 downto 0);
7          Y : out std_logic_vector(7 downto 0)
8      );
9  end decoder_3to8;
10
11 architecture behavioral of decoder_3to8 is
12 begin
13     process(A)

```

```

14     begin
15         case A is
16             when "000" => Y <= "00000001";
17             when "001" => Y <= "00000010";
18             when "010" => Y <= "00000100";
19             when "011" => Y <= "00001000";
20             when "100" => Y <= "00010000";
21             when "101" => Y <= "00100000";
22             when "110" => Y <= "01000000";
23             when "111" => Y <= "10000000";
24             when others => Y <= "00000000";
25         end case;
26     end process;
27 end behavioral;

```

Listing 3.1: 3-8 译码器 (case 实现)

3.8.2 74LS138 风格 (低电平有效 + 使能端)

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity decoder_74ls138 is
5      port (
6          A  : in  std_logic_vector(2 downto 0);
7          G1 : in  std_logic;
8          G2A_n : in  std_logic;
9          G2B_n : in  std_logic;
10         Y_n : out std_logic_vector(7 downto 0)  -- 低电平有效
11     );
12 end decoder_74ls138;
13
14 architecture behavioral of decoder_74ls138 is
15     signal enable : std_logic;
16 begin
17     enable <= G1 and (not G2A_n) and (not G2B_n);
18
19     process(A, enable)
20     begin
21         if enable = '0' then
22             Y_n <= "11111111";  -- 禁止: 所有输出无效
23         else
24             case A is
25                 when "000" => Y_n <= "11111110";

```

```

26     when "001" => Y_n <= "11111101";
27     when "010" => Y_n <= "11111011";
28     when "011" => Y_n <= "11110111";
29     when "100" => Y_n <= "11101111";
30     when "101" => Y_n <= "11011111";
31     when "110" => Y_n <= "10111111";
32     when "111" => Y_n <= "01111111";
33     when others => Y_n <= "11111111";
34   end case;
35 end if;
36 end process;
37 end behavioral;

```

Listing 3.2: 74LS138 风格译码器

3.9 第九步：Testbench 验证

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity decoder_3to8_tb is
6  end decoder_3to8_tb;
7
8  architecture test of decoder_3to8_tb is
9      signal A : std_logic_vector(2 downto 0);
10     signal Y : std_logic_vector(7 downto 0);
11
12     component decoder_3to8 is
13     port (
14         A : in  std_logic_vector(2 downto 0);
15         Y : out std_logic_vector(7 downto 0)
16     );
17     end component;
18 begin
19     uut: decoder_3to8 port map (A, Y);
20
21     stimulus: process
22     begin
23         for i in 0 to 7 loop
24             A <= std_logic_vector(to_unsigned(i, 3));
25             wait for 10 ns;
26         end loop;

```



```
27     wait;  
28     end process;  
29 end test;
```

Listing 3.3: 3-8 译码器 Testbench

3.10 第十步：如何快速解题

译码器快速解题法：

1. 写输出表达式： $Y_k = \prod A_i^{b_i}$ ，其中 b_i 是 k 的二进制第 i 位（0 取反，1 取原）
2. 用译码器实现逻辑函数：将函数表示为最小项之和，将对应输出端用 OR 门连接
3. 级联译码器：高位地址 → 片选信号，低位地址 → 各片译码器的输入
4. 有效电平：高有效用 AND，低有效用 NAND

秒杀口诀：每个输出 = 一个最小项 = 一个 AND 门（输入原/反变量取决于编号）。

最常考：用 3-8 译码器 + OR 门实现任意 3 变量逻辑函数。

例子：实现 $F(A, B, C) = \sum m(0, 2, 4, 6)$ 解： $F = Y_0 + Y_2 + Y_4 + Y_6$ （将四个输出通过 OR 门连接）

3.11 变式详解

第四章 3-8 译码器——5 大变式详解

4.1 变式 1：74LS138 风格译码器（低有效输出 + 三使能端）——必考！

完整教学

【电路】74LS138 是考试中最常出现的译码器芯片。特点：

- 3 位地址输入 A2,A1,A0
- 8 个输出 Y0-Y7，全部低电平有效（选中 =0，未选中 =1）
- 3 个使能端：G1(高有效), G2A(低有效), G2B(低有效)
- 只有三个使能端同时有效（G1=1, G2A=0, G2B=0）时，译码器才工作

【使能逻辑】 $enable = G1 \cdot \overline{G2A} \cdot \overline{G2B}$

【VHDL】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity decoder_74ls138 is
5     port (
6         A      : in  std_logic_vector(2 downto 0);
7         G1      : in  std_logic;
8         G2A_n : in  std_logic;  -- 低有效
9         G2B_n : in  std_logic;  -- 低有效
10        Y_n    : out std_logic_vector(7 downto 0)  -- 低有效
11    );
12 end decoder_74ls138;
13
14 architecture behavioral of decoder_74ls138 is
15     signal enable : std_logic;
```

```

16 begin
17     enable <= G1 and (not G2A_n) and (not G2B_n);
18
19     process(A, enable)
20     begin
21         if enable = '0' then
22             Y_n <= "11111111"; -- 全高=全无效
23         else
24             case A is
25                 when "000" => Y_n <= "11111110"; -- Y0=0
26                 when "001" => Y_n <= "11111101"; -- Y1=0
27                 when "010" => Y_n <= "11111011";
28                 when "011" => Y_n <= "11110111";
29                 when "100" => Y_n <= "11101111";
30                 when "101" => Y_n <= "11011111";
31                 when "110" => Y_n <= "10111111";
32                 when "111" => Y_n <= "01111111";
33                 when others => Y_n <= "11111111";
34             end case;
35         end if;
36     end process;
37 end behavioral;

```

【为什么用 3 个使能端】多个使能端方便级联。G2A 和 G2B 是低有效，G1 是高有效——这种混合设计使得可以用不同信号控制，增加灵活性。

【常考陷阱】

- 看清输出是低有效还是高有效——74LS138 输出 0 表示选中
- 三个使能端必须同时满足条件译码器才工作
- 使能无效时所有输出为 1（全高 = 全无效）

4.2 变式 2：用译码器实现任意逻辑函数（必考！）

完整教学

【核心原理】译码器的每个输出 = 一个最小项。任何组合逻辑函数 = 若干最小项之和（OR）。

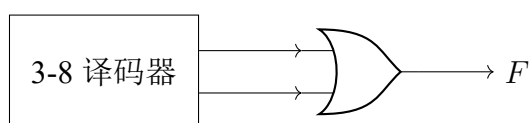
【通用公式】 $F(A, B, C) = \sum m(\dots) = Y_{i1} + Y_{i2} + Y_{i3} + \dots$

【例题】 用 3-8 译码器 + 一个 OR 门实现 $F(A, B, C) = \sum m(1, 3, 5, 7)$ 。

【解法】

1. 译码器产生 8 个最小项: $Y_0=m_0, Y_1=m_1, \dots, Y_7=m_7$
2. $F = m_1 + m_3 + m_5 + m_7 = Y_1 + Y_3 + Y_5 + Y_7$
3. 用 1 个 4 输入 OR 门连接 Y_1, Y_3, Y_5, Y_7

【电路结构】



A2A1A0 = 变量 A,B,C, OR 门输入 = Y_1, Y_3, Y_5, Y_7

【VHDL】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity func_from_decoder is
5      port (
6          ABC : in  std_logic_vector(2 downto 0);
7          F   : out std_logic
8      );
9  end func_from_decoder;
10
11 architecture structural of func_from_decoder is
12     signal Y : std_logic_vector(7 downto 0);
13
14     component decoder_3to8 is
15         port (
16             A : in  std_logic_vector(2 downto 0);
17             Y : out std_logic_vector(7 downto 0)
18         );
19     end component;
20 begin
21     dec: decoder_3to8 port map(A => ABC, Y => Y);
22     F <= Y(1) or Y(3) or Y(5) or Y(7);
23 end structural;

```

【推广：实现任意 3 变量函数】

函数	连接
$F = m_0 + m_2 + m_4 + m_6$	$Y_0 + Y_2 + Y_4 + Y_6$
$F = m_0 + m_1 + m_2 + m_3$	$Y_0 + Y_1 + Y_2 + Y_3$
$F = A$ (即 $m_4 + m_5 + m_6 + m_7$)	$Y_4 + Y_5 + Y_6 + Y_7$
$F = \overline{A}$	$Y_0 + Y_1 + Y_2 + Y_3$

【如果函数用 NAND 实现】用 NAND 门替代 OR 门（德摩根定理）。此时输出是反相的，但功能等价。

考试聚焦

必考！用 3-8 译码器 + 门电路实现任意 3 变量逻辑函数。步骤：(1) 将函数写成最小项之和 (2) 对应 Y 输出用 OR 门连接

4.3 变式 3：用 3-8 译码器构建 4-16 译码器（片选级联）

完整教学

【问题】只有 3-8 译码器，如何得到 4 位地址 → 16 个输出的 4-16 译码器？

【方案】两片 3-8 译码器 + 最高位地址 A3 做片选。

【级联逻辑】

- 片 0（低位）：处理 A3=0 时，地址 0000-0111（译码器输出 Y0-Y7 对应系统 Y0-Y7）
- 片 1（高位）：处理 A3=1 时，地址 1000-1111（译码器输出 Y0-Y7 对应系统 Y8-Y15）
- A3=0 → 片 0 使能，片 1 禁止
- A3=1 → 片 1 使能，片 0 禁止

【VHDL】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
```

```
4 entity decoder_4to16 is
5   port (A : in std_logic_vector(3 downto 0);
6         Y : out std_logic_vector(15 downto 0));
7 end decoder_4to16;
8
9 architecture structural of decoder_4to16 is
10  signal Y_low, Y_high : std_logic_vector(7 downto 0);
11  component decoder_3to8 is
12    port (A : in std_logic_vector(2 downto 0);
13          E : in std_logic; Y : out std_logic_vector(7 downto 0));
14  end component;
15 begin
16  -- 片0: A3=0时使能
17  dec_low: decoder_3to8 port map(
18    A => A(2 downto 0), E => not A(3), Y => Y_low);
19
20  -- 片1: A3=1时使能
21  dec_high: decoder_3to8 port map(
22    A => A(2 downto 0), E => A(3), Y => Y_high);
23
24  Y <= Y_high & Y_low; -- 拼接为16位输出
25 end structural;
```

【通用级联公式】用 2^k 个 n - 2^n 译码器构建 $(n + k)$ - 2^{n+k} 译码器。高 k 位地址 → 片选，低 n 位地址 → 各片输入。

【常见考题】用两片 74LS138 构建 4-16 译码器。利用使能端做片选。

4.4 变式 4：用译码器做地址译码

完整教学

【应用场景】CPU 有 16 位地址总线，但连接了多个外设（RAM、ROM、IO 端口等）。用译码器将地址空间划分为不同区域，产生各外设的片选信号。

【例题】用 3-8 译码器产生 8 个外设的片选信号。地址 A15-A13 做译码输入。

【电路】

A15A14A13	选中输出	外设
000	Y0	RAM1(0x0000-0x1FFF)
001	Y1	RAM2(0x2000-0x3FFF)
010	Y2	ROM (0x4000-0x5FFF)
011	Y3	IO1 (0x6000-0x7FFF)
100	Y4	IO2 (0x8000-0x9FFF)

【核心思想】高位地址 → 译码 → 片选。低位地址 → 片内寻址。

4.5 变式 5：显示译码器（BCD→7 段）

完整教学

【问题】3-8 译码器每次只选中 1 个输出。但 7 段数码管需要同时点亮多个段（如数字”8” 需要 7 段全亮）。

【区别】

- 3-8 译码器： n 输入 → 2^n 输出，输出是互斥的（恰好一个为 1）
- BCD-7 段译码器：4 输入 → 7 输出，输出不互斥（可以同时多个为 1）

【详见第三章 BCD 译码器】BCD-7 段译码器本质上是 7 个独立的组合逻辑函数，不是简单的”最小项展开”。

考试聚焦

关键区别：

- 3-8 译码器：1-of-N 选择（互斥输出）→ 用 AND 门
- BCD-7 段：任意多段同时亮（非互斥）→ 用 ROM/查找表

不搞混这两种”译码器” 是考试的基本要求。

4.6 译码器变式总结

译码器 5 大变式速查

变式	核心考点	常考题型
74LS138 风格	低有效 +3 使能端	写 VHDL, 判断使能条件
译码器 +OR≈ 任意函数	最小项之和	给定函数 → 画电路
级联构建更大译码器	高位地址做片选	74LS138×2→4-16
地址译码	地址空间划分	存储器片选设计
vs BCD-7 段	互斥 vs 非互斥输出	概念辨析

--

第五章 BCD 译码器

5.1 第一步：解决什么问题

计算机内部使用二进制运算，但人类需要看到十进制数字。

问题：FPGA 内部计算结果是 4 位二进制（0000 到 1111），如何在 7 段数码管上显示为人类可读的十进制数字（0 到 9）？

方案：将 4 位 BCD 码（Binary-Coded Decimal）转换为 7 段数码管的驱动信号。

BCD 码：用 4 位二进制数表示一个十进制数字。

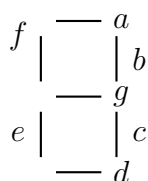
- $0 \rightarrow 0000$
- $1 \rightarrow 0001$
- $2 \rightarrow 0010$
- ...
- $9 \rightarrow 1001$

注意：1010 到 1111（10 到 15）在 BCD 中是非法的，不应出现。

5.2 第二步：输入输出关系

- **输入：** A_3, A_2, A_1, A_0 （4 位 BCD 码，表示 0-9）
- **输出：** a, b, c, d, e, f, g （7 段，每一段控制数码管的一段 LED）

7 段数码管的段命名：



5.3 第三步：真值表

A_3	A_2	A_1	A_0	a	b	c	d	e	f	g	显示
0	0	0	0	1	1	1	1	1	1	0	0
0	0	0	1	0	1	1	0	0	0	0	1
0	0	1	0	1	1	0	1	1	0	1	2
0	0	1	1	1	1	1	1	0	0	1	3
0	1	0	0	0	1	1	0	0	1	1	4
0	1	0	1	1	0	1	1	0	1	1	5
0	1	1	0	1	0	1	1	1	1	1	6
0	1	1	1	1	1	1	0	0	0	0	7
1	0	0	0	1	1	1	1	1	1	1	8
1	0	0	1	1	1	1	1	0	1	1	9

对于 1010 到 1111 (10-15)，输出通常全部熄灭（全 0）或显示特定模式（取决于设计）。

5.4 第四步：逻辑推导

每一段都是一个 4 变量组合逻辑函数。以 a 段为例：

$a = 1$ 当输入为：0(0000), 2(0010), 3(0011), 5(0101), 6(0110), 7(0111), 8(1000), 9(1001)

即： $a = \sum m(0, 2, 3, 5, 6, 7, 8, 9)$

卡诺图化简后（此处省略卡诺图步骤，这是我们有意跳过的部分），得到：

$$a = A_3 + A_1 + \overline{A_2} \cdot \overline{A_0} + A_2 \cdot A_0 \quad (5.1)$$

$$b = \overline{A_2} + \overline{A_1} \cdot \overline{A_0} + A_1 \cdot A_0 \quad (5.2)$$

$$c = \overline{A_1} + A_0 + A_2 \quad (5.3)$$

$$d = \overline{A_2} \cdot \overline{A_0} + A_1 \cdot \overline{A_0} + \overline{A_2} \cdot A_1 + A_2 \cdot \overline{A_1} \cdot A_0 \quad (5.4)$$

$$e = \overline{A_2} \cdot \overline{A_0} + A_1 \cdot \overline{A_0} \quad (5.5)$$

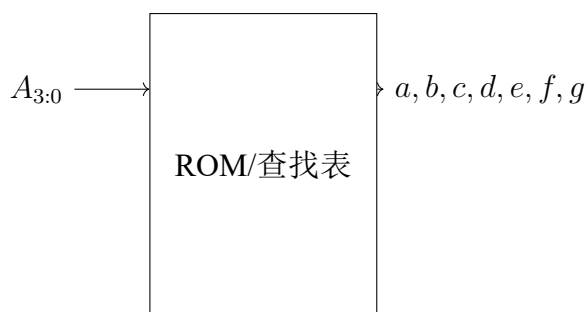
$$f = A_3 + \overline{A_1} \cdot \overline{A_0} + A_2 \cdot \overline{A_1} + A_2 \cdot \overline{A_0} \quad (5.6)$$

$$g = A_3 + \overline{A_2} \cdot A_1 + A_1 \cdot \overline{A_0} + A_2 \cdot \overline{A_1} \quad (5.7)$$

但在考试中，你不会也不需要手算这些表达式。考试更可能考的是理解 BCD 译码器的功能和使用方法。

5.5 第五步：结构化设计

BCD-7 段译码器 = 7 个 4 变量组合逻辑函数，每个函数通过 ROM/查找表实现：



输入：4 位 BCD → 查表 → 输出：7 段控制信号

5.6 第六步：为什么这样设计

1. 为什么用查找表而非单独的门电路？

7 段译码器涉及 7 个 4 变量函数。如果每个都用门电路实现，需要大量 AND 门和 OR 门。使用 ROM（ $16 \times 7 = 112$ bits）更简洁，这也是 FPGA 中 LUT（查找表）的实际实现方式。

2. 为什么 1010-1111 是非法输入？

BCD 码的定义：4 位二进制只使用前 10 个编码（0000-1001）。后 6 个编码（1010-1111）不表示任何十进制数字，是非法 BCD 码。好的设计应该对所有输入都定义行为（全部熄灭或显示异常标记）。

3. 与普通 3-8 译码器的本质区别？

3-8 译码器：每个输出是一个最小项（3 输入 AND）BCD-7 段译码器：每个输出是多个最小项之和（4 输入的组合逻辑函数）

所以 BCD-7 段译码器本质上是 7 个独立的组合逻辑函数。

5.7 第七步：考试常见变式

1. 变式 1：多位 BCD 显示

多个 BCD 译码器 + 多个 7 段数码管，实现多位数显示。通常配合动态扫描技术（快速轮流点亮各数码管，利用人眼视觉暂留）。

2. 变式 2：带锁存功能的 BCD 译码器

输入锁存：在 LE（Latch Enable）信号控制下锁存当前 BCD 输入，数码管保持显示不变。这在需要稳定显示时非常有用。

3. 变式 3：共阳极 vs 共阴极数码管

共阳极：所有 LED 阳极接一起（接 VCC），段驱动需要低电平点亮（输出 0 = 亮）

共阴极：所有 LED 阴极接一起（接 GND），段驱动需要高电平点亮（输出 1 = 亮）

两种类型的输出值恰好互为反码。

4. 变式 4：计数 + 显示系统

这是考试常见综合题：计数器产生 BCD 码 → BCD 译码器 → 7 段数码管显示。
考察对整个数字系统流程的理解。

5. 变式 5：16 进制显示译码器

输入 0-15（全 4 位二进制），输出 0-9 + A-F。比 BCD 译码器多 6 个有效输入状态。

5.8 第八步：VHDL 实现

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity bcd_7seg is
5      port (
6          bcd : in  std_logic_vector(3 downto 0);
7          seg : out std_logic_vector(6 downto 0)  -- a,b,c,d,e,f,g
8      );
9  end bcd_7seg;
10
11 architecture behavioral of bcd_7seg is
12 begin
13     process(bcd)
14     begin
15         case bcd is
16             when "0000" => seg <= "1111110";  -- 0
17             when "0001" => seg <= "0110000";  -- 1
18             when "0010" => seg <= "1101101";  -- 2
19             when "0011" => seg <= "1111001";  -- 3
20             when "0100" => seg <= "0110011";  -- 4
21             when "0101" => seg <= "1011011";  -- 5
22             when "0110" => seg <= "1011111";  -- 6
23             when "0111" => seg <= "1110000";  -- 7
24             when "1000" => seg <= "1111111";  -- 8
25             when "1001" => seg <= "1111011";  -- 9
26             when others => seg <= "0000000";  -- 全灭
```

```
27     end case;  
28   end process;  
29 end behavioral;
```

Listing 5.1: BCD-7 段译码器 (case 版本)

5.9 第九步: Testbench 验证

```
1  library ieee;  
2  use ieee.std_logic_1164.all;  
3  use ieee.numeric_std.all;  
4  
5  entity bcd_7seg_tb is  
6  end bcd_7seg_tb;  
7  
8  architecture test of bcd_7seg_tb is  
9    signal bcd : std_logic_vector(3 downto 0);  
10   signal seg : std_logic_vector(6 downto 0);  
11  
12   component bcd_7seg is  
13     port (  
14       bcd : in  std_logic_vector(3 downto 0);  
15       seg : out std_logic_vector(6 downto 0)  
16     );  
17   end component;  
18   begin  
19     uut: bcd_7seg port map (bcd, seg);  
20  
21     stimulus: process  
22     begin  
23       for i in 0 to 15 loop  
24         bcd <= std_logic_vector(to_unsigned(i, 4));  
25         wait for 10 ns;  
26       end loop;  
27       wait;  
28     end process;  
29   end test;
```

Listing 5.2: BCD-7 段译码器 Testbench

5.10 第十步：如何快速解题

BCD 译码器快速解题法：

1. **case 实现是标准答案**：在 VHDL 中用 case 列举 10 个有效输入，不需要推导门级表达式
2. **注意有效电平**：共阳极（0 亮）vs 共阴极（1 亮），输出值互为反码
3. **非法输入处理**：1010-1111 用 when others，全灭或显示特定标记
4. **综合题**：计数器 + 译码器 + 数码管 = 经典考试题

5.11 变式详解

第六章 BCD 译码器——5 大变式详解

6.1 变式 1：多位 BCD 动态扫描显示

完整教学

【问题】4 位数码管需要 4 个 BCD-7 段译码器吗？如果每个译码器单独驱动 → 需要 $4 \times 7 = 28$ 个 IO 口。

【方案】动态扫描：1 个译码器驱动所有数码管的段，快速轮流点亮各数码管 ($>50\text{Hz}$)，利用人眼视觉暂留。

【电路】计数器 → 2-4 译码器 → 轮流选通 4 个数码管的共阴/共阳极。BCD 译码器输出段码到所有数码管。

【VHDL 核心逻辑】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity dynamic_scan is
6     port (
7         clk          : in  std_logic;
8         digit0       : in  std_logic_vector(6 downto 0);
9         digit1       : in  std_logic_vector(6 downto 0);
10        digit2       : in  std_logic_vector(6 downto 0);
11        digit3       : in  std_logic_vector(6 downto 0);
12        digit_sel    : out std_logic_vector(3 downto 0);
13        seg_data     : out std_logic_vector(6 downto 0)
14    );
15 end dynamic_scan;
16
17 architecture rtl of dynamic_scan is
18     signal scan_cnt : unsigned(1 downto 0) := "00";
19 begin
20     process(clk) -- 扫描时钟 (通常 >200Hz)
21     begin
```

```

22     if rising_edge(clk) then
23         scan_cnt <= scan_cnt + 1;
24         case scan_cnt is
25             when "00" => digit_sel <= "0001"; seg_data <= digit0;
26             when "01" => digit_sel <= "0010"; seg_data <= digit1;
27             when "10" => digit_sel <= "0100"; seg_data <= digit2;
28             when "11" => digit_sel <= "1000"; seg_data <= digit3;
29         end case;
30     end if;
31 end process;
32 end rtl;

```

【常见考题】设计 4 位动态扫描显示电路。

6.2 变式 2: 带锁存的 BCD 译码器

完整教学

【问题】BCD 译码器输入随时钟快速变化 → 数码管闪烁。需要“定格”显示。

【方案】在译码器前加寄存器。LE=1 时锁存当前 BCD 值，LE=0 时保持上次锁存值不变。

【VHDL】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity latched_bcd_decoder is
5      port (
6          clk      : in  std_logic;
7          LE       : in  std_logic;
8          bcd_input : in  std_logic_vector(3 downto 0);
9          seg      : out std_logic_vector(6 downto 0)
10     );
11 end latched_bcd_decoder;
12
13 architecture rtl of latched_bcd_decoder is
14     signal latched_bcd : std_logic_vector(3 downto 0);
15
16     function bcd_to_7seg(bcd : std_logic_vector(3 downto 0))
17         return std_logic_vector is
18     begin

```



```

19     case bcd is
20         when "0000" => return "1111110";
21         when "0001" => return "0110000";
22         when "0010" => return "1101101";
23         when "0011" => return "1111001";
24         when "0100" => return "0110011";
25         when "0101" => return "1011011";
26         when "0110" => return "1011111";
27         when "0111" => return "1110000";
28         when "1000" => return "1111111";
29         when "1001" => return "1111011";
30         when others => return "0000000";
31     end case;
32 end function;
33 begin
34     process(clk)
35     begin
36         if rising_edge(clk) then
37             if LE = '1' then
38                 latched_bcd <= bcd_input;  -- 锁存
39             end if;
40         end if;
41     end process;
42     seg <= bcd_to_7seg(latched_bcd);  -- 译码锁存后的值
43 end rtl;

```

【设计思想】锁存 = 寄存器 = 1 个时钟沿保存数据。本质 = 在数据通路上插入一级寄存器。

6.3 变式 3：共阳极 vs 共阴极数码管

完整教学

【共阳极】所有 LED 阳极接 VCC。段驱动需低电平 (0) 点亮 LED。seg='0'= 亮。【共阴极】所有 LED 阴极接 GND。段驱动需高电平 (1) 点亮 LED。seg='1'= 亮。

【输出值关系】 $seg_{\text{共阳}} = \text{not } seg_{\text{共阴}}$ （互为反码）

【VHDL 适配】

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3

```

```
4 entity bcd_7seg_dual is
5   port (
6     bcd      : in  std_logic_vector(3 downto 0);
7     seg_cath : out std_logic_vector(6 downto 0); -- 共阴极(1亮)
8     seg_anod : out std_logic_vector(6 downto 0)  -- 共阳极(0亮)
9   );
10 end bcd_7seg_dual;
11
12 architecture behavioral of bcd_7seg_dual is
13 begin
14   process(bcd)
15   begin
16     case bcd is
17       -- 共阴极(1亮)          -- 共阳极(0亮)
18       when "0000" => seg_cath <= "1111110"; seg_anod <= "0000001";
19         -- 0
20       when "0001" => seg_cath <= "0110000"; seg_anod <= "1001111";
21         -- 1
22       when "0010" => seg_cath <= "1101101"; seg_anod <= "0010010";
23         -- 2
24       when "0011" => seg_cath <= "1111001"; seg_anod <= "0000110";
25         -- 3
26       when "0100" => seg_cath <= "0110011"; seg_anod <= "1001100";
27         -- 4
28       when "0101" => seg_cath <= "1011011"; seg_anod <= "0100100";
29         -- 5
30       when "0110" => seg_cath <= "1011111"; seg_anod <= "0100000";
31         -- 6
32       when "0111" => seg_cath <= "1110000"; seg_anod <= "0001111";
33         -- 7
34       when "1000" => seg_cath <= "1111111"; seg_anod <= "0000000";
35         -- 8
36       when "1001" => seg_cath <= "1111011"; seg_anod <= "0000100";
37         -- 9
38       when others => seg_cath <= "0000000"; seg_anod <= "1111111";
39     end case;
40   end process;
41 end behavioral;
```

【考试陷阱】 题目未明确说明共阴/共阳时，看段码值判断：1111110(共阴 0) vs 0000001(共阳 0)。

6.4 变式 4：计数器 + 译码器 + 数码管——综合题

完整教学

【经典考题】设计 0-9 计数器 +BCD-7 段译码器 + 数码管显示。

【完整 VHDL】

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity counter_display is
6      port (clk, rst_n : in std_logic;
7            seg : out std_logic_vector(6 downto 0));
8  end counter_display;
9
10 architecture rtl of counter_display is
11     signal cnt : std_logic_vector(3 downto 0);
12     function bcd7seg(bcd : std_logic_vector(3 downto 0))
13         return std_logic_vector is
14     begin
15         case bcd is
16             when "0000" => return "1111110";
17             when "0001" => return "0110000";
18             when "0010" => return "1101101";
19             when "0011" => return "1111001";
20             when "0100" => return "0110011";
21             when "0101" => return "1011011";
22             when "0110" => return "1011111";
23             when "0111" => return "1110000";
24             when "1000" => return "1111111";
25             when "1001" => return "1111011";
26             when others => return "0000000";
27         end case;
28     end function;
29 begin
30     process(clk, rst_n)
31     begin
32         if rst_n='0' then cnt <= "0000";
33         elsif rising_edge(clk) then
34             if cnt = "1001" then cnt <= "0000";
35             else cnt <= cnt + 1;
36             end if;
37         end if;
```

```
38     end process;
39     seg <= bcd7seg(cnt);
40 end rtl;
```

【考点】两个模块整合：计数器 + 译码器。

6.5 变式 5: 16 进制显示译码器 (0-F)

完整教学

【问题】BCD 译码器只显示 0-9。如何显示 0-F (全 4 位二进制)?

【方案】扩展 case 覆盖 1010-1111, 定义 A-F 的段码。

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity hex_7seg is
5      port (
6          hex : in  std_logic_vector(3 downto 0);
7          seg : out std_logic_vector(6 downto 0)
8      );
9  end hex_7seg;
10
11 architecture behavioral of hex_7seg is
12 begin
13     process(hex)
14     begin
15         case hex is
16             when "0000" => seg <= "1111110";  -- 0
17             when "0001" => seg <= "0110000";  -- 1
18             when "0010" => seg <= "1101101";  -- 2
19             when "0011" => seg <= "1111001";  -- 3
20             when "0100" => seg <= "0110011";  -- 4
21             when "0101" => seg <= "1011011";  -- 5
22             when "0110" => seg <= "1011111";  -- 6
23             when "0111" => seg <= "1110000";  -- 7
24             when "1000" => seg <= "1111111";  -- 8
25             when "1001" => seg <= "1111011";  -- 9
26             -- 扩展支持 A-F
27             when "1010" => seg <= "1110111";  -- A
28             when "1011" => seg <= "0011111";  -- b
```

```
29     when "1100" => seg <= "1001110";  -- C
30     when "1101" => seg <= "0111101";  -- d
31     when "1110" => seg <= "1001111";  -- E
32     when "1111" => seg <= "1000111";  -- F
33     when others => seg <= "0000000";
34   end case;
35 end process;
36 end behavioral;
```

【考试】有时问 A-F 的段码值，需要记住典型表示法。

第七章 4 选 1 数据选择器 (MUX)

7.1 第一步：解决什么问题

想象一个场景：你需要从 4 个数据源中选择 1 个传给后续电路。

- 不是同时传 4 个——那需要 4 条数据通路
- 而是：用 2 根选择线指定“要哪一个”，1 根数据线输出

MUX 的本质：数据选择器 (Multiplexer, MUX) = 一个受控的多路开关
 2^n 个数据输入 $\rightarrow n$ 条选择线 $\rightarrow 1$ 个输出
选择线决定“哪一路数据通向输出”

7.2 第二步：输入输出关系

4 选 1 MUX:

- 数据输入: D_0, D_1, D_2, D_3
- 选择输入: S_1, S_0 (2 位, 选择哪个数据输入)
- 输出: Y
- 功能: $Y = D_{(S_1 S_0)}$

7.3 第三步：真值表

S_1	S_0	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

7.4 第四步：逻辑推导

从真值表可以写出：

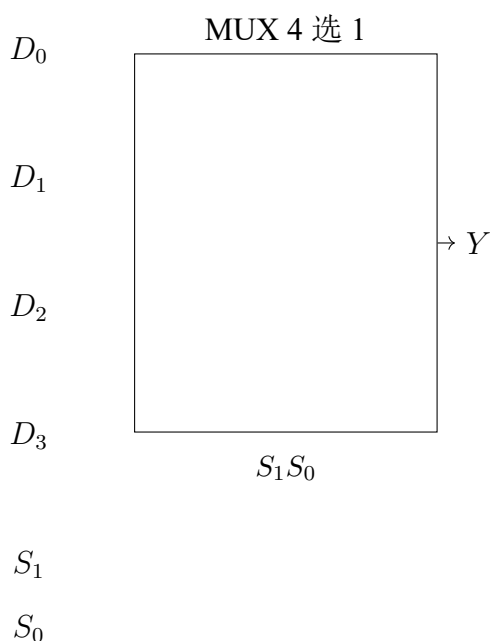
$$Y = \overline{S_1} \cdot \overline{S_0} \cdot D_0 + \overline{S_1} \cdot S_0 \cdot D_1 + S_1 \cdot \overline{S_0} \cdot D_2 + S_1 \cdot S_0 \cdot D_3 \quad (7.1)$$

关键观察： MUX 的每个选择组合对应一个乘积项。

- 选择线产生最小项 ($\overline{S_1} \cdot \overline{S_0}$, $\overline{S_1} \cdot S_0$ 等)
- 每个数据输入与其对应的最小项 AND
- 所有结果 OR 起来

7.5 第五步：结构化设计

4 选 1 MUX 内部结构 = 4 个 3 输入 AND 门 + 1 个 4 输入 OR 门 + 2 个反相器：



7.6 第六步：为什么这样设计

1. 为什么 MUX 是”万能”的组合逻辑？

因为 MUX 输出 = $\sum$ (选择最小项 · 数据输入)。

如果把数据输入当作”该最小项是否参与”的控制：

- $D_k = 1$ ：该最小项参与 (函数包含此最小项)

- $D_k = 0$: 该最小项不参与

所以：任何 n 变量逻辑函数都可以用一个 2^n 选 1 MUX 实现！

只需将函数真值表的输出列接到 MUX 的数据输入端。

2. 为什么 MUX 能节省资源？

将”多路数据”变成”一路分时复用”——不需要为每路数据各建一条独立通路。这是时分复用的基本思想。

3. MUX 与译码器的关系？

译码器产生所有 2^n 个最小项 (n 输入 $\rightarrow 2^n$ 输出)，MUX 选择其中一个输出。实际上，MUX = 译码器 + AND-OR 结构。

7.7 第七步：考试常见变式

1. 变式 1：用 MUX 实现任意逻辑函数（必考！）

方法：将函数的变量接选择线，真值表的输出值接数据输入。

例：实现 $F(A, B, C) = \sum m(0, 1, 4, 6, 7)$

用 8 选 1 MUX: $D_0 = 1, D_1 = 1, D_2 = 0, D_3 = 0, D_4 = 1, D_5 = 0, D_6 = 1, D_7 = 1$

降维技巧：用 2^{n-1} 选 1 MUX 实现 n 变量函数

选择线只接 $n - 1$ 个变量，第 n 个变量（或其反变量）接到某些数据输入端。

例：用 4 选 1 MUX 实现 3 变量函数 $F(A, B, C)$

将 A, B 接选择线 S_1, S_0 ，C（或其反）接到某些数据输入端。

2. 变式 2：双 4 选 1 MUX（8 选 1 功能）

两片 4 选 1 MUX + 一片 2 选 1 MUX = 8 选 1 MUX。

3. 变式 3：带使能端的 MUX

使能端 $E=0$ 时，输出固定为 0（或高阻态），无论选择线是什么。

4. 变式 4：MUX 扩展

n 选 1 MUX 可以通过级联构建更大的 MUX。例如：4 片 4 选 1 MUX + 1 片 4 选 1 MUX = 16 选 1 MUX。

7.8 第八步: VHDL 实现

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity mux4to1 is
5      port (
6          D : in  std_logic_vector(3 downto 0);
7          S : in  std_logic_vector(1 downto 0);
8          Y : out std_logic
9      );
10 end mux4to1;
11
12 -- 方式1: with-select (最简洁)
13 architecture with_select of mux4to1 is
14 begin
15     with S select
16         Y <= D(0) when "00",
17             D(1) when "01",
18             D(2) when "10",
19             D(3) when "11",
20             '0'  when others;
21 end with_select;
22
23 -- 方式2: when-else (条件信号赋值)
24 architecture when_else of mux4to1 is
25 begin
26     Y <= D(0) when S = "00" else
27         D(1) when S = "01" else
28         D(2) when S = "10" else
29         D(3) when S = "11" else
30         '0';
31 end when_else;
32
33 -- 方式3: process + case
34 architecture proc_case of mux4to1 is
35 begin
36     process(S, D)
37     begin
38         case S is
39             when "00" => Y <= D(0);
40             when "01" => Y <= D(1);
41             when "10" => Y <= D(2);
42             when "11" => Y <= D(3);
```

```

43     when others => Y <= '0';
44     end case;
45     end process;
46 end proc_case;

```

Listing 7.1: 4 选 1 MUX (多种实现方式)

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity mux4to1 is
5      port (
6          D : in  std_logic_vector(3 downto 0);
7          S : in  std_logic_vector(1 downto 0);
8          Y : out std_logic
9      );
10 end mux4to1;
11
12 architecture behavioral of mux4to1 is
13 begin
14     with S select
15         Y <= D(0) when "00",
16             D(1) when "01",
17             D(2) when "10",
18             D(3) when "11",
19             '0'  when others;
20 end behavioral;

```

Listing 7.2: 完整 4 选 1 MUX 实体

7.9 第九步：Testbench 验证

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity mux4to1_tb is
5      end mux4to1_tb;
6
7  architecture test of mux4to1_tb is
8      signal D : std_logic_vector(3 downto 0);
9      signal S : std_logic_vector(1 downto 0);
10     signal Y : std_logic;
11

```

```

12  component mux4to1 is
13      port (
14          D : in  std_logic_vector(3 downto 0);
15          S : in  std_logic_vector(1 downto 0);
16          Y : out std_logic
17      );
18  end component;
19  begin
20      uut: mux4to1 port map (D, S, Y);
21
22      stimulus: process
23      begin
24          D <= "1010"; -- 固定数据输入
25          S <= "00"; wait for 10 ns; -- Y = D0 = 0
26          S <= "01"; wait for 10 ns; -- Y = D1 = 1
27          S <= "10"; wait for 10 ns; -- Y = D2 = 0
28          S <= "11"; wait for 10 ns; -- Y = D3 = 1
29          wait;
30      end process;
31  end test;

```

Listing 7.3: 4 选 1 MUX Testbench

7.10 第十步：如何快速解题

MUX 快速解题法：

1. 用 MUX 实现逻辑函数：真值表的输出列 = MUX 的数据输入列（这是最快的做法）
2. 降维法： n 变量函数 $\rightarrow$ 用 2^{n-1} 选 1 MUX。剩余的 1 个变量（或其反）接数据输入
3. 级联扩展：小 MUX + 小 MUX = 大 MUX
4. VHDL: with-select 是最简洁的 MUX 写法，综合结果就是 MUX 硬件

秒杀必考题：用 MUX 实现逻辑函数

题：用 8 选 1 MUX 实现 $F(A, B, C) = \sum m(0, 3, 5, 6)$

解： $D_0 = 1, D_1 = 0, D_2 = 0, D_3 = 1, D_4 = 0, D_5 = 1, D_6 = 1, D_7 = 0$

7.11 变式详解

第八章 4 选 1 MUX——4 大变式详解

8.1 变式 1：用 MUX 实现任意逻辑函数（必考！）

完整教学

【核心原理】MUX 的通用表达式： $Y = \overline{S_1} \cdot \overline{S_0} \cdot D_0 + \overline{S_1} \cdot S_0 \cdot D_1 + S_1 \cdot \overline{S_0} \cdot D_2 + S_1 \cdot S_0 \cdot D_3$

如果把 S_1, S_0 接函数变量， D_k 接“该最小项是否参与=1”： D_k 接 1→该最小项参与函数。 D_k 接 0→不参与。

【结论】任何 n 变量逻辑函数 = 一个 2^n 选 1 MUX

【例题 1】用 8 选 1 MUX 实现 $F(A, B, C) = \sum m(0, 3, 5, 6)$

解： $D_0 = 1, D_1 = 0, D_2 = 0, D_3 = 1, D_4 = 0, D_5 = 1, D_6 = 1, D_7 = 0$

将 m_0 - m_7 对应的 D 分别接 1 或 0。选择线 S_2, S_1, S_0 接 A, B, C 。

【例题 2：降维法】用 4 选 1 MUX 实现 3 变量函数 $F(A, B, C) = \sum m(0, 1, 4, 6, 7)$
 $2^{3-1} = 4$ 选 1 MUX，变量 A, B 接 S_1, S_0 , C 用于产生 D_k 。

AB	m(C=0)	m(C=1)	D 输入	逻辑
00	$m_0=1$	$m_1=1$	D0	1(与 C 无关)
01	$m_2=0$	$m_3=0$	D1	0
10	$m_4=1$	$m_5=0$	D2	$\overline{C}$
11	$m_6=1$	$m_7=1$	D3	1

$D_0=1, D_1=0, D_2=\overline{C}, D_3=1$ 。

【VHDL】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity mux_logic is
5     port (A, B, C : in std_logic; F : out std_logic);
6 end mux_logic;
7 architecture rtl of mux_logic is
8     signal D : std_logic_vector(3 downto 0);
9     signal S : std_logic_vector(1 downto 0);
```

```

10 begin
11     S <= A & B;
12     D <= '1' & (not C) & '0' & '1';  -- D3,D2,D1,D0
13     with S select F <=
14         D(0) when "00", D(1) when "01",
15         D(2) when "10", D(3) when "11",
16         '0'   when others;
17 end rtl;

```

考试聚焦

这是组合逻辑的必考题！两种考法：(1) 2^n 选 1 MUX 实现 n 变量函数 $\rightarrow D_k$ 直接接 0/1 (2) 2^{n-1} 选 1 MUX 实现 n 变量函数 $\rightarrow$ 需要降维，剩余变量或其反接到 D 端

8.2 变式 2：双 4 选 1 MUX 构建 8 选 1 MUX

完整教学

【结构】两片 4 选 1 MUX 处理低 2 位地址。一片 2 选 1 MUX 用最高位地址 S2 选择输出。总 MUX 数：3 片 (2×4 选 1 + 1×2 选 1)。

【VHDL——结构级】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity mux8to1 is
5      port (
6          D : in  std_logic_vector(7 downto 0);
7          S : in  std_logic_vector(2 downto 0);
8          F : out std_logic
9      );
10 end mux8to1;
11
12 architecture structural of mux8to1 is
13     signal F_low, F_high : std_logic;
14
15     component mux4to1 is

```

```

16     port (
17         D : in  std_logic_vector(3 downto 0);
18         S : in  std_logic_vector(1 downto 0);
19         Y : out std_logic
20     );
21     end component;
22 begin
23     -- 片0: S1S0选D0-D3
24     mux_low: mux4to1 port map(
25         D => D(3 downto 0), S => S(1 downto 0), Y => F_low);
26     -- 片1: S1S0选D4-D7
27     mux_high: mux4to1 port map(
28         D => D(7 downto 4), S => S(1 downto 0), Y => F_high);
29     -- 顶层2选1
30     F <= F_low when S(2) = '0' else F_high;
31 end structural;

```

8.3 变式 3: 带使能端的 MUX

完整教学

【功能】E=1 正常选择；E=0 输出高阻态'Z'（三态输出）或固定'0'。

【VHDL——三态输出】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity mux4to1_tristate is
6      port (
7          D : in  std_logic_vector(3 downto 0);
8          S : in  std_logic_vector(1 downto 0);
9          E : in  std_logic;
10         F : out std_logic
11     );
12 end mux4to1_tristate;
13
14 architecture rtl of mux4to1_tristate is
15 begin
16     F <= D(to_integer(unsigned(S))) when E = '1' else 'Z';

```

```
17 end rtl;
```

【应用】多个 MUX 输出可共用一根总线（总线仲裁）。

8.4 变式 4：MUX 扩展——16 选 1

完整教学

【树形结构】4 片 4 选 1 MUX(第一层)+1 片 4 选 1 MUX(第二层)=16 选 1。总 MUX 数：5 片。

【选择位分配】S1S0→ 第一层 (4 片)，S3S2→ 第二层 (1 片)。

第九章 4 位数据比较器

9.1 第一步：解决什么问题

比较两个二进制数的大小关系。

这是数字系统中非常基础的操作：CPU 的跳转指令（BEQ, BNE, BLT, BGT）本质上就是在做数值比较。

比较器的本质：比较器 = 判断 A 和 B 的大小关系

三个输出： $A > B$ 、 $A = B$ 、 $A < B$ （互斥，任意时刻恰好一个为 1）

9.2 第二步：输入输出关系

4 位比较器：

- **数据输入：** $A_3A_2A_1A_0$ （4 位）， $B_3B_2B_1B_0$ （4 位）
- **级联输入：** $I_{A>B}$, $I_{A=B}$, $I_{A<B}$ （用于级联多片比较器）
- **输出：** $Y_{A>B}$, $Y_{A=B}$, $Y_{A<B}$

9.3 第三步：比较逻辑

比较两个多位二进制数的方法：从高位到低位逐位比较。

1. 先看最高位（MSB） A_3 vs B_3
 - 如果 $A_3 > B_3$ （即 $A_3 = 1, B_3 = 0$ ），则 $A > B$ ，比较结束
 - 如果 $A_3 < B_3$ （即 $A_3 = 0, B_3 = 1$ ），则 $A < B$ ，比较结束
 - 如果 $A_3 = B_3$ ，则 继续看下一位
2. 然后看 A_2 vs B_2 （同上逻辑）
3. 然后看 A_1 vs B_1

4. 最后看 A_0 vs B_0

这完全类比人类比较两个数字的方法：先看最高位，相等再看次高位。

9.4 第四步：逻辑推导

一位比较器的基本单元：

- $A_i > B_i$: $A_i \cdot \overline{B_i}$
- $A_i < B_i$: $\overline{A_i} \cdot B_i$
- $A_i = B_i$: $\overline{A_i \oplus B_i}$ (XNOR)

多位比较器 = 从高位到低位的级联比较：

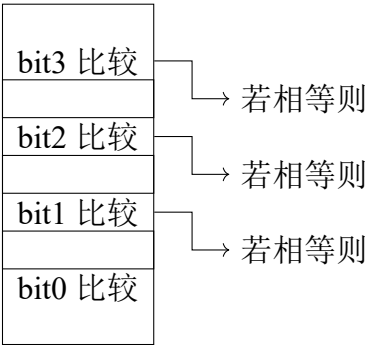
$$\begin{aligned} A > B &= (A_3 > B_3) + (A_3 = B_3) \cdot (A_2 > B_2) \\ &\quad + (A_3 = B_3) \cdot (A_2 = B_2) \cdot (A_1 > B_1) \\ &\quad + (A_3 = B_3) \cdot (A_2 = B_2) \cdot (A_1 = B_1) \cdot (A_0 > B_0) \end{aligned} \tag{9.1}$$

即：在更高位全部相等的前提下，看当前位谁大谁小。

$$\begin{aligned} A = B &= (A_3 = B_3) \cdot (A_2 = B_2) \cdot (A_1 = B_1) \cdot (A_0 = B_0) \tag{9.2} \\ A < B &= (\text{对称逻辑}) \tag{9.3} \end{aligned}$$

9.5 第五步：结构化设计

比较器内部结构 = 逐位比较单元 + 优先级逻辑：



关键设计思想：优先级从高到低。高位比较的结果优先级最高。

9.6 第六步：为什么这样设计

1. 为什么从高位到低位？

因为高位的权重更大。 $A_3 = 1$ 表示 $A \geq 8$ ，而 $A_0 = 1$ 只表示 $A \geq 1$ 。高位 1 比特的差异可以压倒低位所有比特的差异。

这和十进制完全一样：比较 9321 和 9211，百位 $2 > 1$ 就确定了大小，不需要再看十位和个位。

2. 为什么需要级联输入端？

单芯片 4 位比较器不够用时（例如需要比较 8 位或 16 位数），多片级联。级联输入接收低位比较器的结果，当高位相等时输出级联输入的值。

3. 比较器的根本限制？

组合逻辑比较器随着位宽增加，门延迟增加（每级 AND 串联）。对于大位宽（如 32 位、64 位），现代设计中常用减法器 + 标志位来判断大小（但这是另一个话题了）。

9.7 第七步：考试常见变式

1. 变式 1：8 位比较器（用两片 4 位比较器级联）

低位比较器的输出接到高位比较器的级联输入端。当高位相等时，结果由低位决定。

2. 变式 2：只输出“相等”的比较器

只需要 $A = B$ 信号。实现最简单：每一位都相等（XNOR），然后全部 AND。

3. 变式 3：比较器 + MUX 实现取最大值/最小值

比较器判断大小，MUX 选择较大的（或较小的）数输出。这在排序、中值滤波等应用中很常见。

4. 变式 4：带使能的比较器

使能端控制比较器是否工作；不工作时所有输出为 0。

9.8 第八步：VHDL 实现

```
1 library ieee;  
2 use ieee.std_logic_1164.all;
```

```
3
4 entity comparator_4bit is
5   port (
6     A    : in  std_logic_vector(3 downto 0);
7     B    : in  std_logic_vector(3 downto 0);
8     A_gt : out std_logic;
9     A_eq : out std_logic;
10    A_lt : out std_logic
11  );
12 end comparator_4bit;
13
14 architecture behavioral of comparator_4bit is
15 begin
16   process(A, B)
17   begin
18     -- 默认值
19     A_gt <= '0';
20     A_eq <= '0';
21     A_lt <= '0';
22
23     if A > B then
24       A_gt <= '1';
25     elsif A = B then
26       A_eq <= '1';
27     else
28       A_lt <= '1';
29     end if;
30   end process;
31 end behavioral;
```

Listing 9.1: 4 位比较器 VHDL 实现

VHDL 的关键优势：直接用 `if A > B` 比较运算符，综合工具会自动生成合适的比较器硬件。不需要手写门级逻辑。

9.9 第九步：Testbench 验证

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity comparator_4bit_tb is
5 end comparator_4bit_tb;
6
```

```
7 architecture test of comparator_4bit_tb is
8   signal A, B : std_logic_vector(3 downto 0);
9   signal A_gt, A_eq, A_lt : std_logic;
10
11   component comparator_4bit is
12     port (
13       A      : in  std_logic_vector(3 downto 0);
14       B      : in  std_logic_vector(3 downto 0);
15       A_gt   : out std_logic;
16       A_eq   : out std_logic;
17       A_lt   : out std_logic
18     );
19   end component;
20 begin
21   uut: comparator_4bit port map (A, B, A_gt, A_eq, A_lt);
22
23   stimulus: process
24   begin
25     -- A > B
26     A <= "1010"; B <= "0101"; wait for 10 ns;
27     -- A = B
28     A <= "1100"; B <= "1100"; wait for 10 ns;
29     -- A < B
30     A <= "0011"; B <= "1100"; wait for 10 ns;
31     wait;
32   end process;
33 end test;
```

Listing 9.2: 比较器 Testbench

9.10 第十步：如何快速解题

比较器快速解题法：

1. VHDL 解法：直接用 $>$, $=$, $<$ 运算符，综合工具自动处理
2. 门级解法：从高位到低位，用“前级相等 AND 本级比较”的级联结构
3. 级联题：低位输出 $\rightarrow$ 高位级联输入
4. 常考：比较器 + MUX 组合题（找最大/最小值）

9.11 变式详解

第十章 4 位比较器——4 大变式详解

10.1 变式 1：8 位比较器（两片 4 位级联）

完整教学

【级联方式】高位 4 位 → 主比较器。低位 4 位 → 从比较器，输出接主比较器的级联输入端。

【级联逻辑】

- 主比较器比较 A[7:4] 和 B[7:4]
- 如果高位相等，则看级联输入（低位比较器的结果）
- 低位比较器比较 A[3:0] 和 B[3:0]，输出接到主比较器的 I_A>B, I_A=B, I_A<B

【VHDL——结构级】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity comparator_8bit is
5     port (
6         A      : in  std_logic_vector(7 downto 0);
7         B      : in  std_logic_vector(7 downto 0);
8         A_gt   : out std_logic;
9         A_eq   : out std_logic;
10        A_lt   : out std_logic
11    );
12 end comparator_8bit;
13
14 architecture structural of comparator_8bit is
15     signal gt_lo, eq_lo, lt_lo : std_logic;
16
17     component comparator_4bit_cascade is
18         port (
19             A      : in  std_logic_vector(3 downto 0);
```

```

20     B      : in  std_logic_vector(3 downto 0);
21     I_gt   : in  std_logic;
22     I_eq   : in  std_logic;
23     I_lt   : in  std_logic;
24     A_gt   : out std_logic;
25     A_eq   : out std_logic;
26     A_lt   : out std_logic
27 );
28 end component;
29 begin
30     -- 低位比较器
31     cmp_lo: comparator_4bit_cascade
32     port map(A=>A(3 downto 0), B=>B(3 downto 0),
33             I_gt=>'0', I_eq=>'1', I_lt=>'0', -- 最低位: 假设之前
34             相等
35             A_gt=>gt_lo, A_eq=>eq_lo, A_lt=>lt_lo);
36     -- 高位比较器
37     cmp_hi: comparator_4bit_cascade
38     port map(A=>A(7 downto 4), B=>B(7 downto 4),
39             I_gt=>gt_lo, I_eq=>eq_lo, I_lt=>lt_lo, -- 级联!
40             A_gt=>A_gt, A_eq=>A_eq, A_lt=>A_lt);
41 end structural;

```

Listing 10.1: 8 位比较器结构级实现

【行为级——直接用运算符（更简洁）】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity comparator_8bit is
6      port (
7          A      : in  std_logic_vector(7 downto 0);
8          B      : in  std_logic_vector(7 downto 0);
9          A_gt   : out std_logic;
10         A_eq   : out std_logic;
11         A_lt   : out std_logic
12     );
13 end comparator_8bit;
14
15 architecture behavioral of comparator_8bit is
16 begin
17     A_gt <= '1' when A > B else '0';

```

```

18   A_eq <= '1' when A = B else '0';
19   A_lt <= '1' when A < B else '0';
20 end behavioral;

```

Listing 10.2: 8 位比较器行为级实现

综合器自动处理任意位宽的比较！

10.2 变式 2：只输出”相等”的比较器

完整教学

【简化】 只需要 $A = B$ 信号。每一位都相等 $\rightarrow$ XNOR $\rightarrow$ 全部 AND。

【VHDL】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity comparator_eq_only is
5      port (
6          A      : in  std_logic_vector(3 downto 0);
7          B      : in  std_logic_vector(3 downto 0);
8          A_eq   : out std_logic
9      );
10 end comparator_eq_only;
11
12 architecture behavioral of comparator_eq_only is
13 begin
14     A_eq <= '1' when A = B else '0';
15     -- 或门级：
16     -- A_eq <= (A(3) xnor B(3)) and (A(2) xnor B(2)) and
17     --          (A(1) xnor B(1)) and (A(0) xnor B(0));
18 end behavioral;

```

Listing 10.3: 相等比较器完整实现

【应用】 地址译码、标签匹配、Cache Tag 比较。

10.3 变式 3：比较器 +MUX 实现取最大值

完整教学

【电路】比较器判断 $A > B \rightarrow$ 输出作为 MUX 选择信号。 $A > B$ 时 MUX 选 A，否则选 B。

【VHDL】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity max_min_selector is
5      port (
6          A      : in  std_logic_vector(3 downto 0);
7          B      : in  std_logic_vector(3 downto 0);
8          max_val : out std_logic_vector(3 downto 0);
9          min_val : out std_logic_vector(3 downto 0)
10     );
11 end max_min_selector;
12
13 architecture behavioral of max_min_selector is
14 begin
15     max_val <= A when A > B else B;
16     min_val <= A when A < B else B;
17 end behavioral;

```

Listing 10.4: 最大值/最小值选择器

【综合结果】比较器 +MUX $\rightarrow$ 单行代码综合成两者组合。

10.4 变式 4：带使能的比较器

完整教学

【功能】enable=1 时比较器工作；enable=0 时所有输出 =0。

【VHDL】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity comparator_gated is
5      port (

```

```
6      A      : in  std_logic_vector(3 downto 0);
7      B      : in  std_logic_vector(3 downto 0);
8      en     : in  std_logic;
9      A_gt   : out std_logic;
10     A_eq   : out std_logic;
11     A_lt   : out std_logic
12 );
13 end comparator_gated;
14
15 architecture behavioral of comparator_gated is
16 begin
17     process(A, B, en)
18     begin
19         if en = '0' then
20             A_gt <= '0'; A_eq <= '0'; A_lt <= '0';
21         else
22             if A > B then A_gt <= '1'; A_eq <= '0'; A_lt <= '0';
23             elsif A = B then A_gt <= '0'; A_eq <= '1'; A_lt <= '0';
24             else A_gt <= '0'; A_eq <= '0'; A_lt <= '1';
25             end if;
26         end if;
27     end process;
28 end behavioral;
```

Listing 10.5: 带使能的比较器

第十一章 4 位加法器

11.1 第一步：解决什么问题

加法是所有算术运算的基础。

- 减法 = 加法（补码）
- 乘法 = 重复加法
- 除法 = 重复减法 = 重复加法
- ALU 的核心 = 加法器

加法器的本质：加法器 = 实现两个二进制数的加法

$$A + B + C_{in} = Sum + C_{out}$$

每一位：三个输入（ A_i, B_i, C_{in} ）、两个输出（ Sum_i, C_{out} ）

11.2 第二步：输入输出关系

4 位加法器：

- 数据输入： $A_3A_2A_1A_0$ （4 位）， $B_3B_2B_1B_0$ （4 位）， C_{in}
- 数据输出： $S_3S_2S_1S_0$ （4 位和）， C_{out} （进位输出）
- 功能： $A + B + C_{in} = C_{out}S_3S_2S_1S_0$ （5 位结果）

11.3 第三步：一位全加器——最基本的加法单元

在讨论 4 位加法器之前，必须先理解 1 位全加器（Full Adder, FA）。

1 位全加器真值表：

A_i	B_i	C_{in}	S_i	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

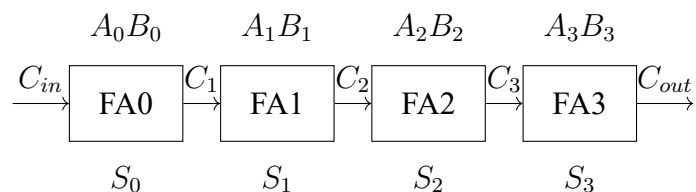
逻辑表达式：

$$S_i = A_i \oplus B_i \oplus C_{in} \quad (\text{三输入异或}) \quad (11.1)$$

$$C_{out} = A_i B_i + A_i C_{in} + B_i C_{in} \quad (\text{三中取二的多数表决}) \quad (11.2)$$

11.4 第四步：行波进位加法器（Ripple Carry Adder）

将 4 个 1 位全加器串联起来：



关键特点：每个 FA 的 C_{out} 接到下一个 FA 的 C_{in} 。进位像水波一样从低位向高位逐级传递。

因此得名：Ripple Carry（行波进位/脉动进位）。

11.5 第五步：结构化设计

4 位行波进位加法器 = 4 个 FA + 进位链：

- 每位独立计算本位的 S_i 和 C_{i+1}
- 进位 C_i 从低位传到高位
- 最关键的路径 = 进位链路径（从 C_{in} 到 C_{out} 经过 4 个 FA）

延迟分析：

- 每个 FA 的进位延迟 = 2 个门延迟 (1 个 AND + 1 个 OR)
- 4 位加法器进位延迟 = $4 \times 2 = 8$ 个门延迟
- S_3 需要等 C_3 稳定后才能计算

11.6 第六步：为什么这样设计

1. 为什么进位要逐位传递？

因为低位的结果（是否有进位）直接影响高位的计算。这是二进制的内在性质：加法有进位传播。

2. 行波进位的优缺点？

优点：结构简单、面积小 缺点：速度慢（进位链延迟随位宽线性增长）

3. 为什么还有超前进位加法器（Carry Lookahead）？

为突破行波进位速度限制而设计——提前并行计算所有进位，不等待进位传递。但考试通常只考行波进位。

11.7 第七步：考试常见变式

1. 变式 1：8 位/16 位加法器（级联 4 位加法器）

低位加法器的 C_{out} 接到高位加法器的 C_{in} 。

2. 变式 2：减法器（用加法器实现减法）

$$A - B = A + (\sim B + 1) = A + \text{补码}(B)$$

将 B 取反， $C_{in} = 1$ ，则： $A + (\sim B) + 1 = A - B$ 。

3. 变式 3：加法器 + 寄存器 = 累加器

每个时钟周期： $ACC \leftarrow ACC + Input$ ——计数器的基础！

4. 变式 4：带溢出检测的加法器

溢出条件（有符号数）：两个正数相加得负数，或两个负数相加得正数。

$$Overflow = (A_3 \odot B_3) \cdot (A_3 \oplus S_3) \quad (\text{最高位的符号变化})$$

5. 变式 5：BCD 加法器

二进制加法后如果结果 >9 ，则 +6 修正。 $C_{out} = (S > 9)$ 或 $(C_{out_binary} = 1)$ 。

11.8 第八步：VHDL 实现

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;  -- 需要此包以支持算术运算
4
5  entity adder_4bit is
6      port (
7          A      : in  std_logic_vector(3 downto 0);
8          B      : in  std_logic_vector(3 downto 0);
9          Cin     : in  std_logic;
10         Sum     : out std_logic_vector(3 downto 0);
11         Cout    : out std_logic
12     );
13 end adder_4bit;
14
15 architecture behavioral of adder_4bit is
16     signal result : std_logic_vector(4 downto 0);
17 begin
18     result <= ('0' & A) + ('0' & B) + Cin;
19     Sum    <= result(3 downto 0);
20     Cout   <= result(4);
21 end behavioral;

```

Listing 11.1: 4 位加法器 VHDL 实现（行为级）

```

1  architecture structural of adder_4bit is
2      signal carry : std_logic_vector(4 downto 0);
3
4      component full_adder is
5          port (A, B, Cin : in std_logic; Sum, Cout : out std_logic);
6      end component;
7  begin
8      carry(0) <= Cin;
9
10     gen_fa: for i in 0 to 3 generate
11         fa_i: full_adder port map (
12             A      => A(i),
13             B      => B(i),
14             Cin     => carry(i),
15             Sum     => Sum(i),
16             Cout    => carry(i+1)
17         );
18     end generate;

```

```

19
20     Cout <= carry(4);
21 end structural;

```

Listing 11.2: 结构级实现（4 个全加器级联）

11.9 第九步：Testbench 验证

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity adder_4bit_tb is
6  end adder_4bit_tb;
7
8  architecture test of adder_4bit_tb is
9      signal A, B, Sum : std_logic_vector(3 downto 0);
10     signal Cin, Cout : std_logic;
11
12     component adder_4bit is
13     port (
14         A      : in  std_logic_vector(3 downto 0);
15         B      : in  std_logic_vector(3 downto 0);
16         Cin     : in  std_logic;
17         Sum     : out std_logic_vector(3 downto 0);
18         Cout    : out std_logic
19     );
20     end component;
21 begin
22     uut: adder_4bit port map (A, B, Cin, Sum, Cout);
23
24     stimulus: process
25     begin
26         Cin <= '0';
27         A <= "0011"; B <= "0101"; wait for 10 ns;  -- 3+5=8
28         A <= "1010"; B <= "0110"; wait for 10 ns;  -- 10+6=16 -> Cout=1
29         A <= "1111"; B <= "0001"; wait for 10 ns;  -- 15+1=16 -> Cout=1
30         A <= "0000"; B <= "0000"; wait for 10 ns;  -- 0+0=0
31         wait;
32     end process;
33 end test;

```

Listing 11.3: 4 位加法器 Testbench

11.10 第十步：如何快速解题

加法器快速解题法：

1. 行为级 **VHDL**：直接用 + 运算符，最简洁
2. 结构级：generate 循环实例化 FA，进位链串联
3. 减法： $\sim B + 1 + A = A - B$
4. 常考组合：加法器 + 寄存器 = 累加器（计数器的基础！）
5. 溢出判断：有符号数用 $C_n \oplus C_{n-1}$ ，无符号数用 C_{out}

11.11 变式详解

第十二章 4 位加法器——5 大变式详解

12.1 变式 1: 8 位/16 位加法器（级联）

完整教学

【方案】两片 4 位加法器级联。低位 C_{out} → 高位 C_{in} 。

【VHDL——结构级】

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity adder_8bit is
5      port (
6          A      : in  std_logic_vector(7 downto 0);
7          B      : in  std_logic_vector(7 downto 0);
8          Cin     : in  std_logic;
9          Sum     : out std_logic_vector(7 downto 0);
10         Cout    : out std_logic
11     );
12 end adder_8bit;
13
14 architecture structural of adder_8bit is
15     signal carry : std_logic;
16
17     component adder_4bit is
18         port (
19             A      : in  std_logic_vector(3 downto 0);
20             B      : in  std_logic_vector(3 downto 0);
21             Cin     : in  std_logic;
22             Sum     : out std_logic_vector(3 downto 0);
23             Cout    : out std_logic
24         );
25     end component;
26 begin
27     add_lo: adder_4bit port map(A=>A(3 downto 0), B=>B(3 downto 0),
```

```

28      Cin=>Cin, Sum=>Sum(3 downto 0), Cout=>carry);
29      add_hi: adder_4bit port map(A=>A(7 downto 4), B=>B(7 downto 4),
30      Cin=>carry, Sum=>Sum(7 downto 4), Cout=>Cout);
31  end structural;

```

Listing 12.1: 8 位加法器结构级实现 (两片 4 位级联)

【行为级——推荐】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity adder_8bit is
6      port (
7          A      : in  std_logic_vector(7 downto 0);
8          B      : in  std_logic_vector(7 downto 0);
9          Sum     : out std_logic_vector(7 downto 0);
10         Cout    : out std_logic
11     );
12 end adder_8bit;
13
14 architecture behavioral of adder_8bit is
15     signal result : std_logic_vector(8 downto 0);
16 begin
17     result <= ('0' & A) + ('0' & B);
18     Sum <= result(7 downto 0); Cout <= result(8);
19 end behavioral;

```

Listing 12.2: 8 位加法器行为级实现

综合器自动优化任意位宽加法。

12.2 变式 2: 减法器 (用加法器实现 A-B)

完整教学

【补码原理】 $A - B = A + (-B) = A + (\sim B + 1) = A + \sim B + 1$

【电路】 B 取反 → 接加法器 B 输入。 $C_{in} = 1$ (实现 +1)。 C_{in} 就是补码中的“+1”。

【VHDL】

```

1  library ieee;

```

```

2 use ieee.std_logic_1164.all;
3 use ieee.std_logic_unsigned.all;
4
5 entity subtractor_4bit is
6     port (
7         A      : in  std_logic_vector(3 downto 0);
8         B      : in  std_logic_vector(3 downto 0);
9         diff   : out std_logic_vector(4 downto 0)
10    );
11 end subtractor_4bit;
12
13 architecture behavioral of subtractor_4bit is
14 begin
15     diff <= ('0' & A) + ('0' & (not B)) + "00001";  -- A-B = A + (~B
16         ) + 1
17     -- 或直接用减法运算符
18     -- diff <= ('0' & A) - ('0' & B);
19 end behavioral;

```

Listing 12.3: 减法器（用加法器实现 A-B）

考试聚焦

常考：为什么 $C_{in} = 1$ 能实现减法？答案： $A - B = A + (\sim B) + 1 \rightarrow +1$ 部分由 $C_{in} = 1$ 提供。

12.3 变式 3：加法器 + 寄存器 = 累加器

完整教学

【结构】加法器计算 $ACC + Input$ 。结果在时钟沿存入 ACC。ACC 输出反馈回加法器输入端。

【VHDL】

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.std_logic_unsigned.all;
4
5 entity accumulator is

```

```

6   port (
7       clk          : in  std_logic;
8       rst_n        : in  std_logic;
9       input_data    : in  std_logic_vector(7 downto 0);
10      acc           : out std_logic_vector(7 downto 0)
11  );
12  end accumulator;
13
14  architecture behavioral of accumulator is
15      signal acc_reg : std_logic_vector(7 downto 0);
16  begin
17      process(clk, rst_n)
18      begin
19          if rst_n='0' then
20              acc_reg <= (others=>'0');
21          elsif rising_edge(clk) then
22              acc_reg <= acc_reg + input_data;  -- 每个时钟累加一次
23          end if;
24      end process;
25      acc <= acc_reg;
26  end behavioral;

```

Listing 12.4: 累加器完整实现

【本质】 累加器 = 计数器的一种（不是每次 +1，而是 + 任意值）。

【应用】 DSP 中的乘累加 (MAC)、积分器。

12.4 变式 4: 带溢出检测的加法器

完整教学

【无符号数溢出】 $C_{out} = 1$ = 溢出（结果超过最大值）。

【有符号数溢出】 $Overflow = C_n \oplus C_{n-1}$ （最高位进位和次高位进位异或）。

【VHDL】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity adder_overflow is
6      port (

```

```

7      A      : in  std_logic_vector(3 downto 0);
8      B      : in  std_logic_vector(3 downto 0);
9      Sum     : out std_logic_vector(3 downto 0);
10     Cout    : out std_logic;
11     Overflow : out std_logic
12 );
13 end adder_overflow;
14
15 architecture behavioral of adder_overflow is
16     signal result : std_logic_vector(4 downto 0); -- 5位
17 begin
18     result <= ('0' & A) + ('0' & B);
19     Sum <= result(3 downto 0);
20     Cout <= result(4);
21     -- 有符号溢出
22     Overflow <= result(4) xor result(3); -- C4 xor C3
23 end behavioral;

```

Listing 12.5: 带溢出检测的加法器

【直观判断】两个正数相加得负数 → 溢出 (正溢出) 两个负数相加得正数 → 溢出 (负溢出)

12.5 变式 5: BCD 加法器

完整教学

【问题】二进制加法后，如果结果 >9(1001)，需要 +6(0110) 修正为 BCD 格式。

【VHDL】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity bcd_adder is
6      port (
7          A      : in  std_logic_vector(3 downto 0);
8          B      : in  std_logic_vector(3 downto 0);
9          Sum     : out std_logic_vector(3 downto 0);
10         Cout    : out std_logic
11     );

```

```
12 end bcd_adder;
13
14 architecture behavioral of bcd_adder is
15     signal bin_sum : std_logic_vector(4 downto 0);
16     signal bcd_sum : std_logic_vector(3 downto 0);
17 begin
18     bin_sum <= ('0' & A) + ('0' & B);
19     bcd_sum <= bin_sum(3 downto 0) when bin_sum <= 9
20         else bin_sum(3 downto 0) + "0110"; -- >9时 +6修正
21     Cout <= '1' when bin_sum > 9 else '0';
22     Sum <= bcd_sum;
23 end behavioral;
```

Listing 12.6: BCD 加法器完整实现

【例子】 $5+7=12(1100)$ 。 $1100+0110=10010 \rightarrow \text{BCD: } 0001\ 0010(12)$ 。

第十三章 级联二选一数据选择器

13.1 第一步：解决什么问题

单个 2 选 1 MUX 只能从 2 路数据中选择 1 路。如果要从更多路数据中选择，怎么办？

方案：用多个 2 选 1 MUX 级联，构建更大的 MUX。

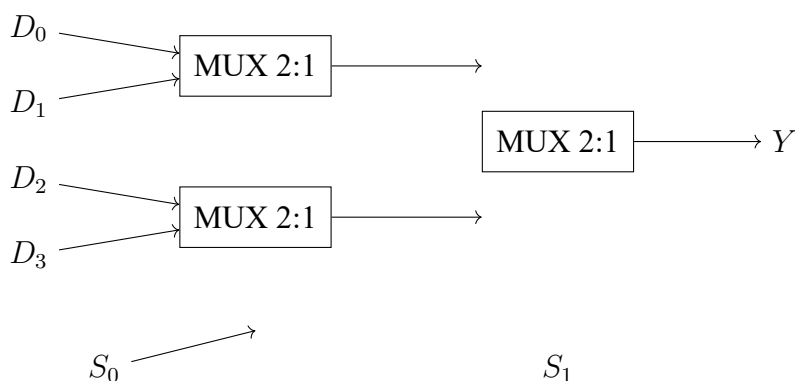
级联 MUX 的本质：用最基础的 2 选 1 MUX 作为构建单元，通过分层选择的方式扩展选择能力。

2 选 1 $\rightarrow$ 4 选 1 $\rightarrow$ 8 选 1 $\rightarrow$ 16 选 1 $\rightarrow$...

13.2 第二步：用 2 选 1 MUX 构建 4 选 1 MUX

先看 2 选 1 MUX 的基本符号： $Y = \bar{S} \cdot D_0 + S \cdot D_1$

用 3 个 2 选 1 MUX 构建 4 选 1 MUX：



工作原理：

- S_0 控制第一层两个 MUX：分别从 (D_0, D_1) 和 (D_2, D_3) 中各选一个
- S_1 控制第二层 MUX：从第一层两个结果中选一个
- 结果： (S_1, S_0) 决定了选择 D_0, D_1, D_2, D_3 中的哪一个

13.3 第三步：真值表

S_1	S_0	第一层左 MUX 输出	第一层右 MUX 输出	最终输出 Y
0	0	D_0	D_2	D_0
0	1	D_1	D_3	D_1
1	0	D_0	D_2	D_2
1	1	D_1	D_3	D_3

13.4 第四步：逻辑推导

级联 MUX 的逻辑表达式可以通过代入法推导：

第一层输出：

$$F_0 = \overline{S_0} \cdot D_0 + S_0 \cdot D_1 \quad (13.1)$$

$$F_1 = \overline{S_0} \cdot D_2 + S_0 \cdot D_3 \quad (13.2)$$

第二层输出：

$$Y = \overline{S_1} \cdot F_0 + S_1 \cdot F_1 \quad (13.3)$$

$$= \overline{S_1} \cdot (\overline{S_0} \cdot D_0 + S_0 \cdot D_1) + S_1 \cdot (\overline{S_0} \cdot D_2 + S_0 \cdot D_3) \quad (13.4)$$

$$= \overline{S_1} \cdot \overline{S_0} \cdot D_0 + \overline{S_1} \cdot S_0 \cdot D_1 + S_1 \cdot \overline{S_0} \cdot D_2 + S_1 \cdot S_0 \cdot D_3 \quad (13.5)$$

这正是标准 4 选 1 MUX 的逻辑表达式！证明级联等价于一个完整的 2^n 选 1 MUX。

13.5 第五步：结构化设计——通用级联规律

树形结构：

- 第一层： 2^{n-1} 个 2 选 1 MUX（用最低位 S_0 选择）
- 第二层： 2^{n-2} 个 2 选 1 MUX（用 S_1 选择）
- ...
- 第 n 层：1 个 2 选 1 MUX（用最高位 S_{n-1} 选择）

总 MUX 数量： $2^n - 1$ 个 2 选 1 MUX。

13.6 第六步：为什么这样设计

1. 为什么 2 选 1 是基础构建块？

2 选 1 MUX 是最简单的 MUX（1 位选择线），在 FPGA 中通常对应一个 LUT（查找表）。任何更大的 MUX 都可以由 2 选 1 MUX 级联构建。

2. 树形结构的深层意义？

每一层将数据组数减半：

- 输入： 2^n 路数据
- 第一层后： 2^{n-1} 路
- 第二层后： 2^{n-2} 路
- ...
- 第 n 层后：1 路（最终输出）

这本质上是锦标赛淘汰赛的结构——每一轮淘汰一半。

3. 延迟分析：

级联 MUX 的关键路径（从数据输入到输出）经过 n 级 2 选 1 MUX。每级延迟约 1-2 个门延迟。

13.7 第七步：考试常见变式

1. 变式 1：用 2 选 1 MUX 构建 8 选 1 MUX

需要 $2^3 - 1 = 7$ 个 2 选 1 MUX。三层结构：4+2+1。

2. 变式 2：用 4 选 1 MUX 构建 16 选 1 MUX

5 片 4 选 1 MUX：4 片第一层 + 1 片第二层（作为最终选择）。

3. 变式 3：不对称级联

例如：用 1 片 4 选 1 + 1 片 2 选 1 构建 6 选 1 MUX（实际数据输入不足 2^n 个时的处理方法）。

4. 变式 4：MUX 树实现逻辑函数

将逻辑函数的变量从低位到高位接选择线，用 MUX 树实现多变量函数。这是 MUX 做通用逻辑的关键应用。

13.8 第八步：VHDL 实现

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity mux2to1 is
5      port (
6          D0, D1 : in  std_logic;
7          S      : in  std_logic;
8          Y      : out std_logic
9      );
10 end mux2to1;
11
12 architecture behavioral of mux2to1 is
13 begin
14     Y <= D0 when S = '0' else D1;
15 end behavioral;
16
17 -- =====
18
19 library ieee;
20 use ieee.std_logic_1164.all;
21
22 entity mux4to1_cascade is
23     port (
24         D : in  std_logic_vector(3 downto 0);
25         S : in  std_logic_vector(1 downto 0);
26         Y : out std_logic
27     );
28 end mux4to1_cascade;
29
30 architecture structural of mux4to1_cascade is
31     signal F0, F1 : std_logic;  -- 第一层输出
32
33     component mux2to1 is
34         port (D0, D1 : in std_logic; S : in std_logic; Y : out std_logic)
35         ;
36     end component;
37
38 begin
39     -- 第一层: S(0)选择
40     mux_a: mux2to1 port map (D(0), D(1), S(0), F0);
41     mux_b: mux2to1 port map (D(2), D(3), S(0), F1);
42
43     -- 第二层: S(1)选择
44     mux_c: mux2to1 port map (F0, F1, S(1), Y);

```

```
42 end structural;
```

Listing 13.1: 用 2 选 1 MUX 级联构建 4 选 1 MUX（结构级）

13.9 第九步：Testbench 验证

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity mux4to1_cascade_tb is
5  end mux4to1_cascade_tb;
6
7  architecture test of mux4to1_cascade_tb is
8      signal D : std_logic_vector(3 downto 0);
9      signal S : std_logic_vector(1 downto 0);
10     signal Y : std_logic;
11
12     component mux4to1_cascade is
13     port (
14         D : in  std_logic_vector(3 downto 0);
15         S : in  std_logic_vector(1 downto 0);
16         Y : out std_logic
17     );
18     end component;
19     begin
20         uut: mux4to1_cascade port map (D, S, Y);
21
22         stimulus: process
23         begin
24             D <= "1010";
25             S <= "00"; wait for 10 ns;  -- Y=D0=0
26             S <= "01"; wait for 10 ns;  -- Y=D1=1
27             S <= "10"; wait for 10 ns;  -- Y=D2=0
28             S <= "11"; wait for 10 ns;  -- Y=D3=1
29             wait;
30         end process;
31     end test;
```

Listing 13.2: 级联 MUX Testbench

13.10 第十步：如何快速解题

级联 MUX 快速解题法：

1. n 选 1 MUX 所需 2 选 1 MUX 数目 $= n - 1$
2. 树形结构：每层将数据组数减半
3. 选择位分配：最低位控制第一层，最高位控制最后一层
4. 关键路径 $= n$ 级 2 选 1 MUX 延迟 ($n = \log_2(N)$)

秒杀：用 2 选 1 MUX 构建任意 MUX 的公式

2^n 选 1 MUX 需要 $(2^n - 1)$ 个 2 选 1 MUX，分 n 层。第 k 层（从 1 开始）有 2^{n-k} 个 MUX，由 S_{k-1} 控制。

13.11 变式详解

第十四章 级联 MUX——4 大变式详解

14.1 变式 1：用 2 选 1 MUX 构建 8 选 1 MUX

完整教学

【树形结构】3 层：第一层 4 个 → 第二层 2 个 → 第三层 1 个 = 7 个 2 选 1 MUX。

【选择位分配】

- S0 → 第一层 (4 个 MUX)：每两组数据中选一个
- S1 → 第二层 (2 个 MUX)：第一层结果中两两选择
- S2 → 第三层 (1 个 MUX)：第二层两个结果中最终选择

【通用公式】 2^n 选 1 MUX 需要 $2^n - 1$ 个 2 选 1 MUX，分 n 层。

【VHDL——generate 结构级】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity mux8to1_cascade is
5     port (
6         D : in  std_logic_vector(7 downto 0);
7         S : in  std_logic_vector(2 downto 0);
8         Y : out std_logic
9     );
10 end mux8to1_cascade;
11
12 architecture behavioral of mux8to1_cascade is
13     -- 递归描述流水线 MUX 树
14     signal layer0 : std_logic_vector(3 downto 0);
15     signal layer1 : std_logic_vector(1 downto 0);
16 begin
17     gen0: for i in 0 to 3 generate
18         layer0(i) <= D(2*i) when S(0)='0' else D(2*i+1);
19     end generate;
```

```

20  gen1: for i in 0 to 1 generate
21      layer1(i) <= layer0(2*i) when S(1)='0' else layer0(2*i+1);
22  end generate;
23  Y <= layer1(0) when S(2)='0' else layer1(1);
24  end behavioral;

```

Listing 14.1: 8 选 1 MUX (2 选 1 树形级联)

14.2 变式 2: 用 4 选 1 MUX 构建 16 选 1 MUX

完整教学

【结构】 第一层 4 片 4 选 1 MUX，第二层 1 片 4 选 1 MUX。共 5 片。

S1S0 → 第一层 (4 片 4 选 1 MUX)。S3S2 → 第二层 (1 片 4 选 1 MUX 选择第一层输出之一)。

14.3 变式 3: 不对称级联 (构建 6 选 1 MUX)

完整教学

【问题】 需要 6 选 1，但 MUX 只有 2^n 选 1。用 8 选 1 实现 6 选 1 (两个输入悬空或接地)。

【VHDL——4 选 1+2 选 1】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity mux6to1_asymmetric is
5      port (
6          D : in  std_logic_vector(5 downto 0);
7          S : in  std_logic_vector(2 downto 0);
8          Y : out std_logic
9      );
10 end mux6to1_asymmetric;
11
12 architecture structural of mux6to1_asymmetric is
13     signal mux4_out : std_logic;
14

```

```

15  component mux4to1 is
16      port (
17          D : in  std_logic_vector(3 downto 0);
18          S : in  std_logic_vector(1 downto 0);
19          Y : out std_logic
20      );
21  end component;
22  begin
23      -- 第一层: 4选1处理 D0-D3
24      mux4: mux4to1 port map(D=>D(3 downto 0), S=>S(1 downto 0), Y=>
          mux4_out);
25      -- 顶层: 2选1在 mux4_out 和 D4/D5 间选择
26      Y <= mux4_out when S(2)='0' else
27          D(4) when S(1 downto 0)="00" else D(5);
28      -- 注意: S2=1时 S1S0 只需表示 2 种状态即可
29  end structural;

```

Listing 14.2: 6 选 1 MUX (4 选 1+2 选 1 级联)

【简化方案】直接用 8 选 1 MUX，D6 和 D7 接地 (固定为 0)。

14.4 变式 4: MUX 树实现逻辑函数

完整教学

【本质】MUX= 通用逻辑实现器。MUX 树 = 多层选择 = 分而治之。

【Shannon 展开定理】 $F(x_1, x_2, \dots, x_n) = \overline{x_1} \cdot F(0, x_2, \dots, x_n) + x_1 \cdot F(1, x_2, \dots, x_n)$

这正是 2 选 1 MUX 的表达式！递归应用 = 构建 MUX 树。

【结论】任何组合逻辑函数都可用 2 选 1 MUX 树实现——这其实就是 FPGA 中 LUT 的工作原理。

第十五章 组合逻辑电路：统一设计套路

15.1 什么是组合逻辑电路

回顾我们学过的 7 个电路，它们有一个共同特征：

组合逻辑电路的定义：输出仅由当前输入决定，与历史状态无关。

没有记忆、没有反馈、没有时钟。

给定输入 $\rightarrow$ 直接得到输出（经过一定的门延迟）。

15.2 组合逻辑电路谱系

所有组合逻辑电路可以分为三类：

1. 编码/译码类

- 编码器（8-3）： $2^n \rightarrow n$ 压缩，OR 逻辑
- 译码器（3-8）： $n \rightarrow 2^n$ 展开，AND 逻辑（最小项）
- BCD 译码器：4 位 BCD $\rightarrow$ 7 段显示

编码器和译码器是互为逆过程。

2. 数据通路类

- MUX（4 选 1）：选择数据来源
- 级联 MUX：从小 MUX 构建大 MUX

MUX 是组合逻辑的”万能积木”——任何组合逻辑函数都可以用 MUX 实现。

3. 运算类

- 比较器：判断 $A > B, A = B, A < B$ ，从高位到低位优先级
- 加法器： $A + B + C_{in}$ ，进位链串联

15.3 统一设计套路

当你面对一个需要设计组合逻辑电路的考试题时，按以下步骤：

组合逻辑电路统一设计流程：

1. **明确输入输出**：几个输入？几位？几个输出？每位含义？
2. **列真值表**：穷举所有输入组合，写出对应输出
3. **推导逻辑表达式**：从真值表提取每个输出位的逻辑函数
4. **选择实现方式**：
 - 门级实现：手写与或表达式
 - MUX 实现：真值表输出列接到 MUX 数据输入端
 - 译码器 +OR 实现：最小项之和
 - VHDL 实现：case / with-select / if-else（推荐！）
5. **写 VHDL**：case 覆盖所有情况，when others 兜底
6. **验证**：检查边界条件

15.4 考试中最高频的组合逻辑考点

1. **用 MUX 实现任意逻辑函数（必考）**
 2^n 选 1 MUX 实现 n 变量函数：选择线接变量，数据输入接函数值
2. **用译码器 +OR 门实现任意逻辑函数（必考）**
译码器产生所有最小项，OR 门选择需要的项
3. **优先级编码器的 VHDL 实现（常考）**
if-else 级联：优先级从高到低
4. **74LS138 使能端级联扩展（常考）**
高位地址做片选，低位地址接输入
5. **加法器 + 寄存器 = 累加器（承上启下）**
这是从组合逻辑过渡到时序逻辑的关键桥梁

15.5 从组合逻辑到时序逻辑

到目前为止，所有电路都有一个根本限制：

组合逻辑不能存储信息。

电路的输出只取决于当前输入。当输入消失或改变，旧的结果就丢失了。

这不是设计缺陷——这是组合逻辑的本质。

如果我们需要”记住”一个值，需要存储”当前状态”，那就必须引入**时序逻辑**。

而这个过渡的桥梁就是——**D 触发器**。

组合逻辑 + 存储单元（D 触发器）= 时序逻辑 = 状态机

这将在第二部分详细展开。

第二部分

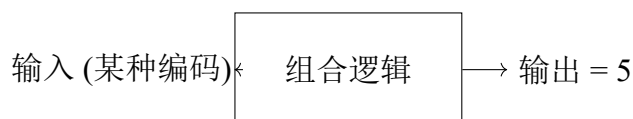
D 触发器体系

第十六章 D 触发器深度解析

16.1 为什么组合逻辑不能存储信息

让我们做一个思维实验。

假设你想”记住”数字 5。你用一个组合逻辑电路产生输出 5：



问题来了：如果切断输入（或输入改变），5 就消失了。

组合逻辑的输出**始终等于** $f(\text{输入})$ 。没有输入就没有输出。输出不能独立于输入而存在。

组合逻辑的根本限制：组合逻辑 = 无记忆电路

输出 $(t) = f(\text{输入}(t))$ ——输出在任何时刻都直接由当前输入决定。
它不能脱离输入而独立”保持”一个值。

16.2 为什么需要存储

数字系统中大量的操作需要”记住”值：

- 计数器需要记住”当前计到几了”
- 寄存器需要记住”上次存了哪个数”
- 状态机需要记住”我当前在哪个状态”
- 累加器需要记住”累加和是多少”

这些都是”信息需要跨越时间而保持”的场景。

跨越时间 = 记忆 = 存储 = 需要一种能”保持”信息的元件。

16.3 什么叫状态？什么叫记忆？

状态：状态 = 系统在某个时刻的全部内部信息的快照。

时钟驱动下，状态的变迁构成了时序电路的行为。

记忆：记忆 = 在输入消失后，信息仍然被保留的能力。

D 触发器的记忆：在时钟上升沿捕获 D 输入，之后即使 D 改变，Q 输出保持上一次捕获的值不变，直到下一个时钟上升沿。

16.4 D 触发器到底保存了什么

D 触发器的本质：D 触发器是一个 1-bit 存储单元。

- 它在每个时钟上升沿“采样”D 输入的值
- 将采样到的值保持到输出 Q 上
- 这个值一直保持到下一个时钟上升沿

16.4.1 D 触发器的输入输出

- **D (Data)：**数据输入——下一个时钟沿要存储什么值
- **CLK (Clock)：**时钟信号——决定“什么时候”采样
- **Q：**数据输出——当前存储的值
- $\overline{Q}$ ：Q 的反相（部分 D 触发器有此输出）

16.5 时钟上升沿发生什么

当时钟从 0 变 1 的那一瞬间（上升沿 $\uparrow$ ），D 触发器做一件事：

采样 D 的值 $\rightarrow$ 锁存到内部 $\rightarrow$ 驱动到 Q 输出

在上升沿之前：Q 保持旧值（上一次采样时的 D 值）

在上升沿那一瞬间：Q 更新为新值（当前 D 的值）

在上升沿之后：即使 D 变化，Q 保持不变（直到下一个上升沿）

16.5.1 逐拍分析：D 触发器的一次工作过程

假设初始状态： $Q = 0$ 。

D 触发器单次采样过程：

时刻	CLK	D	Q
上升沿到达前	0	1	0（保持旧值）
上升沿瞬间	0→1（↑）	1	1（采样并更新！）
上升沿之后	1	0	1（保持采样值，不理 D 变化）
上升沿之后	1	x（任意）	1（仍然保持）
下一次上升沿	0→1（↑）	0	0（采样新的 D 值！）

关键洞察：

- 在非上升沿时刻，D 可以随便变化，Q 纹丝不动
- 只有上升沿那一瞬间，D 的值被”捕获”
- 捕获后 Q 保持该值，像一个”时间冻结”的样本

16.6 $Q(n)$ 和 $Q(n+1)$ 到底是什么意思

这是数字逻辑中最核心的概念之一：

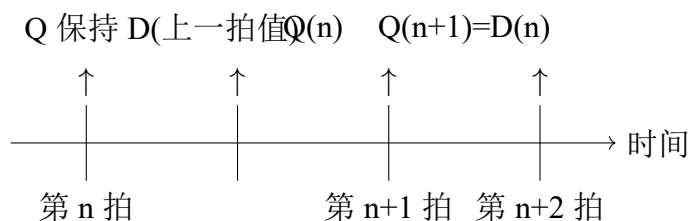
$Q(n)$ vs $Q(n+1)$:

- $Q(n)$ = 第 n 个时钟周期内（上升沿之后），触发器的**当前**输出
 - $Q(n+1)$ = 第 $n+1$ 个时钟上升沿到达后，触发器的**下一个**输出
- $Q(n)$ 和 $Q(n+1)$ 之间恰好差一个时钟周期。

对于 D 触发器：

$$Q(n+1) = D(n) \quad (16.1)$$

含义：下一个时钟上升沿之后 Q 的值 = 当前上升沿时采样的 D 值。

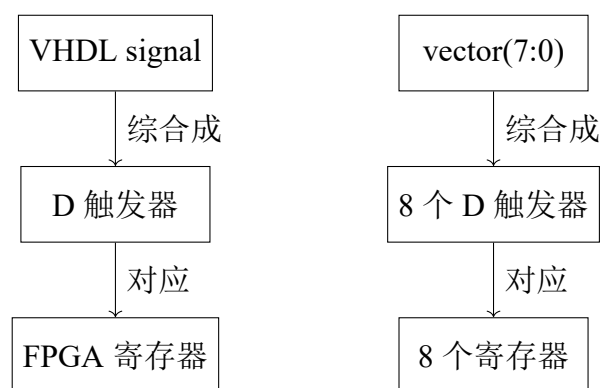


16.7 FPGA 中的寄存器资源与 D 触发器关系

FPGA 寄存器的本质：FPGA 中的”寄存器”(Register/Flip-Flop) 就是一个 D 触发器。

- 每个 LE (Logic Element) /Slice 包含若干个 D 触发器
- 当你在 VHDL 中写 `signal x : std_logic` 并在 `process` 中赋值，综合器就把 `x` 映射到一个 D 触发器
- `std_logic_vector(7 downto 0) = 8 个 D 触发器并排`

总结关系：



16.8 D 触发器的 VHDL 实现

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity dff is
5     port (
6         clk    : in  std_logic;
7         rst_n   : in  std_logic;  -- 异步复位（低有效）
8         D       : in  std_logic;
9         Q       : out std_logic
10    );
11 end dff;
12
13 architecture behavioral of dff is
14 begin
15     process(clk, rst_n)
16     begin
17         if rst_n = '0' then
```

```

18     Q <= '0';           -- 异步复位
19     elsif rising_edge(clk) then
20         Q <= D;         -- 时钟上升沿：采样D并输出到Q
21     end if;
22 end process;
23 end behavioral;

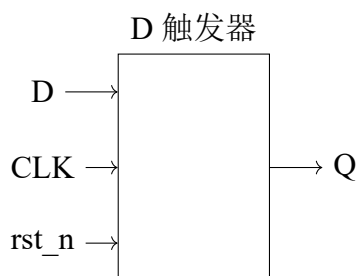
```

Listing 16.1: D 触发器 VHDL 实现（带异步复位）

16.8.1 VHDL 中的关键点

1. process(clk, rst_n): 敏感信号列表包含 CLK 和复位
2. if rst_n = '0' then: 异步复位优先——复位不受时钟控制
3. elsif rising_edge(clk) then: 这才是“边沿触发”——只有在上升沿才执行
4. Q <= D: 这是 D 触发器的核心行为—— $Q(n+1) = D(n)$

16.8.2 综合成什么硬件



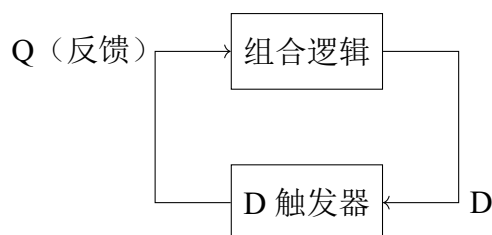
内部：主从锁存器结构

16.9 D 触发器 + 组合逻辑 = 一切时序电路

现在我们有：

- **组合逻辑**：能做计算，但不能存储
- **D 触发器**：能存储 1 bit，但不能计算

把它们结合起来：



这就构成了状态机！

Q（状态输出） → 组合逻辑计算 → 产生新的 D → 下一个时钟沿存入 D 触发器 → 新状态！

这就是所有同步时序电路的核心结构。

时序电路核心模型：

当前状态 $\xrightarrow{\text{组合逻辑}}$ 下一状态 $\xrightarrow{\text{时钟沿}}$ 存入 D 触发器

时钟驱动状态向前流动。每个时钟周期，状态更新一次。

16.10 Testbench 验证

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity dff_tb is
5  end dff_tb;
6
7  architecture test of dff_tb is
8      signal clk, rst_n, D, Q : std_logic;
9      constant clk_period : time := 20 ns;
10
11     component dff is
12         port (clk, rst_n, D : in std_logic; Q : out std_logic);
13     end component;
14 begin
15     uut: dff port map (clk, rst_n, D, Q);
16
17     -- 时钟生成
18     clk_process: process
19     begin
20         clk <= '0'; wait for clk_period/2;
  
```

```
21     clk <= '1'; wait for clk_period/2;
22 end process;
23
24 -- 测试序列
25 stimulus: process
26 begin
27     rst_n <= '0'; D <= '0'; wait for 30 ns; -- 复位
28     rst_n <= '1';          wait for 5 ns;
29
30     D <= '1'; wait for clk_period; -- 上升沿采样 D=1
31     D <= '0'; wait for clk_period; -- 上升沿采样 D=0
32     D <= '1'; D <= '0' after 5 ns; -- 在非上升沿改变 D (Q 不会变)
33     wait for clk_period;
34     wait;
35 end process;
36 end test;
```

Listing 16.2: D 触发器 Testbench

16.11 D 触发器体系总结

1. 一个 D 触发器 = 1 bit 存储：能记住一个二进制位
2. 时钟沿触发：只有在上升沿（或下降沿）才采样和更新
3. $Q(n+1) = D(n)$ ：D 触发器的特征方程，一切推导的基础
4. N 个 D 触发器并排：可以存储 N 位数据（寄存器）
5. 组合逻辑 + D 触发器：构成一切同步时序电路

下一步：将 D 触发器串联起来——得到计数器、移位寄存器、状态机等一切有趣的电路。

16.12 变式详解

第十七章 D 触发器——核心变式详解

17.1 变式 1：带异步复位的 D 触发器 vs 带同步复位的 D 触发器

完整教学

【异步复位】复位不受时钟控制，rst=0 立即清零（不等时钟沿）。【同步复位】复位只在时钟沿有效，rst=0 后需等下一个时钟上升沿才清零。

【VHDL 对比】

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  -- ===== 异步复位D触发器 =====
5  entity dff_async_rst is
6      port (
7          clk      : in  std_logic;
8          rst_n     : in  std_logic;
9          D         : in  std_logic;
10         Q         : out std_logic
11     );
12 end dff_async_rst;
13
14 architecture behavioral of dff_async_rst is
15 begin
16     -- 异步复位（rst在敏感列表中，且在rising_edge之前判断）
17     process(clk, rst_n)
18     begin
19         if rst_n = '0' then Q <= '0';           -- 立即响应
20         elsif rising_edge(clk) then Q <= D;
21         end if;
22     end process;
23 end behavioral;
```

```

25  -- ===== 同步复位D触发器 =====
26  entity dff_sync_rst is
27      port (
28          clk    : in  std_logic;
29          rst_n   : in  std_logic;
30          D       : in  std_logic;
31          Q       : out std_logic
32      );
33  end dff_sync_rst;
34
35  architecture behavioral of dff_sync_rst is
36  begin
37      -- 同步复位 (rst不在敏感列表, 在rising_edge内部判断)
38      process(clk)
39      begin
40          if rising_edge(clk) then
41              if rst_n = '0' then Q <= '0';           -- 等时钟沿
42              else Q <= D;
43              end if;
44          end if;
45      end process;
46  end behavioral;

```

Listing 17.1: 异步复位 D 触发器 vs 同步复位 D 触发器

【区别】

	异步复位	同步复位
敏感列表	clk, rst_n	clk
复位延迟	立即	等 1 个时钟周期
抗噪性	差 (毛刺可触发复位)	好 (只在沿采样)
综合结果	DFF+ 异步 CLR	DFF+ 同步 CLR

考试聚焦

必考判断: 看 process 敏感列表有没有 rst_n。

- 有 → 异步复位
- 没有 → 同步复位

17.2 变式 2：带时钟使能的 D 触发器

完整教学

【功能】CE=1 时正常采样 D；CE=0 时保持 Q 不变。

【VHDL】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity dff_ce is
5     port (
6         clk : in  std_logic;
7         CE  : in  std_logic;
8         D   : in  std_logic;
9         Q   : out std_logic
10    );
11 end dff_ce;
12
13 architecture behavioral of dff_ce is
14 begin
15     process(clk)
16     begin
17         if rising_edge(clk) then
18             if CE = '1' then Q <= D; end if;
19             -- CE='0': 不赋值=保持
20         end if;
21     end process;
22 end behavioral;
```

Listing 17.2: 带时钟使能的 D 触发器

【综合结果】D 触发器 + 时钟使能逻辑（MUX 反馈 Q 或采样 D）。

【应用】所有带使能的寄存器/计数器都基于这个结构。

17.3 变式 3：多位 D 触发器 = 寄存器

完整教学

【原理】N 个 D 触发器并行 → N 位寄存器。同一时钟沿采样 N 位输入。

【VHDL】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity register_8bit is
5      port (
6          clk      : in  std_logic;
7          data_in  : in  std_logic_vector(7 downto 0);
8          reg_out  : out std_logic_vector(7 downto 0)
9      );
10 end register_8bit;
11
12 architecture behavioral of register_8bit is
13     signal reg : std_logic_vector(7 downto 0);
14 begin
15     process(clk) begin
16         if rising_edge(clk) then reg <= data_in; end if;
17     end process;
18     reg_out <= reg;
19 end behavioral;

```

Listing 17.3: 8 位寄存器（多位 D 触发器）

【综合】8 个 D 触发器 → 8 个 FPGA 寄存器。

【应用】流水线寄存器、数据缓冲、状态寄存器。

17.4 变式 4: D 触发器构成 2 分频器

完整教学

【原理】 $\overline{Q}$ 接回 $D \rightarrow Q(n+1) = \overline{Q(n)} \rightarrow$ 每拍翻转 $\rightarrow 2$ 分频。

【VHDL】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity div2 is
5      port (
6          clk : in  std_logic;
7          Q   : out std_logic
8      );
9  end div2;

```

```
10
11 architecture behavioral of div2 is
12     signal Q_int : std_logic := '0';
13 begin
14     process(clk) begin
15         if rising_edge(clk) then Q_int <= not Q_int; end if;
16     end process;
17     Q <= Q_int;
18 end behavioral;
```

Listing 17.4: 2 分频器 (D 触发器自反馈)

【这是分频器和计数器的原子单元。】

17.5 变式 5: 建立/保持时间与最大工作频率

完整教学

【建立时间 t_{su} 】时钟沿到来前, D 输入必须稳定多久。【保持时间 t_h 】时钟沿到来后, D 输入必须保持多久。【 t_{cko} 】时钟沿到 Q 输出有效的延迟。

【最大频率】 $f_{max} = 1/(t_{cko} + t_{comb} + t_{su})$

t_{comb} = 触发器之间的组合逻辑延迟。

【考试计算】给定 $t_{cko}=2\text{ns}$, $t_{su}=1\text{ns}$, 组合逻辑延迟 $=5\text{ns}$ 。 $f_{max} = 1/(2+5+1) = 125\text{MHz}$ 。

第三部分

计数器体系

第十八章 计数器统一框架：从状态机视角理解计数器

18.1 最重要的一句话

计数器的本质：计数器就是一个状态机，它的状态转移函数是：

$$\text{下一状态} = \text{当前状态} + 1 \quad (\text{模 } N)$$

计数器不是“一个能计数的东西”，而是“一个状态在 0 到 $N-1$ 之间循环的时序电路”。

18.2 什么是状态

回顾 D 触发器：每个 D 触发器保存 1 bit 信息。

4 个 D 触发器并排：



$$\times 1 \quad \times 2 \quad \times 4 \quad \times 8$$

$$\text{存储的值} = 8 \times Q_3 + 4 \times Q_2 + 2 \times Q_1 + 1 \times Q_0$$

状态 = 这四个 Q 值在某个时刻的具体组合。

例如：

- 状态“5” = $Q_3Q_2Q_1Q_0 = 0101$
- 状态“12” = $Q_3Q_2Q_1Q_0 = 1100$
- 状态“0” = $Q_3Q_2Q_1Q_0 = 0000$

18.3 什么是状态转移

状态转移：在时钟沿驱动下，系统从当前状态变为下一个状态。

计数器中的状态转移规则只有一条：

如果当前状态 = i ($i < N - 1$)

则下一状态 = $i + 1$

如果当前状态 = $N - 1$

则下一状态 = 0

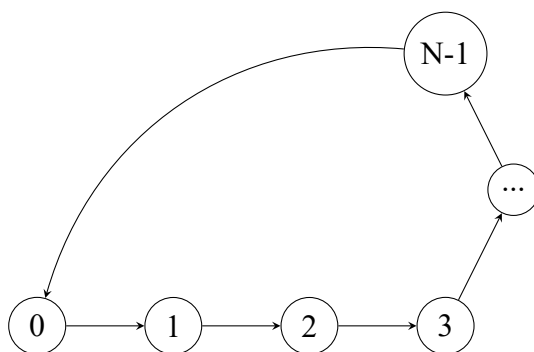
N = 模值——计数器在 0 到 $N-1$ 之间循环。

18.4 为什么计数器本质是状态机

状态机有三个要素：

1. **当前状态：**计数器当前计数值
2. **状态转移函数：** $S_{next} = (S_{current} + 1) \bmod N$
3. **时钟驱动：**每个时钟上升沿，状态更新一次

计数器完美符合状态机的定义。



状态循环： $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow N-1 \rightarrow 0 \rightarrow 1 \rightarrow \dots$

18.5 为什么计数器本质是”当前状态 +1”

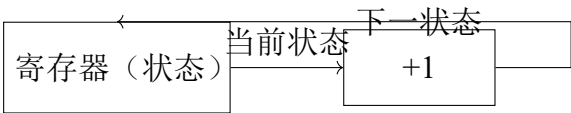
这听起来像废话——但这恰恰是最重要的理解。
计数器的核心操作是：

$$\text{Reg}[3 : 0] \leftarrow \text{Reg}[3 : 0] + 1$$

(18.1)

用硬件实现：

- 1. 寄存器（D 触发器组）存储当前计数值（当前状态）
- 2. 加法器（组合逻辑）计算”当前状态 + 1”（下一状态）
- 3. 时钟沿到来时，加法器的输出存入寄存器（状态更新）



反馈环：状态 → +1 → 下一状态 → 存回寄存器

这个反馈环是计数器（以及所有状态机）的硬件本质。

18.6 通用计数器设计流程

通用计数器设计流程——适用于任何模值：

- 1. 确定模值 **N**：计数器从 0 计到 N-1，共 N 个状态
- 2. 确定寄存器位宽 **k**： $k = \lceil \log_2 N \rceil$ （能满足表示 N-1 的最小位数）
- 3. 设计状态转移：当前状态 + 1（这是所有二进制计数器的共同逻辑）
- 4. 设计回零条件：当当前状态 = N-1 时（这是不同计数器的唯一区别）
 - 进位置 1（表示一轮计数完成）
 - 下一状态 = 0
- 5. 验证状态完整性：确保从 0 到 N-1 的每个状态都能正确转移，且状态 N 不会出现
- 6. 转换成 VHDL：
 - 标准写法：if rising_edge(clk) then count <= count + 1;
 - 加上清零条件：if count = N-1 then count <= 0;

18.7 为什么所有计数器本质上是一个问题

看下面几个需求：

- ”设计一个模 12 计数器”： $0 \rightarrow 1 \rightarrow \dots \rightarrow 11 \rightarrow 0 \rightarrow 1 \rightarrow \dots$
- ”设计一个模 24 计数器”： $0 \rightarrow 1 \rightarrow \dots \rightarrow 23 \rightarrow 0 \rightarrow 1 \rightarrow \dots$
- ”设计一个模 60 计数器”： $0 \rightarrow 1 \rightarrow \dots \rightarrow 59 \rightarrow 0 \rightarrow 1 \rightarrow \dots$

它们的区别只有一个：清零条件不同。

- 模 12：count = 11 时清零（即 1100 时回零）
- 模 24：count = 23 时清零（即 10111 时回零）
- 模 60：count = 59 时清零（即 111011 时回零）

除此之外，其余所有逻辑完全相同！

这就是为什么我们要建立一个统一框架来理解所有计数器。

计数器统一公式：任意模 N 二进制计数器：

```
1  if rising_edge(clk) then
2      if count = N-1 then
3          count <= 0;
4      else
5          count <= count + 1;
6      end if;
7  end if;
```

唯一的参数： N （模值）

18.8 接下来的学习顺序

在建立了统一框架之后，我们将用这个框架统一推导：

- 4 位计数器（模 16， $N = 2^4$ ）——最基础
- 模 12 计数器—— $N = 12$ ，需要 4 位 + 回零逻辑
- 模 24 计数器—— $N = 24$ ，需要 5 位 + 回零逻辑
- 模 60 计数器—— $N = 60$ ，需要 6 位 + 回零逻辑

- **模为任意值的计数器**——通用设计方法
- **BCD 计数器**（模 10，B 个位/十位分开编码）
- **BCD 模 24/模 60**——多位 BCD 级联

你会发现它们都是从同一个模板稍加修改得到的。

第十九章 4 位加法计数器——模 2^N 二进制计数器

19.1 应用统一框架

1. 模值 N : $N = 2^4 = 16$ (4 位二进制能表示 0 到 15)
2. 寄存器位宽 k : $k = 4$ (Q3 Q2 Q1 Q0)
3. 状态转移: $\text{count} \leftarrow \text{count} + 1$
4. 回零条件: 当 $\text{count} = 1111$ (15) 时, 下一状态自动归 0
 - 注意: 4 位二进制 $1111 + 1 = 10000$ (5 位), 但寄存器只有 4 位, 最高位 (进位) 自然丢弃
 - 所以不需要显式清零——自然溢出就是模 16

19.2 逐拍状态分析

这是全书第一个需要逐拍分析的电路。请仔细看每个时钟周期内发生了什么。

19.2.1 初始状态

时钟第 0 拍 (初始):

- $Q_3Q_2Q_1Q_0 = 0000$ (状态 0)
- 加法器计算: $0000 + 1 = 0001$
- $D_3D_2D_1D_0 = 0001$ (准备好下一状态)

19.2.2 时钟第 1 拍到第 16 拍

4 位计数器完整 16 拍：

时钟拍	上升沿前 Q 值	加法器输出 (Q+1)	上升沿后 Q 值	十进制	进位
1	0000	0001	0001	1	0
2	0001	0010	0010	2	0
3	0010	0011	0011	3	0
4	0011	0100	0100	4	0
5	0100	0101	0101	5	0
6	0101	0110	0110	6	0
7	0110	0111	0111	7	0
8	0111	1000	1000	8	0
9	1000	1001	1001	9	0
10	1001	1010	1010	10	0
11	1010	1011	1011	11	0
12	1011	1100	1100	12	0
13	1100	1101	1101	13	0
14	1101	1110	1110	14	0
15	1110	1111	1111	15	0
16	1111	0000	0000	0	1

关键观察：

- 第 16 拍：1111 + 1 = 10000，但在 4 位寄存器中只保存低 4 位（0000），进位自然的被丢弃
- 计数器不需要任何清零逻辑就能自动归零——因为 4 位自然溢出
- 这就是模 16 的本质：利用 2^N 的自然溢出

19.3 为什么 Q0 翻转最快

观察 Q0（最低位）的变化：

0, 1, 0, 1, 0, 1, 0, 1, ...

Q0 在每个时钟周期都翻转！频率 = $f_{clk}/2$ 。

观察 Q1 的变化：

0, 0, 1, 1, 0, 0, 1, 1, ...

Q1 每两个时钟周期翻转一次！频率 = $f_{clk}/4$ 。

观察 Q2 的变化：频率 = $f_{clk}/8$ 。

规律：Q_k 的频率 = $f_{clk}/2^{k+1}$ 。

这就是分频器的原理——我们将在第四部分详细展开。

19.4 VHDL 实现

```

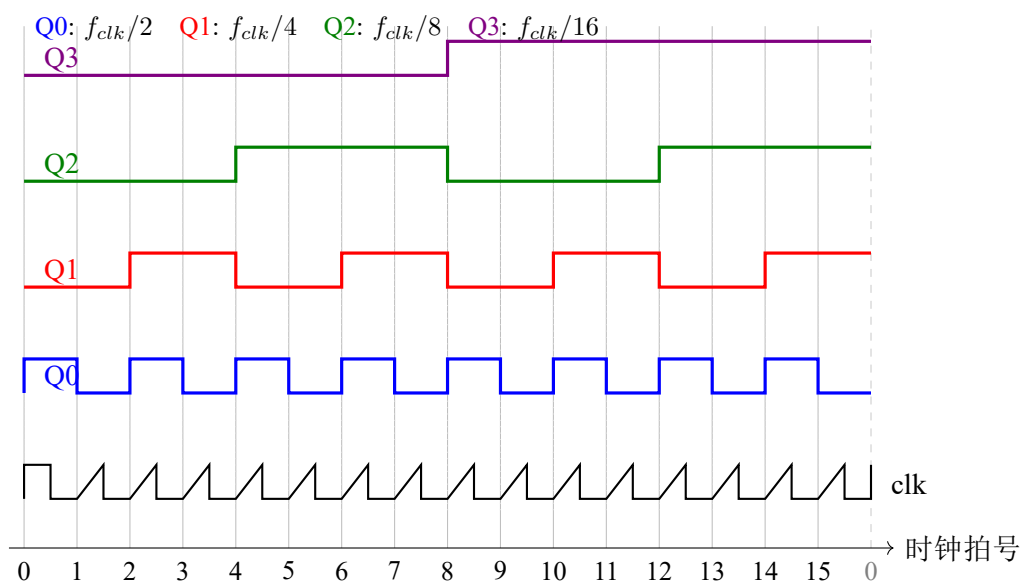
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity counter_4bit is
6      port (
7          clk      : in  std_logic;
8          rst_n    : in  std_logic;
9          count    : out std_logic_vector(3 downto 0);
10         carry    : out std_logic  -- 计数一轮完成标志
11     );
12 end counter_4bit;
13
14 architecture behavioral of counter_4bit is
15     signal cnt : std_logic_vector(3 downto 0);
16 begin
17     process(clk, rst_n)
18     begin
19         if rst_n = '0' then
20             cnt <= (others => '0');
21         elsif rising_edge(clk) then
22             cnt <= cnt + 1;  -- 自然溢出, 1111+1=0000
23         end if;
24     end process;
25
26     count <= cnt;
27     carry <= '1' when cnt = "1111" else '0';
28 end behavioral;

```

Listing 19.1: 4 位二进制计数器（模 16）

19.5 内部寄存器变化过程

将每个时钟上升沿后寄存器内容画出来：



关键观察:

- Q0 在**每个**时钟上升沿翻转——这是二分频的基础
- Q1 每 2 个时钟周期翻转一次——Q0 的翻转频率是 Q1 的 2 倍
- Q2 每 4 个时钟周期翻转一次——Q1 的翻转频率是 Q2 的 2 倍
- Q3 每 8 个时钟周期翻转一次——Q2 的翻转频率是 Q3 的 2 倍
- 规律: Q_k 的频率 = $f_{clk}/2^{k+1}$, 这正是**分频器**的核心原理

19.6 Testbench

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity counter_4bit_tb is
5  end counter_4bit_tb;
6
7  architecture test of counter_4bit_tb is
8      signal clk, rst_n, carry : std_logic;
9      signal count : std_logic_vector(3 downto 0);
10     constant clk_period : time := 20 ns;
11
12     component counter_4bit is
13         port (clk, rst_n : in std_logic;
14             count : out std_logic_vector(3 downto 0);
15             carry : out std_logic);

```

```
16     end component;  
17 begin  
18     uut: counter_4bit port map (clk, rst_n, count, carry);  
19  
20     clk_process: process  
21     begin  
22         clk <= '0'; wait for clk_period/2;  
23         clk <= '1'; wait for clk_period/2;  
24     end process;  
25  
26     stimulus: process  
27     begin  
28         rst_n <= '0'; wait for 30 ns;  
29         rst_n <= '1'; wait for 500 ns;  -- 观察完整16拍循环  
30         wait;  
31     end process;  
32 end test;
```

Listing 19.2: 4 位计数器 Testbench

第二十章 模 12 加法计数器

20.1 应用统一框架

1. 模值 N : $N = 12$ (状态: 0, 1, 2, ..., 10, 11, 0, ...)
2. 寄存器位宽 k : $k = \lceil \log_2 12 \rceil = 4$ (需要 4 位, 但只用到 12 个状态)
3. 状态转移: $\text{count} \leftarrow \text{count} + 1$ (与模 16 完全相同!)
4. 回零条件: 这是唯一与模 16 不同的地方
 - 模 16: 自然溢出 ($1111+1=0000$)
 - 模 12: 需要显式清零——当 $\text{count} = 11(1011)$ 时, 下一拍强制归 0

20.2 逐拍状态分析

20.2.1 关键问题: 为什么模 12 不能自然溢出

模 16: $2^4 = 16$, 计数器恰好用完所有 4 位状态, $1111 + 1$ 自然溢出为 0000。

模 12: $2^4 = 16 > 12$, 计数器有 4 个状态 (1100 到 1111) 不应当出现。如果没有清零逻辑, 计数器的行为是:

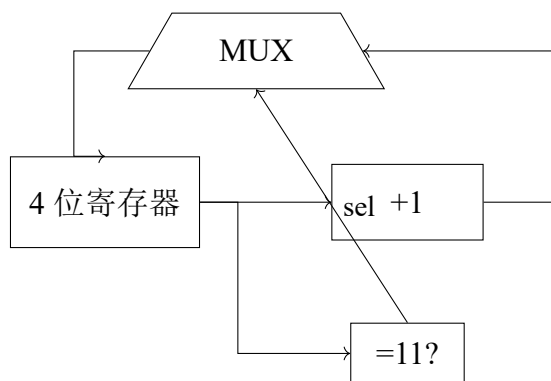
$\dots \rightarrow 11(1011) \xrightarrow{+1} 12(1100) \xrightarrow{+1} 13(1101) \xrightarrow{+1} 14(1110) \xrightarrow{+1} 15(1111) \xrightarrow{+1} 0(0000)$

这计到了 16 个状态而不是 12 个! 所以必须在状态 11 之后强制归零。

模 12 计数器前 13 拍：

时钟拍	上升沿前 Q	上升沿后 Q	说明
1	0000	0001 (1)	正常 +1
2	0001	0010 (2)	正常 +1
...	...	...	...
11	1010	1011 (11)	正常 +1
12	1011	0000 (0)	检测到 11 → 强制清零!
13	0000	0001 (1)	重新开始新一轮计数

20.3 模 12 计数器内部硬件结构



与模 16 相比只多了“=11 检测 +MUX 清零”部分

与模 16 计数器的区别只有两处：

1. 增加了一个比较器（检测 $\text{count} = 11$ ）
2. 增加了一个 MUX（选择：+1 的结果还是 0）

其余部分一模一样。

20.4 VHDL 实现

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity counter_mod12 is
6  port (
7      clk    : in  std_logic;
8      rst_n  : in  std_logic;

```

```

9      count : out std_logic_vector(3 downto 0);
10      carry : out std_logic
11  );
12  end counter_mod12;
13
14  architecture behavioral of counter_mod12 is
15      signal cnt : std_logic_vector(3 downto 0);
16  begin
17      process(clk, rst_n)
18      begin
19          if rst_n = '0' then
20              cnt <= (others => '0');
21          elsif rising_edge(clk) then
22              if cnt = "1011" then      -- = 11
23                  cnt <= (others => '0');  -- 强制归零
24              else
25                  cnt <= cnt + 1;        -- 正常+1
26              end if;
27          end if;
28      end process;
29
30      count <= cnt;
31      carry <= '1' when cnt = "1011" else '0';
32  end behavioral;

```

Listing 20.1: 模 12 计数器

20.5 常见错误分析

模 12 计数器的常见错误:

1. 错误: 忘记清零

如果只写 `cnt <= cnt + 1` 而不加清零条件, 得到的是模 16 计数器, 不是模 12。

2. 错误: 清零条件写错

清零条件是 `cnt = 11`, 不是 `cnt = 12`。因为计数器永远不会到达状态 12——它在 11 的下一拍就被清零了。

3. 错误: 位宽不够

4 位最大值是 15(1111), 11(1011) 在 4 位范围内, 但如果你只用 3 位 (最大值 7), 就无法表示 8-11 的状态。

20.6 Testbench

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity counter_mod12_tb is
5  end counter_mod12_tb;
6
7  architecture test of counter_mod12_tb is
8      signal clk, rst_n, carry : std_logic;
9      signal count : std_logic_vector(3 downto 0);
10     constant clk_period : time := 20 ns;
11
12     component counter_mod12 is
13         port (clk, rst_n : in std_logic;
14             count : out std_logic_vector(3 downto 0);
15             carry : out std_logic);
16     end component;
17 begin
18     uut: counter_mod12 port map (clk, rst_n, count, carry);
19
20     clk_process: process
21     begin
22         clk <= '0'; wait for clk_period/2;
23         clk <= '1'; wait for clk_period/2;
24     end process;
25
26     stimulus: process
27     begin
28         rst_n <= '0'; wait for 30 ns;
29         rst_n <= '1';
30         -- 观察2个完整循环（24拍）以确保清零正确
31         wait for clk_period * 25;
32         wait;
33     end process;
34 end test;
```

Listing 20.2: 模 12 计数器 Testbench——验证完整 12 拍循环

验证关键：第 12 拍后 count 应该归 0，第 13 拍 count = 1，证明循环正确。

第二十一章 模 24 计数器

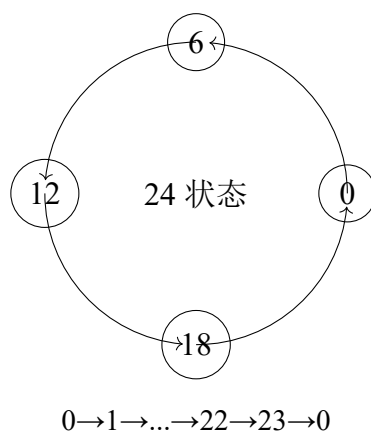
21.1 应用统一框架

1. 模值 N : $N = 24$
2. 寄存器位宽 k : $k = \lceil \log_2 24 \rceil = 5$ (需要 5 位, 最多表示 31)
3. 状态转移: $\text{count} \leftarrow \text{count} + 1$
4. 回零条件: $\text{count} = 23(10111)$ 时, 强制归零

21.2 关键状态分析

模 24 计数器的状态范围: $00000(0) \rightarrow 10111(23) \rightarrow 00000(0)$

模 24 常与时钟应用相关: 24 小时制时钟的小时位。



21.3 为什么是 5 位

$2^4 = 16 < 24 < 2^5 = 32$, 所以需要 5 位寄存器。

5 位二进制最大值 $11111 = 31$, 但计数器只用到 0-23 (24 个状态)。状态 24-31 (11000-11111) 永远不会出现。

位宽确定公式：对于模 N 计数器，所需寄存器位数 = $\lceil \log_2 N \rceil$

- 模 16 $\rightarrow$ 4 位（恰好用完）
- 模 12 $\rightarrow$ 4 位（有 4 个冗余状态）
- 模 24 $\rightarrow$ 5 位（有 8 个冗余状态）
- 模 60 $\rightarrow$ 6 位（有 4 个冗余状态）
- 模 10 $\rightarrow$ 4 位（有 6 个冗余状态）

21.4 VHDL 实现

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity counter_mod24 is
6      port (
7          clk      : in  std_logic;
8          rst_n    : in  std_logic;
9          count    : out std_logic_vector(4 downto 0);
10         carry    : out std_logic
11     );
12 end counter_mod24;
13
14 architecture behavioral of counter_mod24 is
15     signal cnt : std_logic_vector(4 downto 0);
16 begin
17     process(clk, rst_n)
18     begin
19         if rst_n = '0' then
20             cnt <= (others => '0');
21         elsif rising_edge(clk) then
22             if cnt = "10111" then      -- = 23
23                 cnt <= (others => '0');
24             else
25                 cnt <= cnt + 1;
26             end if;
27         end if;
28     end process;
29
30     count <= cnt;
```



```
31   carry <= '1' when cnt = "10111" else '0';
32 end behavioral;
```

Listing 21.1: 模 24 计数器

21.5 与模 12 的比较

参数	模 12	模 24
模值 N	12	24
位宽	4 位	5 位
清零条件	cnt = 11(1011)	cnt = 23(10111)
其余逻辑	完全相同	

结论：模 12 和模 24 的 VHDL 代码，区别只有一行——清零条件不同。这就是统一框架的力量。

第二十二章 模 60 计数器

22.1 应用统一框架

1. 模值 N : $N = 60$
2. 寄存器位宽 k : $k = \lceil \log_2 60 \rceil = 6$ ($2^5 = 32 < 60 < 2^6 = 64$)
3. 状态转移: $\text{count} \leftarrow \text{count} + 1$
4. 回零条件: $\text{count} = 59(111011)$ 时, 强制归零

22.2 模 60 的实际意义

模 60 计数器是数字钟表的核心:

- 秒: 0-59 (模 60)
- 分: 0-59 (模 60)

一个完整的数字钟 = 模 60 秒 + 模 60 分 + 模 24 (或模 12) 时。

22.3 VHDL 实现

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.std_logic_unsigned.all;
4
5 entity counter_mod60 is
6     port (
7         clk    : in  std_logic;
8         rst_n  : in  std_logic;
9         count  : out std_logic_vector(5 downto 0);
10        carry  : out std_logic
11    );
```

```

12 end counter_mod60;
13
14 architecture behavioral of counter_mod60 is
15     signal cnt : std_logic_vector(5 downto 0);
16 begin
17     process(clk, rst_n)
18     begin
19         if rst_n = '0' then
20             cnt <= (others => '0');
21         elsif rising_edge(clk) then
22             if cnt = "111011" then      -- = 59
23                 cnt <= (others => '0');
24             else
25                 cnt <= cnt + 1;
26             end if;
27         end if;
28     end process;
29
30     count <= cnt;
31     carry <= '1' when cnt = "111011" else '0';
32 end behavioral;

```

Listing 22.1: 模 60 二进制计数器

22.4 三种模值计数器的统一对比

参数	模 12	模 24	模 60
模值 N	12	24	60
位宽	4 位	5 位	6 位
2^k	16	32	64
冗余状态数	4	8	4
清零条件	cnt=11	cnt=23	cnt=59
清零二进制	1011	10111	111011
VHDL 差异	仅修改：位宽 + 清零比较值		

所有二进制计数器的统一模板：修改任意模 N 计数器的三个步骤：

1. 计算位宽： $k = \lceil \log_2 N \rceil$ 2. 修改信号位宽为 k 3. 修改清零条件为：cnt = N-1

其余逻辑完全不变。

第二十三章 模为任意值的二进制计数器

23.1 通用设计公式

前面我们分别讨论了模 12、模 24、模 60。现在给出适用于任意模值 N 的通解。

模 N 计数器通用设计：步骤 1：确定位宽

$$k = \lceil \log_2 N \rceil$$

步骤 2：确定清零条件

$$\text{清零} \iff cnt = N - 1$$

步骤 3：编写 VHDL

```
1  if rising_edge(clk) then
2    if cnt = (N-1) then
3      cnt <= (others => '0');
4    else
5      cnt <= cnt + 1;
6    end if;
7  end if;
```

23.2 为什么这个模板适用于任意 N

计数器的行为是确定的：

1. 状态序列： $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow N - 1 \rightarrow 0 \rightarrow 1 \rightarrow \dots$
2. 转移规则： $S_{next} = S_{current} + 1$ （除了在 $N - 1$ 处）

3. 在 $S = N - 1$ 处: $S_{next} = 0$

这个行为的硬件实现正好对应上述 VHDL 模板。

23.3 实例：模 7 计数器

1. $N = 7$
2. $k = \lceil \log_2 7 \rceil = 3$ (3 位, 最多表示 7)
3. 清零条件: $\text{cnt} = 6(110)$

状态序列: $000 \rightarrow 001 \rightarrow 010 \rightarrow 011 \rightarrow 100 \rightarrow 101 \rightarrow 110 \rightarrow 000 \rightarrow \dots$

```

1 signal cnt : std_logic_vector(2 downto 0);
2 ...
3 if rising_edge(clk) then
4   if cnt = "110" then      -- = 6
5     cnt <= "000";
6   else
7     cnt <= cnt + 1;
8   end if;
9 end if;
```

Listing 23.1: 模 7 计数器

23.4 实例：模 5 计数器

1. $N = 5$
2. $k = \lceil \log_2 5 \rceil = 3$
3. 清零条件: $\text{cnt} = 4(100)$

状态序列: $000 \rightarrow 001 \rightarrow 010 \rightarrow 011 \rightarrow 100 \rightarrow 000 \rightarrow \dots$

23.5 实例：模 100 计数器

1. $N = 100$
2. $k = \lceil \log_2 100 \rceil = 7$ ($2^6 = 64 < 100 < 2^7 = 128$)
3. 清零条件: $\text{cnt} = 99(1100011)$

23.6 边界情况分析

23.6.1 $N = 2^k$ 的情况

当 N 恰好是 2 的幂（如模 16： $N = 2^4$ ）：

- 不需要显式清零逻辑——自然溢出即可
- 但加上清零逻辑也能工作（只是多余）
- **考试注意：**模 2^N 计数器不需要 if cnt=N-1，自然溢出即可

23.6.2 $N = 1$ 的情况

模 1 计数器：只有状态 0。每个时钟周期都是 $0 \rightarrow 0 \rightarrow 0 \rightarrow \dots$ 但实际中没人设计模 1 计数器——无意义。

23.7 考试常见变式

1. 变式 1：给定模值 N ，写完整 VHDL

直接套用模板。唯一需要注意：位宽要正确（考 $\lceil \log_2 N \rceil$ ）

2. 变式 2：给定计数器电路图，判断模值

看图识别清零逻辑：清零条件是 $\text{cnt} = X$ ，则模值 $= X + 1$ 。

3. 变式 3：带使能的模 N 计数器

增加 enable 输入：enable=1 时计数，enable=0 时保持。

4. 变式 4：带加载初值的模 N 计数器

增加 load 输入和初值输入：load=1 时 $\text{cnt} = \text{初值}$ ，load=0 时正常计数。

5. 变式 5：递减计数器（倒计时）

$\text{Count} \leftarrow \text{Count} - 1$ ，到 0 时加载 $N-1$ 。

6. 变式 6：模 N 计数器级联（如模 60 = 模 10 × 模 6）

详见后续 BCD 计数器章节。

23.8 统一思考题

1. 设计模 13 计数器 ($N = 13$), 写出 VHDL。
2. 设计模 30 计数器 ($N = 30$), 写出 VHDL。
3. 设计模 9 计数器 ($N = 9$), 分析需要多少位寄存器, 写出 VHDL。

答案提示: 全部套用同一模板, 只改 $N-1$ 的值和位宽。

模 N 计数器秒杀法: 1. $k = \lceil \log_2 N \rceil$ 确定位宽 2. 清零条件 = $N-1$ 的二进制
3. 套用模板: `if cnt = N-1 then cnt <= 0; else cnt <= cnt+1;`
这就是所有模 N 计数器的答案。

23.9 变式详解

第二十四章 计数器——统一框架变式详解

24.1 变式 1：带使能的模 N 计数器

完整教学

【电路】在标准计数器上增加 en 控制。【VHDL】

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity modn_en_counter is
6      generic (N : integer := 10);
7      port (
8          clk    : in  std_logic;
9          rst_n  : in  std_logic;
10         en     : in  std_logic;
11         cnt    : out std_logic_vector(3 downto 0)
12     );
13 end modn_en_counter;
14
15 architecture behavioral of modn_en_counter is
16     signal cnt_reg : std_logic_vector(3 downto 0);
17 begin
18     process(clk, rst_n)
19     begin
20         if rst_n = '0' then
21             cnt_reg <= (others => '0');
22         elsif rising_edge(clk) then
23             if en = '1' then
24                 if cnt_reg = N-1 then
25                     cnt_reg <= (others => '0');
```



```

26         else
27             cnt_reg <= cnt_reg + 1;
28         end if;
29     end if;
30 end if;
31 end process;
32 cnt <= cnt_reg;
33 end behavioral;

```

【考点】en=0 时不赋值 → 保持。这是所有可控计数器的基本范式。

24.2 变式 2：带同步加载的模 N 计数器

完整教学

【功能】load=1 时 cnt=data_in。load=0 时正常计数。【优先级】load > 计数（if-else 中 load 在前 = 优先级高）。【VHDL】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity modn_load_counter is
6      generic (N : integer := 10);
7      port (
8          clk      : in  std_logic;
9          rst_n    : in  std_logic;
10         load     : in  std_logic;
11         data_in  : in  std_logic_vector(3 downto 0);
12         cnt      : out std_logic_vector(3 downto 0)
13     );
14 end modn_load_counter;
15
16 architecture behavioral of modn_load_counter is
17     signal cnt_reg : std_logic_vector(3 downto 0);
18 begin
19     process(clk, rst_n)
20     begin
21         if rst_n = '0' then
22             cnt_reg <= (others => '0');
23         elsif rising_edge(clk) then

```

```

24     if load = '1' then
25         cnt_reg <= data_in;
26     elsif cnt_reg = N-1 then
27         cnt_reg <= (others => '0');
28     else
29         cnt_reg <= cnt_reg + 1;
30     end if;
31 end if;
32 end process;
33 cnt <= cnt_reg;
34 end behavioral;

```

24.3 变式 3: 递增/递减双向计数器

完整教学

【VHDL】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity updown_counter is
6      generic (WIDTH : integer := 4);
7      port (
8          clk      : in  std_logic;
9          rst_n    : in  std_logic;
10         up       : in  std_logic;
11         cnt      : out std_logic_vector(WIDTH-1 downto 0)
12     );
13 end updown_counter;
14
15 architecture behavioral of updown_counter is
16     signal cnt_reg : std_logic_vector(WIDTH-1 downto 0);
17 begin
18     process(clk, rst_n)
19     begin
20         if rst_n = '0' then
21             cnt_reg <= (others => '0');
22         elsif rising_edge(clk) then

```

```

23     if up = '1' then
24         cnt_reg <= cnt_reg + 1;
25     else
26         cnt_reg <= cnt_reg - 1;
27     end if;
28 end if;
29 end process;
30 cnt <= cnt_reg;
31 end behavioral;

```

【递减自然溢出】 $0000 - 1 = 1111$ （4 位无符号），自动模 16 循环。

24.4 变式 4：倒计时计数器（预置 $\rightarrow 0 \rightarrow$ 预置）

完整教学

【VHDL】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity countdown_counter is
6      generic (PRE : integer := 9);
7      port (
8          clk    : in  std_logic;
9          rst_n  : in  std_logic;
10         cnt    : out std_logic_vector(3 downto 0)
11     );
12 end countdown_counter;
13
14 architecture behavioral of countdown_counter is
15     signal cnt_reg : unsigned(3 downto 0);
16 begin
17     process(clk, rst_n)
18     begin
19         if rst_n = '0' then
20             cnt_reg <= (others => '0');
21         elsif rising_edge(clk) then
22             if cnt_reg = 0 then
23                 cnt_reg <= to_unsigned(PRE, cnt_reg'length);

```

```
24         else
25             cnt_reg <= cnt_reg - 1;
26         end if;
27     end if;
28 end process;
29 cnt <= std_logic_vector(cnt_reg);
30 end behavioral;
```

24.5 变式 5：级联计数器——总模值 = 乘积

完整教学

【级联方式】低位 carry→高位 en。总 $M=M_1 \times M_2$ 。

【实例】模 8+模 5 级联 → 总模 40。模 60(6×10)BCD 级联。

【VHDL 关键】高位以低位 carry 为时钟使能——确保高位只在低位满一轮时才 +1。

24.6 变式 6：格雷码计数器

完整教学

【特点】相邻状态仅 1 位不同。适合 FIFO 指针、跨时钟域。【VHDL】case 枚举所有格雷码状态。【3 位格雷码序列】000-001-011-010-110-111-101-100-000。

第二十五章 模 10 BCD 计数器

25.1 BCD 计数器与二进制计数器的本质区别

到目前为止，我们讨论的计数器都是二进制计数器：

- 数是用**整个二进制数**表示的
- 模 12 计数器：4 位二进制（0000-1011），一个整体
- 模 60 计数器：6 位二进制（000000-111011），一个整体

BCD 计数器完全不同：

BCD 计数器的本质：BCD（Binary-Coded Decimal）计数器将**每个十进制位**单独用一个 4 位二进制计数器来表示。

- 每个十进制位 = 一个独立的模 10 计数器（0-9，4 位）
- 个位计到 9 后，产生进位给十位
- 十位每收到一个进位，加 1

25.2 一位 BCD 计数器 = 模 10 计数器

一位十进制数字（0-9）需要一个模 10 计数器。

1. 模值 N ： $N = 10$
2. 寄存器位宽： $k = 4$ ($\lceil \log_2 10 \rceil = 4$)
3. 状态序列： $0000 \rightarrow 0001 \rightarrow 0010 \rightarrow \dots \rightarrow 1001(9) \rightarrow 0000(0)$
4. 清零条件： $\text{cnt} = 9(1001)$

25.3 逐拍状态分析

模 10 BCD 计数器:

时钟拍	上升沿后 Q	十进制	进位
1	0001	1	0
2	0010	2	0
3	0011	3	0
4	0100	4	0
5	0101	5	0
6	0110	6	0
7	0111	7	0
8	1000	8	0
9	1001	9	0
10	0000	0	1
11	0001	1	0

关键观察:模 10 计数器在状态 9(1001) 之后强制归零——而 1001+1 本应是 1010(10)。所以状态 1010 到 1111(10-15) 永远不会出现。

25.4 VHDL 实现

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity bcd_counter is
6      port (
7          clk    : in  std_logic;
8          rst_n  : in  std_logic;
9          count  : out std_logic_vector(3 downto 0);
10         carry  : out std_logic  -- 进位到十位
11     );
12 end bcd_counter;
13
14 architecture behavioral of bcd_counter is
15     signal cnt : std_logic_vector(3 downto 0);
16 begin
17     process(clk, rst_n)
18     begin
19         if rst_n = '0' then

```

```
20      cnt <= (others => '0');
21      elsif rising_edge(clk) then
22          if cnt = "1001" then      -- = 9
23              cnt <= (others => '0');
24          else
25              cnt <= cnt + 1;
26          end if;
27      end if;
28  end process;
29
30  count <= cnt;
31  carry <= '1' when cnt = "1001" else '0';
32  end behavioral;
```

Listing 25.1: 模 10 BCD 计数器（个位）

25.5 BCD 模 10 vs 二进制模 10

特性	二进制模 10	BCD 模 10
位宽	4 位	4 位
状态数	10 个	10 个
表示方式	二进制（整体）	一个十进制位
清零条件	cnt=1001(9)	cnt=1001(9)
本质	完全相同——都是模 10 计数器！	

25.6 BCD 模 10 vs 多位 BCD 计数器

单个 BCD 计数器 = 模 10 计数器。

真正的 BCD 计数器是指多位 BCD 级联：个位 + 十位 + 百位 +...

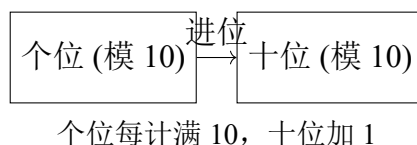
每个十进制位都是独立的模 10 计数器，个位向十位进位，十位向百位进位。

这将在下一章详细展开。

第二十六章 模 24 BCD 计数器与模 60 BCD 计数器

26.1 BCD 级联的基本思想

多位 BCD 计数器 = 多个独立的模 10 计数器级联。



级联的本质：低位计数器每完成一轮（回到 0），给高位计数器发一个“进位”脉冲，高位计数器加 1。

26.2 模 24 BCD 计数器

模 24 BCD = 十位（模 2：0-2，计到 23 后归零）+ 个位（模 10：0-9）
等一下——这里有个问题。

26.2.1 两种实现方案

方案 A：纯 BCD 级联（个位模 10 + 十位模 3）

个位：模 10（0-9），满 9 进位到十位
十位：收到进位 +1，但十位只在 0-2 之间循环（模 3）

清零条件：十位 = 2 且个位 = 3（即 23），下一拍归零
注意不是十位 = 2 且个位 = 9（即 29）——那会得到 30 个状态！

方案 B：两位 BCD 整体计数，满 24 清零

两位合并为 8 位（十位 4 位 + 个位 4 位），整体 +1，到 24(0010 0100) 时清零。

26.2.2 方案 A：BCD 级联实现


```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.std_logic_unsigned.all;
4
5 entity bcd_counter_mod24 is
6     port (
7         clk      : in  std_logic;
8         rst_n     : in  std_logic;
9         tens      : out std_logic_vector(3 downto 0);
10        units     : out std_logic_vector(3 downto 0)
11    );
12 end bcd_counter_mod24;
13
14 architecture behavioral of bcd_counter_mod24 is
15     signal units_cnt : std_logic_vector(3 downto 0);
16     signal tens_cnt  : std_logic_vector(3 downto 0);
17     signal units_carry : std_logic;
18 begin
19     process(clk, rst_n)
20     begin
21         if rst_n = '0' then
22             units_cnt <= (others => '0');
23             tens_cnt  <= (others => '0');
24             units_carry <= '0';
25
26         elsif rising_edge(clk) then
27             -- 检测满24清零
28             if tens_cnt = "0010" and units_cnt = "0011" then -- 23
29                 units_cnt <= (others => '0');
30                 tens_cnt  <= (others => '0');
31                 units_carry <= '0';
32             else
33                 -- 个位计数
34                 if units_cnt = "1001" then -- 9
35                     units_cnt <= (others => '0');
36                     units_carry <= '1';
37                 else
38                     units_cnt <= units_cnt + 1;
39                     units_carry <= '0';
40                 end if;
41
42                 -- 十位计数（收到进位时）
43                 if units_carry = '1' then
44                     tens_cnt <= tens_cnt + 1;
```

```

45         end if;
46     end if;
47 end if;
48 end process;
49
50 tens    <= tens_cnt;
51 units  <= units_cnt;
52 end behavioral;

```

Listing 26.1: 模 24 BCD 计数器（方案 A：级联）

26.2.3 方案 B：统一计数

```

1  -- 更简洁的实现：直接在同一个 process 中处理
2  process(clk, rst_n)
3  begin
4      if rst_n = '0' then
5          units_cnt <= (others => '0');
6          tens_cnt  <= (others => '0');
7      elsif rising_edge(clk) then
8          -- 先判断是否清零
9          if tens_cnt = "0010" and units_cnt = "0011" then -- 23
10             units_cnt <= (others => '0');
11             tens_cnt  <= (others => '0');
12             -- 个位进位判断
13             elsif units_cnt = "1001" then -- 9
14                 units_cnt <= (others => '0');
15                 tens_cnt  <= tens_cnt + 1;
16             else
17                 units_cnt <= units_cnt + 1;
18             end if;
19         end if;
20     end process;

```

Listing 26.2: 模 24 BCD 计数器（方案 B：统一进位判断）

26.3 模 60 BCD 计数器

模 60 BCD = 十位（模 6：0-5）+ 个位（模 10：0-9）

清零条件：十位 = 5 且个位 = 9（即 59）

```

1  library ieee;

```

```
2 use ieee.std_logic_1164.all;
3 use ieee.std_logic_unsigned.all;
4
5 entity bcd_counter_mod60 is
6     port (
7         clk    : in  std_logic;
8         rst_n   : in  std_logic;
9         tens    : out std_logic_vector(3 downto 0);
10        units   : out std_logic_vector(3 downto 0)
11    );
12 end bcd_counter_mod60;
13
14 architecture behavioral of bcd_counter_mod60 is
15     signal units_cnt : std_logic_vector(3 downto 0);
16     signal tens_cnt  : std_logic_vector(3 downto 0);
17 begin
18     process(clk, rst_n)
19     begin
20         if rst_n = '0' then
21             units_cnt <= (others => '0');
22             tens_cnt  <= (others => '0');
23         elsif rising_edge(clk) then
24             -- 满59清零
25             if tens_cnt = "0101" and units_cnt = "1001" then -- 59
26                 units_cnt <= (others => '0');
27                 tens_cnt  <= (others => '0');
28                 -- 个位满9进位
29             elsif units_cnt = "1001" then
30                 units_cnt <= (others => '0');
31                 tens_cnt  <= tens_cnt + 1;
32             else
33                 units_cnt <= units_cnt + 1;
34             end if;
35         end if;
36     end process;
37
38     tens    <= tens_cnt;
39     units   <= units_cnt;
40 end behavioral;
```

Listing 26.3: 模 60 BCD 计数器

26.4 BCD 级联的通用模板

多位 BCD 计数器通用模板：个位计数器：

- 模 10：计到 9 后清零，产生进位

十位计数器：

- 收到进位后 +1
- 有自己独立的模值限制（取决于应用）
- 模 24：十位模 3（0-2），满 23 清零
- 模 60：十位模 6（0-5），满 59 清零

整体清零条件（同时清除个位和十位）：

- 模 24：十位 =2 且个位 =3
- 模 60：十位 =5 且个位 =9
- 模 12：十位 =1 且个位 =1（如果要做成 12 的 BCD）

26.5 数字钟表的完整结构

一个数字钟表 = 三个 BCD 计数器级联：



秒满 60 → 分 +1, 分满 60 → 时 +1, 时满 24 → 清零

这正是模 24 和模 60 BCD 计数器的典型应用场景。

26.6 计数器部分总结

计数器统一框架总结：所有计数器本质上只有一个公式：

在时钟上升沿： $cnt \leftarrow cnt + 1$ （如果 $cnt = N - 1$ 则 $cnt \leftarrow 0$ ）

- 二进制计数器：一个整体二进制数（4/5/6 位）
- BCD 计数器：每个十进制位独立（个位 + 十位各 4 位）
- 区别：表示方式不同，但核心转移逻辑完全相同
- 模 2^N 计数器：唯一不需要显式清零的（自然溢出）
- 所有其他模值：都需要“if $cnt=N-1$ then $cnt \leftarrow 0$ ”的显式清零

第四部分

分频器体系

第二十七章 用计数器设计分频器

27.1 什么是分频

分频：将输入时钟信号的频率降低。

- 输入： f_{clk} （例如 100 MHz）
- 输出： $f_{out} = f_{clk}/M$ （例如 50 MHz，25 MHz，1 Hz 等）
- M 称为分频比

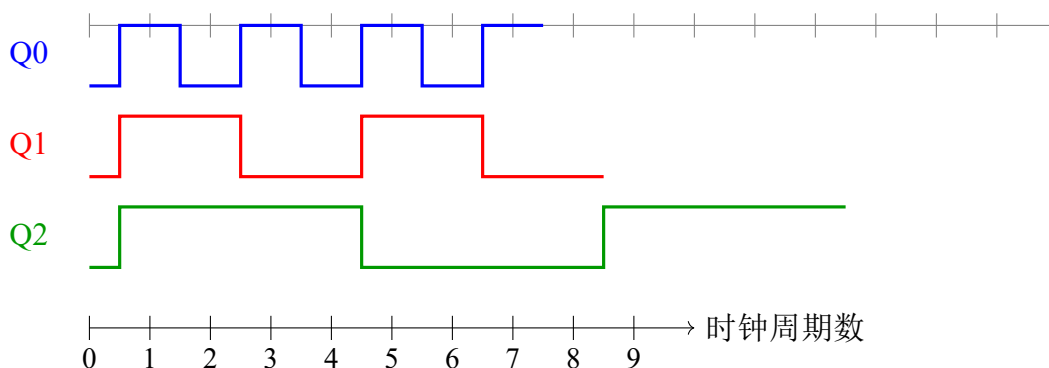
分频器的本质：分频器 = 计数器 + 利用计数器的特定位作为输出

分频器不需要额外的硬件——计数器本身就是分频器！

27.2 为什么计数器能够分频：从状态变化过程理解

27.2.1 回顾 4 位计数器的状态序列

让我们重新观察 4 位二进制计数器的 Q0、Q1、Q2、Q3 的波形：



27.2.2 关键观察

- **Q0：**每 1 个时钟周期翻转一次 $\rightarrow$ 周期 = $2T_{clk}$ $\rightarrow$ 频率 = $f_{clk}/2$

- **Q1**: 每 2 个时钟周期翻转一次 $\rightarrow$ 周期 = $4T_{clk}$ $\rightarrow$ 频率 = $f_{clk}/4$
- **Q2**: 每 4 个时钟周期翻转一次 $\rightarrow$ 周期 = $8T_{clk}$ $\rightarrow$ 频率 = $f_{clk}/8$
- **Q3**: 每 8 个时钟周期翻转一次 $\rightarrow$ 周期 = $16T_{clk}$ $\rightarrow$ 频率 = $f_{clk}/16$

27.3 逐拍分析：为什么 Q0 天然二分频

Q0 为什么二分频： 观察 Q0 在每一拍的变化：

时钟拍	计数器值 (Q3Q2Q1Q0)	Q0	Q0 相对于上一拍
0	0000	0	—
1	0001	1	翻转 (0 $\rightarrow$ 1)
2	0010	0	翻转 (1 $\rightarrow$ 0)
3	0011	1	翻转 (0 $\rightarrow$ 1)
4	0100	0	翻转 (1 $\rightarrow$ 0)

原因： Q0 是二进制计数的最低位。每次 +1 时：

- 如果当前 Q0=0，则 +1 后 =1（翻转）
- 如果当前 Q0=1，则 +1 后 =0 且产生进位（翻转）

无论什么情况，**Q0 一定翻转**。

因此：Q0 每 2 个时钟周期完成一次 0 $\rightarrow$ 1 $\rightarrow$ 0 的完整循环，频率恰好是时钟的一半。

27.4 逐拍分析：为什么 Q1 天然四分频

Q1 为什么四分频：

时钟拍	计数器值	Q1	Q1 变化原因
0	0000	0	初始
1	0001	0	Q0 未产生进位给 Q1
2	0010	1	Q0=1, +1 使 Q0 翻转为 0, 进位使 Q1 翻转为 1
3	0011	1	Q0 从 0 翻转 1, 无进位, Q1 不变
4	0100	0	Q0=1, +1 进位, Q1 从 1 翻转为 0

原因： Q1 只有在 Q0 产生进位时才翻转。Q0 每 2 拍产生一次进位，所以 Q1 每 4 拍完成一个 0 $\rightarrow$ 1 $\rightarrow$ 0 的循环。

27.5 通用规律

分频规律：在 N 位二进制计数器中：

- Q_0 频率 = $f_{clk}/2$ (2 分频)
- Q_1 频率 = $f_{clk}/4$ (4 分频)
- Q_2 频率 = $f_{clk}/8$ (8 分频)
- Q_k 频率 = $f_{clk}/2^{k+1}$ (2^{k+1} 分频)

这不是需要记忆的公式——这是计数器二进制表示的自然结果。

27.6 偶数分频

偶数分频是最简单的分频。用一个计数器，计到 $M/2 - 1$ 时翻转输出。

27.6.1 6 分频器设计

分频比 $M=6$ ，即每 6 个时钟周期，输出完成一个完整周期（3 个时钟高电平 + 3 个时钟低电平）。

方案：使用模 6 计数器（0→1→2→3→4→5→0），当计数值 <3 时输出 0， ≥ 3 时输出 1。

6 分频器逐拍分析：

时钟拍	计数器	输出	说明
0	0	0	初始
1	1	0	$\text{cnt} < 3$
2	2	0	$\text{cnt} < 3$
3	3	1	$\text{cnt} \geq 3$ （占空比 50%，3 拍高）
4	4	1	$\text{cnt} \geq 3$
5	5	1	$\text{cnt} \geq 3$
6	0	0	归零，开始新周期

输出周期 = 6 个时钟周期 → 频率 = $f_{clk}/6$

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.std_logic_unsigned.all;
4
5 entity div6 is
```

```

6  port (
7      clk    : in  std_logic;
8      rst_n  : in  std_logic;
9      clk_div6 : out std_logic
10 );
11 end div6;
12
13 architecture behavioral of div6 is
14     signal cnt : std_logic_vector(2 downto 0); -- 3位, 够表示0-5
15 begin
16     process(clk, rst_n)
17     begin
18         if rst_n = '0' then
19             cnt <= (others => '0');
20         elsif rising_edge(clk) then
21             if cnt = "101" then -- = 5
22                 cnt <= (others => '0');
23             else
24                 cnt <= cnt + 1;
25             end if;
26         end if;
27     end process;
28
29     -- cnt < 3 时输出0, cnt >= 3 时输出1 (50%占空比)
30     clk_div6 <= '0' when cnt < "011" else '1';
31 end behavioral;

```

Listing 27.1: 6 分频器 VHDL 实现

27.7 奇数分频

奇数分频比偶数分频复杂。以 3 分频为例。

27.7.1 3 分频器设计

要求: $f_{out} = f_{clk}/3$, 占空比接近 50% (但不可能完全 50%, 因为奇数个时钟周期)。

方案: 使用上升沿和下降沿分别产生两个信号, 然后组合。

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity div3 is

```

```
6   port (
7       clk      : in  std_logic;
8       rst_n    : in  std_logic;
9       clk_div3 : out std_logic
10  );
11  end div3;
12
13  architecture behavioral of div3 is
14      signal cnt : std_logic_vector(1 downto 0);
15      signal clk_pos, clk_neg : std_logic;
16  begin
17      -- 上升沿计数
18      process(clk, rst_n)
19      begin
20          if rst_n = '0' then
21              cnt <= (others => '0');
22          elsif rising_edge(clk) then
23              if cnt = "10" then -- = 2
24                  cnt <= (others => '0');
25              else
26                  cnt <= cnt + 1;
27              end if;
28          end if;
29      end process;
30
31      -- 上升沿产生的信号
32      process(clk, rst_n)
33      begin
34          if rst_n = '0' then
35              clk_pos <= '0';
36          elsif rising_edge(clk) then
37              if cnt = 0 then
38                  clk_pos <= '1';
39              elsif cnt = 1 then
40                  clk_pos <= '0';
41              end if;
42          end if;
43      end process;
44
45      -- 下降沿产生的信号
46      process(clk, rst_n)
47      begin
48          if rst_n = '0' then
49              clk_neg <= '0';
50          elsif falling_edge(clk) then
```

```

51     if cnt = 0 then
52         clk_neg <= '1';
53     elsif cnt = 1 then
54         clk_neg <= '0';
55     end if;
56 end if;
57 end process;
58
59 clk_div3 <= clk_pos or clk_neg;
60 end behavioral;

```

Listing 27.2: 3 分频器 VHDL 实现（占空比 50%）

27.8 任意整数分频的统一设计流程

任意整数分频设计流程：步骤 1：确定分频比 M

步骤 2：设计模 M 计数器

$$cnt : 0 \rightarrow 1 \rightarrow 2 \rightarrow \cdots \rightarrow M - 1 \rightarrow 0$$

步骤 3：确定输出逻辑

偶数分频（ M 为偶数）：

- 在 $cnt = 0$ 到 $M/2 - 1$ 时输出低电平
- 在 $cnt = M/2$ 到 $M - 1$ 时输出高电平
- 占空比恰好 50%

奇数分频（ M 为奇数）：

- 用上升沿 + 下降沿双计数器产生两个信号
- $clk_out = clk_pos \text{ OR } clk_neg$
- 得到接近 50% 占空比的输出

27.9 考试常见变式

1. 变式 1：用计数器直接取 Q_n 输出实现 2^n 分频

最简单： $Q_0=2$ 分频， $Q_1=4$ 分频， $Q_2=8$ 分频

2. 变式 2：设计任意偶数分频器

套用统一模板， $M/2$ 作为阈值。

3. 变式 3：设计占空比可调的分频器

改变输出翻转的阈值点。例如对于 $M=10$ 分频，如果想让占空比为 20%，则在 $\text{cnt}=1$ 时变高， $\text{cnt}=3$ 时变低。

4. 变式 4：多级分频器级联（分频比相乘）

先 2 分频 $\rightarrow$ 再 5 分频 = 10 分频。总 $M = M_1 \times M_2$ 。

5. 变式 5：用分频器产生特定频率

如：100MHz 时钟，产生 1Hz 信号 $\rightarrow M = 100,000,000$ 。需要计数器能计到一亿。

6. 变式 6：秒脉冲发生器

$M = 50,000,000$ （50MHz 时钟 $\rightarrow$ 1Hz 秒脉冲）。这是一个很大的计数器。

27.10 分频器与计数器的关系总结

核心关系：分频器 = 计数器 + 利用输出位

- 计数器提供“每 M 个时钟循环一次”的机制
- 分频器利用了这个循环来产生低频信号
- 计数器中的每一位 Q_k 天然就是 2^{k+1} 分频输出
- 任意分频比 = 模 M 计数器 + 输出比较逻辑

分频器不需要独立的电路——它是计数器的一种应用方式。

27.11 变式详解

第二十八章 分频器——全部变式详解

28.1 变式 1: 直接取 Qn 实现 2^n 分频

完整教学

【原理】 Q_k 每 2^{k+1} 拍完成一次 $0 \rightarrow 1 \rightarrow 0$ 循环。

【VHDL——一行搞定】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.std_logic_unsigned.all;
4
5 entity div_power2 is
6     generic (WIDTH : integer := 4);
7     port (
8         clk      : in  std_logic;
9         rst_n     : in  std_logic;
10        clk_div2  : out std_logic;
11        clk_div4  : out std_logic;
12        clk_div8  : out std_logic
13    );
14 end div_power2;
15
16 architecture behavioral of div_power2 is
17     signal cnt : std_logic_vector(WIDTH-1 downto 0);
18 begin
19     process(clk, rst_n)
20     begin
21         if rst_n = '0' then
22             cnt <= (others => '0');
23         elsif rising_edge(clk) then
24             cnt <= cnt + 1;
25         end if;
26     end process;
27     clk_div2 <= cnt(0);    -- 2分频
```

```
28   clk_div4 <= cnt(1);    -- 4分频
29   clk_div8 <= cnt(2);    -- 8分频
30 end behavioral;
```

【优点】不需要任何额外逻辑。计数器本身就是分频器。

28.2 变式 2：偶数分频（任意 M）

完整教学

【统一模板】

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity even_divider is
6      generic (M : integer := 6);
7      port (
8          clk      : in  std_logic;
9          rst_n     : in  std_logic;
10         clk_out  : out std_logic
11     );
12 end even_divider;
13
14 architecture behavioral of even_divider is
15     signal cnt : std_logic_vector(3 downto 0);
16 begin
17     process(clk, rst_n)
18     begin
19         if rst_n = '0' then
20             cnt <= (others => '0');
21         elsif rising_edge(clk) then
22             if cnt = M-1 then
23                 cnt <= (others => '0');
24             else
25                 cnt <= cnt + 1;
26             end if;
27         end if;
28     end process;
29     clk_out <= '1' when cnt >= M/2 else '0'; -- 50%占空比
```

```
30 end behavioral;
```

28.3 变式 3：奇数分频（双沿法）

完整教学

【原理】上升沿 + 下降沿各产生一个信号 → OR 组合。得到接近 50% 占空比的奇数分频。【详见解析册题 6】

28.4 变式 4：占空比可调分频器

完整教学

【原理】改变输出翻转的阈值。占空比 = 高拍数/M。

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity duty_divider is
6      generic (M : integer := 10; DUTY : integer := 3);
7      port (
8          clk      : in  std_logic;
9          rst_n    : in  std_logic;
10         clk_out   : out std_logic
11     );
12 end duty_divider;
13
14 architecture behavioral of duty_divider is
15     signal cnt : std_logic_vector(3 downto 0);
16 begin
17     process(clk, rst_n)
18     begin
19         if rst_n = '0' then
20             cnt <= (others => '0');
21         elsif rising_edge(clk) then
22             if cnt = M-1 then
23                 cnt <= (others => '0');
24             else
```



```
25     cnt <= cnt + 1;  
26     end if;  
27     end if;  
28 end process;  
29 clk_out <= '1' when cnt < DUTY else '0';  
30 end behavioral;
```

28.5 变式 5：多级分频器（分频比相乘）

完整教学

【原理】M1 分频 → M2 分频 = 总 M1×M2 分频。【注意】后级以前级输出为时钟 → 跨时钟域风险。FPGA 中推荐用使能信号而非门控时钟。

28.6 变式 6：秒脉冲发生器

完整教学

【问题】50MHz→1Hz。M=50,000,000。需 26 位计数器。【VHDL】同偶数分频模板，M=50000000。

第五部分

序列发生器体系

第二十九章 用计数器设计序列发生器

29.1 什么是序列

序列：一组按照预定顺序循环输出的二进制值。

例子：0110 0110 0110 ...（重复输出 0, 1, 1, 0）

序列发生器的本质：序列发生器 = 产生周期性重复的二进制序列的电路

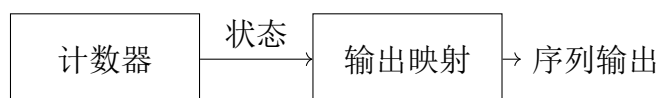
$$\text{输出}(t) = \text{序列}[\text{内部状态}(t)]$$

29.2 为什么计数器可以产生序列

计数器有一个天然能力：它按照固定的顺序经过所有状态。

序列发生器利用这一点：

1. 计数器按顺序经过状态
2. 每个状态对应序列中的一个输出值
3. 输出逻辑 = 状态到输出的映射



计数器提供”顺序”，映射提供”每个状态的输出值”

29.3 方案 1：计数器 + 组合逻辑映射

29.3.1 设计序列：0110 0110 0110 ...

序列长度 $L=4$ ，重复：0, 1, 1, 0。

1. 使用模 4 计数器（状态：0, 1, 2, 3, 0, 1, ...）

2. 输出映射表:

计数器状态	输出
0(00)	0
1(01)	1
2(10)	1
3(11)	0

逻辑表达式: $output = \overline{Q_1} \cdot Q_0 + Q_1 \cdot \overline{Q_0}$ (即 $Q_1 \oplus Q_0$)

```

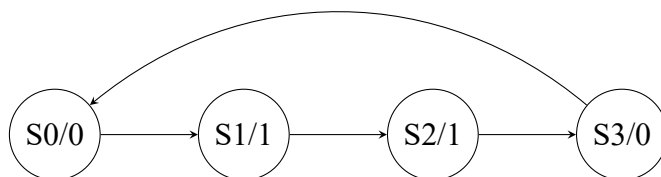
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity seq_gen_0110 is
6      port (
7          clk    : in  std_logic;
8          rst_n  : in  std_logic;
9          seq_out : out std_logic
10     );
11 end seq_gen_0110;
12
13 architecture rtl of seq_gen_0110 is
14     signal cnt : std_logic_vector(1 downto 0); -- 模4计数器
15 begin
16     -- 模4计数器
17     process(clk, rst_n)
18     begin
19         if rst_n = '0' then
20             cnt <= (others => '0');
21         elsif rising_edge(clk) then
22             if cnt = "11" then
23                 cnt <= (others => '0');
24             else
25                 cnt <= cnt + 1;
26             end if;
27         end if;
28     end process;
29
30     -- 输出映射: 状态0和3输出0, 状态1和2输出1
31     -- 即: Q1 XOR Q0
32     seq_out <= cnt(1) xor cnt(0);
33 end rtl;

```

Listing 29.1: 序列 0110 发生器 (计数器 + 组合逻辑)

29.4 方案 2：状态机型序列发生器

同样的序列可以用状态机实现：



```

1 architecture fsm of seq_gen_0110 is
2     type state_type is (S0, S1, S2, S3);
3     signal state : state_type;
4 begin
5     process(clk, rst_n)
6     begin
7         if rst_n = '0' then
8             state <= S0;
9         elsif rising_edge(clk) then
10            case state is
11                when S0 => state <= S1;
12                when S1 => state <= S2;
13                when S2 => state <= S3;
14                when S3 => state <= S0;
15            end case;
16        end if;
17    end process;
18
19    -- 输出逻辑
20    seq_out <= '0' when (state = S0 or state = S3) else '1';
21 end fsm;
  
```

Listing 29.2: 序列 0110 发生器（状态机版本）

29.5 方案 3：移位寄存器型序列发生器

有些序列恰好可以用移位寄存器 + 反馈产生。例如伪随机序列（LFSR）。这将在第六部分（移位寄存器）和第七部分（环形计数器）中详细展开。

29.6 方案 4：ROM/查找表型序列发生器

适用于长序列：

- 将整个序列预存在 ROM 中
- 计数器作为 ROM 的地址
- ROM 输出 = 序列值

29.7 方案比较

方案	适用场景	优点	缺点
计数器 + MUX	序列有规律	资源少	需要推导逻辑
状态机	任意序列	通用	状态多时复杂
移位寄存器	LFSR/循环型	结构规整	仅限特定类型
ROM/查找表	长序列	通用	资源多（需要 ROM）

29.8 考试常见变式

1. 变式 1：设计任意周期性的序列

例：产生序列 10110 10110 ...（长度 5）解：模 5 计数器 + 状态到输出映射表

2. 变式 2：多路输出序列

同时输出多个相关序列（如正交序列）

3. 变式 3：伪随机序列（LFSR）

用移位寄存器 + 反馈产生看似随机的序列。这是通信系统中的重要应用。

4. 变式 4：可配置的序列发生器

序列可以动态改变（通过改变映射表或 ROM 内容）

序列发生器秒杀法：1. 确定序列长度 L 2. 设计模 L 计数器 3. 建立状态 → 输出映射表（真值表）4. 推导输出逻辑（或用 case 直接写）
计数器 + case 语句是最通用的解法，适用于任何序列。

29.9 变式详解

第三十章 序列发生器——全部变式详解

30.1 变式 1：任意周期序列（计数器 + 映射表）

完整教学

【通用方案】模 L 计数器 + 状态 → 输出映射表。【例题】产生序列 10110(长度 5)。
模 5 计数器 + case 映射：

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.std_logic_unsigned.all;
4
5 entity seq_gen_10110 is
6     port (
7         clk      : in  std_logic;
8         rst_n     : in  std_logic;
9         seq_out   : out std_logic
10    );
11 end seq_gen_10110;
12
13 architecture behavioral of seq_gen_10110 is
14     signal cnt : std_logic_vector(2 downto 0);
15 begin
16     process(clk, rst_n)
17     begin
18         if rst_n = '0' then
19             cnt <= (others => '0');
20         elsif rising_edge(clk) then
21             if cnt = "100" then
22                 cnt <= (others => '0');
23             else
24                 cnt <= cnt + 1;
25             end if;
26         end if;
27     end process;
```

```

28
29 process(cnt)
30 begin
31     case cnt is
32         when "000" => seq_out <= '1';
33         when "001" => seq_out <= '0';
34         when "010" => seq_out <= '1';
35         when "011" => seq_out <= '1';
36         when "100" => seq_out <= '0';
37         when others => seq_out <= '0';
38     end case;
39 end process;
40 end behavioral;

```

【适用范围】任意序列。长度 $L \rightarrow$ 模 L 计数器。

30.2 变式 2：状态机型序列发生器

完整教学

【方案】每个状态输出一个序列值。状态转移 = 序列顺序。【优点】状态名有意义（如“S_1”、“S_0”、“S_1”、“S_1”、“S_0”）。但状态多时代码冗长。【比较】推荐用计数器+case——代码更简洁。

30.3 变式 3：移位寄存器 +LFSR 发生器

完整教学

【LFSR】线性反馈移位寄存器。反馈 = 选择位的 XOR。【应用】伪随机序列 (PRBS)、CRC、扰码。【4 位 LFSR(X_4+X_3+1)】反馈 = $Q_3 \text{ xor } Q_2$ 。产生 15 个伪随机状态。【注意】全 0 状态非法——LFSR 会锁死在 0000。必须初始化为非 0 值。

30.4 变式 4：ROM/查找表型序列发生器

完整教学

【方案】预存整个序列在 ROM 中。计数器 = 地址。【优点】任意序列、可重配置。适合长序列。【VHDL】


```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4  use ieee.numeric_std.all;
5
6  entity rom_seq_gen is
7      port (
8          clk      : in  std_logic;
9          rst_n    : in  std_logic;
10         seq_out   : out std_logic_vector(3 downto 0)
11     );
12 end rom_seq_gen;
13
14 architecture behavioral of rom_seq_gen is
15     signal cnt : std_logic_vector(3 downto 0);
16     type rom_type is array (0 to 15) of std_logic_vector(3 downto 0)
17     ;
18     constant SEQ_ROM : rom_type := (
19         x"1", x"0", x"1", x"1", x"0",
20         x"0", x"0", x"0", x"0", x"0",
21         x"0", x"0", x"0", x"0", x"0", x"0"
22     );
23 begin
24     process(clk, rst_n)
25     begin
26         if rst_n = '0' then
27             cnt <= (others => '0');
28         elsif rising_edge(clk) then
29             cnt <= cnt + 1;
30         end if;
31     end process;
32     seq_out <= SEQ_ROM(to_integer(unsigned(cnt)));
33 end behavioral;
```

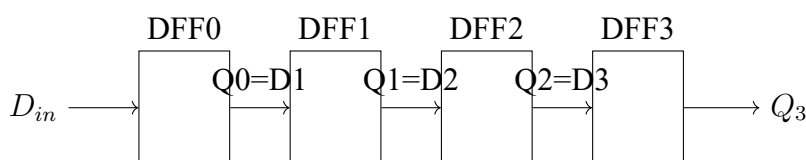
第六部分

移位寄存器体系

第三十一章 移位寄存器

31.1 先建立数据流动的思维模型

回顾 D 触发器：一个 D 触发器在时钟上升沿采样 D 输入，输出到 Q。
现在把多个 D 触发器首尾连接：



每个 D 触发器的 Q 接到下一个 D 触发器的 D

移位寄存器的核心结构：移位寄存器 = N 个 D 触发器串联

$$\text{DFF}_k \text{ 的 } Q \rightarrow \text{DFF}_{k+1} \text{ 的 } D$$

在时钟上升沿，每个 DFF 采样前一级 DFF 的 Q 输出。结果：所有数据同时向右移动一位。

31.2 逐拍分析：数据如何移动

这是全书最重要的状态分析之一。请仔细跟着每一拍。

31.2.1 初始条件

假设 4 个 D 触发器初始值：Q3Q2Q1Q0 = 0000

串行输入 D_{in} 序列：1, 0, 1, 1, 0, ...

（每个时钟周期，一个新的 D_{in} 值被送入最左边的 DFF0）

31.2.2 时钟第 0 拍（初始状态）

- 寄存器内容：Q3=0, Q2=0, Q1=0, Q0=0 $\rightarrow$ [0 0 0 0]

- 当前 $D_{in} = 1$ （准备好，等待下一个上升沿）

31.2.3 时钟第 1 拍上升沿

移位寄存器——时钟第 1 拍：上升沿到达时发生的事情（所有 DFF 同时动作）：

DFF	D 输入 (上升沿前)	上升沿后 Q	DFF 动作
DFF0	$D_{in}=1$	$Q0=1$	采样外部输入 1
DFF1	$Q0=0$	$Q1=0$	采样前一拍 $Q0$ 的值 (0)
DFF2	$Q1=0$	$Q2=0$	采样前一拍 $Q1$ 的值 (0)
DFF3	$Q2=0$	$Q3=0$	采样前一拍 $Q2$ 的值 (0)

上升沿后寄存器内容：**[0 0 0 1]**（1 进入最右端）

原来在 $Q0$ 的 0 移到了 $Q1$ ，原来在 $Q1$ 的 0 移到了 $Q2$ ，原来在 $Q2$ 的 0 移到了 $Q3$ 。 $Q3$ 原来的值 (0) 被”挤出去”了——这就是串行输出。

31.2.4 时钟第 2 拍上升沿

现在 $D_{in} = 0$

移位寄存器——时钟第 2 拍：

DFF	D 输入 (上升沿前)	上升沿后 Q	DFF 动作
DFF0	$D_{in}=0$	$Q0=0$	采样外部输入 0
DFF1	$Q0=1$	$Q1=1$	采样 $Q0$ 的值 (1) → 1 右移
DFF2	$Q1=0$	$Q2=0$	采样 $Q1$ 的值 (0) → 0 右移
DFF3	$Q2=0$	$Q3=0$	采样 $Q2$ 的值 (0) → 0 右移

上升沿后寄存器内容：**[0 0 1 0]**（之前 $Q0$ 的 1 现在移到了 $Q1$ ）

31.2.5 时钟第 3 拍上升沿

现在 $D_{in} = 1$

移位寄存器——时钟第 3 拍：

DFF	D 输入 (上升沿前)	上升沿后 Q	说明
DFF0	$D_{in}=1$	$Q0=1$	新数据进入
DFF1	$Q0=0$	$Q1=0$	
DFF2	$Q1=1$	$Q2=1$	1 继续右移
DFF3	$Q2=0$	$Q3=0$	

上升沿后寄存器内容：**[0 1 0 1]**（可以看到 1 在向右移动！）

31.2.6 时钟第 4 拍上升沿

现在 $D_{in} = 1$

上升沿后寄存器内容: [1 0 1 1]

31.2.7 完整数据移动轨迹

时钟拍	D_{in}	Q3	Q2	Q1	Q0
0 (初始)	—	0	0	0	0
1	1	0	0	0	1
2	0	0	0	1	0
3	1	0	1	0	1
4	1	1	0	1	1
5	0	0	1	1	0

观察数据流：每个 1 从 D_{in} 进入，经过 4 个时钟周期后从 Q3 串行输出。
就像水管中的水流——从一端进入，经过管道，从另一端流出。

31.3 核心概念总结

1. 串行输入 (Serial In): 数据从最左边的 DFF 一个 bit 一个 bit 地进入
2. 串行输出 (Serial Out): 数据从最右边的 DFF 一个 bit 一个 bit 地流出
3. 并行输出 (Parallel Out): 所有 Q 值同时观察——看到”管道”中当前的内容
4. 并行输入 (Parallel Load): 直接设置所有 DFF 的值 (跳过程移位过程)

31.4 单向移位寄存器: VHDL 实现

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity shift_reg_right is
5     port (
6         clk      : in  std_logic;
7         rst_n     : in  std_logic;
8         si        : in  std_logic;  -- 串行输入
9         so        : out std_logic;  -- 串行输出
10        po        : out std_logic_vector(3 downto 0)  -- 并行输出
11    );

```

```

12 end shift_reg_right;
13
14 architecture behavioral of shift_reg_right is
15     signal reg : std_logic_vector(3 downto 0);
16 begin
17     process(clk, rst_n)
18     begin
19         if rst_n = '0' then
20             reg <= (others => '0');
21         elsif rising_edge(clk) then
22             reg <= si & reg(3 downto 1); -- 右移：新数据进最高位
23         end shift;
24     end process;
25
26     so <= reg(0); -- 串行输出 = 最低位
27     po <= reg;   -- 并行输出 = 整个寄存器
28 end behavioral;

```

Listing 31.1: 4 位右移移位寄存器（串入串出/并出）

关键语句: `reg <= si & reg(3 downto 1)`

这表示:

- `reg(3)`（原来的最高位）→ `reg(2)`
- `reg(2)` → `reg(1)`
- `reg(1)` → `reg(0)`
- `si`（新数据）→ `reg(3)`

数据向右移动一位。新数据从左边进入，最右边的数据（`reg(0)`）被挤出。

31.5 双向移位寄存器

双向 = 既可以左移也可以右移，由方向控制信号决定。

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity shift_reg_bidir is
5     port (
6         clk      : in  std_logic;
7         rst_n    : in  std_logic;
8         dir      : in  std_logic; -- 方向: '1'=右移, '0'=左移

```

```

9      si      : in  std_logic;  -- 串行输入
10     po      : out std_logic_vector(3 downto 0)
11   );
12 end shift_reg_bidir;
13
14 architecture behavioral of shift_reg_bidir is
15     signal reg : std_logic_vector(3 downto 0);
16 begin
17     process(clk, rst_n)
18     begin
19         if rst_n = '0' then
20             reg <= (others => '0');
21         elsif rising_edge(clk) then
22             if dir = '1' then
23                 reg <= si & reg(3 downto 1);  -- 右移
24             else
25                 reg <= reg(2 downto 0) & si;  -- 左移
26             end if;
27         end if;
28     end process;
29
30     po <= reg;
31 end behavioral;

```

Listing 31.2: 双向移位寄存器

31.6 带并行装载的移位寄存器

增加一个 load 信号，允许直接设置所有 D 触发器的值：

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity shift_reg_load is
5      port (
6          clk      : in  std_logic;
7          rst_n     : in  std_logic;
8          load      : in  std_logic;  -- 并行装载使能
9          data_in   : in  std_logic_vector(3 downto 0);  -- 并行输入
10         si        : in  std_logic;  -- 串行输入
11         po        : out std_logic_vector(3 downto 0)
12     );
13 end shift_reg_load;
14

```

```

15 architecture behavioral of shift_reg_load is
16     signal reg : std_logic_vector(3 downto 0);
17 begin
18     process(clk, rst_n)
19     begin
20         if rst_n = '0' then
21             reg <= (others => '0');
22         elsif rising_edge(clk) then
23             if load = '1' then
24                 reg <= data_in;  -- 并行装载
25             else
26                 reg <= si & reg(3 downto 1);  -- 右移
27             end if;
28         end if;
29     end process;
30
31     po <= reg;
32 end behavioral;

```

Listing 31.3: 带并行装载的移位寄存器

31.7 四种输入输出组合

类型	输入方式	输出方式	典型应用
SISO	串行入	串行出	延迟线
SIPO	串行入	并行出	串并转换
PISO	并行入	串行出	并串转换
PIPO	并行入	并行出	通用寄存器

SISO = Serial In Serial Out (串入串出) SIPO = Serial In Parallel Out (串入并行) PISO = Parallel In Serial Out (并入串出) PIPO = Parallel In Parallel Out (并入并行)

31.8 移位寄存器实现分频器

31.8.1 基本原理

移位寄存器天然具有分频能力。回顾环形计数器和 Johnson 计数器:

- N 位环形计数器: N 个状态循环, 任何 Q 输出的频率 = f_{clk}/N
- N 位 Johnson 计数器: 2N 个状态循环, 任何 Q 输出的频率 = $f_{clk}/(2N)$

因为移位寄存器中的“1”（或状态模式）每 N 拍（或 $2N$ 拍）才经过同一个触发器一次，所以每个 Q 输出的翻转频率恰好是时钟的 $1/N$ （或 $1/2N$ ）。

31.8.2 逐拍分析：4 位环形计数器做 4 分频

假设 4 位环形计数器初始状态 1000，反馈 $Q_0 \rightarrow D_3$ （闭环）。

时钟拍	Q3	Q2	Q1	Q0	说明
0	1	0	0	0	初始
1	0	1	0	0	1 右移
2	0	0	1	0	
3	0	0	0	1	
4	1	0	0	0	回到初始！

Q_3 的输出序列：1, 0, 0, 0, 1, 0, 0, 0, ... 周期 = 4 个时钟 $\rightarrow$ 频率 = $f_{clk}/4$ 。
每个 Q 输出都是 4 分频信号，只是相位不同。

31.8.3 Johnson 计数器做分频器

4 位 Johnson 计数器（8 状态）：

0000 $\rightarrow$ 1000 $\rightarrow$ 1100 $\rightarrow$ 1110 $\rightarrow$ 1111 $\rightarrow$ 0111 $\rightarrow$ 0011 $\rightarrow$ 0001 $\rightarrow$ 0000

任意 Q 输出的周期 = 8 个时钟 $\rightarrow$ 频率 = $f_{clk}/8$ 。
规律：N 位 Johnson 计数器 = $2N$ 分频（所有 Q 输出频率相同，相位不同）。

31.8.4 VHDL 实现：移位寄存器型分频器

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity shift_div4 is
5     port(
6         clk, rst : in  std_logic;
7         clk_out  : out std_logic
8     );
9 end shift_div4;
10
11 architecture behav of shift_div4 is
12     signal reg : std_logic_vector(3 downto 0);
13 begin
14     process(clk, rst)
```

```

15  begin
16      if rst = '1' then reg <= "1000";
17      elsif rising_edge(clk) then
18          reg <= reg(0) & reg(3 downto 1);
19      end if;
20  end process;
21  clk_out <= reg(3);  -- Q3 = 4分频输出
22  end behav;

```

Listing 31.4: 4 位环形计数器做 4 分频器

31.8.5 与计数器分频的对比

分频方式	结构	可得分频比
二进制计数器取 Q_n	加法器 + 寄存器	2^n ($Q_0=2, Q_1=4, Q_2=8\dots$)
环形计数器	N 个 DFF+ 反馈	N (4 位 $\rightarrow$ 4 分频)
Johnson 计数器	N 个 DFF+ 反相反馈	2N (4 位 $\rightarrow$ 8 分频)
模 M 计数器 + 比较	加法器 + 寄存器 + 比较器	任意 M

移位寄存器分频的优缺点：

- 优点：结构规整（只有 D 触发器和反馈线），关键路径极短（1 级 DFF 延迟），适合超高速分频
- 缺点：分频比固定（N 或 2N），灵活性不如计数器方案

31.9 移位寄存器实现序列发生器

31.9.1 方法一：循环移位型（只能循环输出初值，不是真正的序列发生器）

先说清楚：这不是真正的序列发生器。它只是把装进去的值循环输出——装 1001 就循环出 1001，装 0101 就循环出 0101。你无法用它”设计”出一个新的序列。

那为什么还要讲？因为**硬件结构**和真正的序列发生器一模一样：都是移位寄存器 + 反馈线。理解了循环移位，查表法就水到渠成。

方案：移位寄存器 + 反馈逻辑（ Q_0 直接反馈给 D_3 ）。每拍数据右移一位，最低位移回最高位。

例题：4 位寄存器，反馈 = $Q_0 \rightarrow D_3$ ，初值”1001”。

时钟拍	Q3	Q2	Q1	Q0
0	1	0	0	1（初始）
1	1	1	0	0（Q0=1 反馈到 D3）
2	0	1	1	0
3	0	0	1	1
4	1	0	0	1（回到初始!）

序列长度 = 4 拍。输出就是初值本身，循环重复。这不是序列”生成”，只是”循环播放”。

【VHDL——循环右移序列发生器】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity shift_seq_gen is
5     port(clk, rst : in std_logic; seq_out : out std_logic_vector(3
6         downto 0));
7 end shift_seq_gen;
8
9 architecture behav of shift_seq_gen is
10     signal reg : std_logic_vector(3 downto 0);
11 begin
12     process(clk, rst)
13     begin
14         if rst='1' then reg <= "1001";
15         elsif rising_edge(clk) then
16             reg <= reg(0) & reg(3 downto 1);  -- 循环右移
17         end if;
18     end process;
19     seq_out <= reg;
20 end behav;
```

31.9.2 方法二：查表反馈型（通用序列发生器）

循环移位只能产生移位相关的序列。要产生任意序列，需要用查表法（CASE 语句）根据当前状态决定反馈值。

核心结构：移位寄存器 + CASE 查表器。CASE 根据寄存器的当前值，查表输出下一位要移入的 din 值。

31.9.3 例题：用 3 位移位寄存器产生序列 00011101 00011101...

设计分析：3 位移位寄存器共 8 个状态，每个状态通过 CASE 指定反馈位 d_{in} 的值。输出取寄存器最高位 $q(2)$ 。

31.9.4 核心问题：CASE 表是怎么得到的？

这是初学者最容易困惑的地方。答案：用 3 位滑动窗口在目标序列上滑动。

目标序列是：00011101（周期 8，循环重复）。

目标序列 (循环):

0 0 0 1 1 1 0 1

窗口 0 $\Rightarrow$ 000 下一个是 1 $\rightarrow d_{in}=1$

窗口 1 $\Rightarrow$ 001 $\rightarrow state="000"$
窗口 2 $\Rightarrow$ 011 $\rightarrow d_{in}=1$
窗口 3 $\Rightarrow$ 111 $\rightarrow d_{in}=1$
窗口 4 $\Rightarrow$ 110 $\rightarrow d_{in}=1$
窗口 5 $\Rightarrow$ 101 $\rightarrow d_{in}=0$
窗口 6 $\Rightarrow$ 010 $\rightarrow d_{in}=0$
窗口 7 $\Rightarrow$ 001 $\rightarrow d_{in}=0$

推导步骤：

1. 把目标序列写成循环：00011101|00011101|...（首尾相接，无限重复）
2. 用宽度为 3 的窗口，从序列开头每次移动一位，读取窗口内的 3 个 bit——这就是 3 个 D 触发器的当前状态
3. 窗口右边紧挨着的下一个 bit——就是需要反馈进入的 d_{in} 值
4. 覆盖所有 8 种可能的 3-bit 组合（因为 3 位有 $2^3 = 8$ 种组合，恰好一个周期遍历完）

图解推导——3 位滑动窗口法：

目标序列（无限循环，用竖线标出 8 个 3 位窗口）

		000	1	1	1	0	1	0	0	0	1	...	← 窗口 0
	0	001	1	1	0	1	0	0	0	1	...	← 窗口 1	
	0	0	011	1	0	1	0	0	0	1	...	← 窗口 2	
	0	0	0	111	0	1	0	0	0	1	...	← 窗口 3	
	0	0	0	1	110	1	0	0	0	1	...	← 窗口 4	
	0	0	0	1	1	101	0	0	0	1	...	← 窗口 5	
	0	0	0	1	1	1	010	0	0	1	...	← 窗口 6	
	0	0	0	1	1	1	0	100	0	1	...	← 窗口 7	

关键理解：

- 框内 3 位 = 3 个 D 触发器的当前状态 (q_{in})
- 框右边紧跟着的 bit = 下一个要移入的 d_{in} 值

- 框最左边的 bit = 当前输出值 (Q2)
- 下一个时钟沿: din 从右边移入, 所有 bit 左移一位 → 窗口向右滑动一位 → Q2 输出下一个 bit

31.9.5 关键技巧: 需要几个 D 触发器? ——试 3 个不行就换 4 个

窗口宽度 = 寄存器的个数。窗口太窄, 不同位置的窗口值会重复(”撞车”), 导致 CASE 表出现矛盾——同一个状态对应两个不同的 din 值。

举例: 序列 00110011... (8 位循环), 先试 3 位窗口

001 10011 001 10011...
窗口 0 窗口 4

窗口位置	窗口值 (3 位)	下一个 bit(din)
0	001	1
1	011	0
2	110	0
3	100	1
4	001	? ← 窗口值 =001 又出现了! 但这次下一个 bit 是 1? 0?

窗口 0 (001) 的下一个是 1, 窗口 4 (001) 的下一个本来是接着序列... 但同一个状态”001”在 CASE 表中只能有一行, 不能同时对应 din=1 和 din=0。这就是”撞车”。

解决方案: 窗口加宽 → 换 4 个寄存器

窗口位置	窗口值 (4 位)	下一个 bit(din)
0	0011	0
1	0110	0
2	1100	1
3	1001	1
4	0011	0 ← 和窗口 0 一样! 但下一个 bit 也一样 (都是 0), 不冲突 □

4 位窗口下, 0011→下一个是 0, 再次出现 0011 时下一个还是 0, CASE 表不矛盾。

通用法则:

寄存器位数的确定：先试 $k = \lceil \log_2 L \rceil$ 位 (L = 序列长度)，用滑动窗口检查有没有“撞车”。

- 8 种窗口值全不同 $\rightarrow k$ 位够用
- 有重复但重复处的 din 也相同 $\rightarrow k$ 位够用
- 有重复且 din 不同 $\rightarrow$ 窗口加宽为 $k + 1$ 位，重新检查
选最小不撞车的位数作为寄存器个数。

31.9.6 完整例题：你需要几个寄存器？

题目：用查表法产生序列 0011001 0011001... (7 位循环)，最少需要几个 D 触发器？

第一步：算起步位数 $\lceil \log_2 7 \rceil = 3$ 位。先用 3 位窗口试。

把序列循环展开写两遍，方便看环绕：0011001|0011001

位置	3 位窗口值 (qin)	右边一个 bit(din)
0	001	1
1	011	0
2	110	0
3	100	0
4	001	0 ?
5	010	0
6	100	1 ?

查撞车：

- 位置 0 的 001 $\rightarrow \text{din}=1$ ，位置 4 的 001 $\rightarrow \text{din}=0$ 。同一个 001， din 不同！撞车！
- 位置 3 的 100 $\rightarrow \text{din}=0$ ，位置 6 的 100 $\rightarrow \text{din}=1$ 。又撞车！

3 位不行 $\rightarrow$ 换 4 位窗口。

第二步：4 位窗口

序列：0011001|0011001

位置	4 位窗口值 (qin)	下一个 bit(din)
0	0011	0
1	0110	0
2	1100	1
3	1001	0
4	0010	0
5	0100	1
6	1001	1 ?

位置 3(1001)→din=0，位置 6(1001)→din=1。还是撞！→换 5 位。

第三步：5 位窗口

位置	5 位窗口值 (qin)	下一个 bit(din)
0	00110	0
1	01100	1
2	11001	0
3	10010	0
4	00100	1
5	01001	0
6	10011	(回 0)0

7 个 5 位窗口全部不同！→需要 5 个寄存器。

结论： $\lceil \log_2 7 \rceil = 3$ 只是理论下界。实际寄存器数取决于序列本身的结构——有些序列 3 位够用（如 00011101），有些需要更多（如 0011001 需要 5 位）。试到不撞车为止。

31.9.7 速查：多少位寄存器够用？

序列长度 L	$\lceil \log_2 L \rceil$ （起步）	最坏需要	常见需要
4	2	2	2
5	3	3	3
6	3	3~4	3
7	3	5	3~4
8	3	3	3

考试做法：先用 $\lceil \log_2 L \rceil$ 位，画滑动窗口表，检查有无撞车（同一窗口值对应不同 din）。不撞 → 够用；撞 → 加一位重新画。

31.9.8 VHDL 实现：5 位寄存器产生序列 0011001

5 位寄存器共 $2^5 = 32$ 种可能状态，但循环只经过 7 个。CASE 只写这 7 个合法状态，其余 25 个状态统一扔到 when others 回初始态。

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.all;
3
4  entity seqgen_5bit is
5      port(clk, rst : in std_logic; q : out std_logic);
6  end seqgen_5bit;
7
8  architecture behav of seqgen_5bit is
9      signal qin : std_logic_vector(4 downto 0);
10     signal din : std_logic;
11 begin
12     -- 查表产生反馈位 din
13     process(clk)
14     begin
15         if rising_edge(clk) then
16             case qin is
17                 when "00110" => din <= '0';
18                 when "01100" => din <= '1';
19                 when "11001" => din <= '0';
20                 when "10010" => din <= '0';
21                 when "00100" => din <= '1';
22                 when "01001" => din <= '0';
23                 when "10011" => din <= '0';
24                 when others => din <= '0'; -- 非法状态兜底
25             end case;
26         end if;
27     end process;
28
29     -- 移位寄存器：左移，din 进入最低位
30     process(clk, rst)
31     begin
32         if rst = '1' then
33             qin <= "00110"; -- 初始窗口 0
34         elsif rising_edge(clk) then
35             qin <= qin(3 downto 0) & din;
36         end if;
37     end process;
38
39     q <= qin(4); -- 输出最高位
40 end behav;

```


逐拍验证：

拍	qin(当前)	din	q(输出)=qin(4)
0	00110	0	0
1	01100	1	0
2	11001	0	1
3	10010	0	1
4	00100	1	0
5	01001	0	0
6	10011	0	1
7	00110	—	0 ← 回到拍 0

输出 q: 0,0,1,1,0,0,1,0,0,1,1,0,0,1,... ← 确实就是 0011001 循环。

这就是 CASE 表的来源：不撞车的窗口值 → 下一个 bit，整理成表。

这就是 CASE 表的来源：把上面 8 个窗口的”窗口值 → 下一个 bit”整理成表，就是 CASE 表。

完整 CASE 表推导：

窗口编号	窗口内容 (qin)	窗口右边 bit(din)	下一拍 qin(左移后)	输出 q(2)
0	000	1	001	0
1	001	1	011	0
2	011	1	111	0
3	111	0	110	1
4	110	1	101	1
5	101	0	010	1
6	010	0	100	0
7	100	0	000	1

逐拍追踪：

时钟拍	qin(当前)	CASE 查表 →din	下一拍 qin	输出 q(2)
0	000	1	001	0
1	001	1	011	0
2	011	1	111	0
3	111	0	110	1
4	110	1	101	1
5	101	0	010	1
6	010	0	100	0
7	100	0	000	1
8	000	—	—	回到拍 0

输出序列 q(2): 0,0,0,1,1,1,0,1,0,0,0,1,1,1,0,1,... 周期=8。与目标序列完全一致!

【VHDL——查表反馈型序列发生器】

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.all;
3
4  entity seqgen_table is
5      port(clk, rst : in std_logic; q : out std_logic);
6  end seqgen_table;
7
8  architecture behav of seqgen_table is
9      signal qin : std_logic_vector(2 downto 0);
10     signal din : std_logic;
11 begin
12     -- 第一段: CASE查表产生反馈位 din
13     comb_proc: process(clk)
14     begin
15         if rising_edge(clk) then
16             case qin is
17                 when "000" => din <= '1';
18                 when "001" => din <= '1';
19                 when "011" => din <= '1';
20                 when "111" => din <= '0';
21                 when "110" => din <= '1';
22                 when "101" => din <= '0';
23                 when "010" => din <= '0';
24                 when "100" => din <= '0';
25                 when others => din <= '0';
26             end case;
27         end if;
28     end process;
29
30     -- 第二段: 移位寄存器 (左移, din进入最低位)
31     shift_proc: process(clk, rst)
32     begin
33         if rst = '1' then qin <= (others => '0');
34         elsif rising_edge(clk) then
35             qin <= qin(1 downto 0) & din;
36         end if;
37     end process;
38
39     q <= qin(2); -- 输出最高位
40 end behav;

```

【设计思想】查表法的本质: 把想要的序列”写死”在 CASE 表里。当前状态 → 查

表 → 下一个输入位。只要改 CASE 表的内容，就能生成任何 8 位循环序列。这是通用序列发生器的设计范式。

31.9.9 两种方案对比

方案	循环移位型	查表反馈型
反馈逻辑	reg(0) → D(N-1)（一根线）	CASE 表（多条件判断）
序列来源	就是初值本身（装入什么输出什么）	CASE 表任意指定（真正的设计）
代码复杂度	1 行	N 行（每个状态 1 行）
修改序列	改初值（但只能输出初值循环）	修改 CASE 表内容（任意序列）
适用场景	学习移位寄存器工作原理	实际的序列发生器设计

31.10 移位寄存器实现串行加法器

31.10.1 原理

并行加法器（如行波进位）一次完成所有位的加法。串行加法器逐位相加，每次处理 1 位，N 拍后得到 N 位结果。**核心结构：**1 位全加器 + 进位 D 触发器 + 两个移位寄存器（存放操作数 A 和 B）+ 一个移位寄存器（存放结果）。

31.10.2 逐拍工作过程

以 4 位加法 1011 + 0110 为例：

拍	A 移位器 (Q0)	B 移位器 (Q0)	Cin(DFF)	Sum 位	Cout
0（初）	1	0	0	—	—
1	1	0	0	1	0
2	1	1	0	0	1
3	0	1	1	0	1
4	1	0	1	0	1

每拍：A 和 B 各右移 1 位（最低位进入全加器），全加器计算 Sum 和 Cout，Cout 存入进位 DFF 作为下一拍的 Cin。结果 Sum 逐位存入结果移位寄存器。4 拍后结果移位寄存器 = 0001 0001（即 17，1011+0110=17）。

【VHDL——4 位串行加法器】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity serial_adder is
```

```

5  port(
6      clk, rst, start : in std_logic;
7      A, B : in std_logic_vector(3 downto 0);
8      Sum : out std_logic_vector(3 downto 0);
9      done : out std_logic
10 );
11 end serial_adder;
12
13 architecture behav of serial_adder is
14     signal regA, regB, regSum : std_logic_vector(3 downto 0);
15     signal carry : std_logic := '0';
16     signal bit_cnt : integer range 0 to 4 := 0;
17     signal working : boolean := false;
18     signal s, c : std_logic;
19 begin
20     s <= regA(0) xor regB(0) xor carry;
21     c <= (regA(0) and regB(0)) or (regA(0) and carry) or (regB(0) and
22         carry);
23
24     process(clk, rst)
25     begin
26         if rst='1' then
27             regA <= (others=>'0'); regB <= (others=>'0');
28             regSum <= (others=>'0'); carry <= '0';
29             bit_cnt <= 0; working <= false; done <= '0';
30         elsif rising_edge(clk) then
31             done <= '0';
32             if start='1' and not working then
33                 regA <= A; regB <= B; regSum <= (others=>'0');
34                 carry <= '0'; bit_cnt <= 0; working <= true;
35             elsif working then
36                 regA <= '0' & regA(3 downto 1);
37                 regB <= '0' & regB(3 downto 1);
38                 regSum <= s & regSum(3 downto 1);
39                 carry <= c;
40                 if bit_cnt=3 then
41                     working <= false; done <= '1';
42                     else bit_cnt <= bit_cnt+1; end if;
43             end if;
44         end if;
45     end process;
46     Sum <= regSum;
47 end behav;

```

31.11 移位寄存器实现串行累加器

31.11.1 原理

累加器 = 加法器 + 累加寄存器 (ACC)。串行累加器用移位寄存器做 ACC：每输入一个新操作数，将其与 ACC 中的值串行相加，结果存回 ACC。

结构：1 位全加器 + 进位 DFF + ACC 移位寄存器 + 输入移位寄存器。

【VHDL 核心逻辑——累加循环】

```

1  -- ACC累加: ACC = ACC + Input
2  -- 两个移位寄存器: acc_reg (累加值) 和 in_reg (新输入)
3  -- 每拍: acc_reg(0) + in_reg(0) + carry + sum位存入acc高位, 进位更新
4  ...
5  if working then
6      acc_reg <= sum_bit & acc_reg(7 downto 1);  -- 结果右移存入
7      in_reg  <= '0' & in_reg(7 downto 1);      -- 输入右移
8      carry   <= cout;
9      if bit_cnt = 7 then working <= false; end if;
10 end if;

```

31.12 移位寄存器实现乘法器

31.12.1 原理——移位相加法

二进制乘法可以分解为移位 + 相加：

$$A \times B = A \times (B_3 \cdot 2^3 + B_2 \cdot 2^2 + B_1 \cdot 2^1 + B_0 \cdot 2^0)$$

逐位检查乘数 B 的每一位：

- $B_0 = 1$ ：累加 A（不移位）
- $B_1 = 1$ ：累加 A 左移 1 位（即 $A \times 2$ ）
- $B_2 = 1$ ：累加 A 左移 2 位
- ...

更高效的方法（移位-加乘法器）：每拍检查 B 的最低位，若为 1 则 $ACC += A$ ，然后 A 左移 1 位，B 右移 1 位。N 拍后 ACC 中为乘积。

逐拍分析： $5 \times 3 = 0101 \times 0011 = 15$

拍	B(最低位)	操作	ACC	A(左移)
0	1	ACC+=A(0101)	0101	0101
1	1	ACC+=A(1010)	1111	1010
2	0	不加	1111	0100(移出)
3	0	不加	1111	—

结果 ACC=1111=15，正确。

【VHDL——4 位移位-加乘法器】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity shift_multiplier is
6      port(
7          clk, rst, start : in std_logic;
8          A_in, B_in : in unsigned(3 downto 0);
9          product : out unsigned(7 downto 0);
10         done : out std_logic
11     );
12 end shift_multiplier;
13
14 architecture behav of shift_multiplier is
15     signal A_reg : unsigned(7 downto 0);
16     signal B_reg : unsigned(3 downto 0);
17     signal ACC : unsigned(7 downto 0) := (others=>'0');
18     signal cnt : integer range 0 to 4 := 0;
19     signal working : boolean := false;
20 begin
21     process(clk, rst)
22     begin
23         if rst='1' then
24             ACC<=(others=>'0'); cnt<=0; working<=false; done<='0';
25         elsif rising_edge(clk) then
26             done <= '0';
27             if start='1' and not working then
28                 A_reg <= "0000" & A_in;
29                 B_reg <= B_in;
30                 ACC <= (others=>'0');
31                 cnt <= 0; working <= true;
32             elsif working then
33                 if cnt=4 then
34                     working <= false; done <= '1';
35                 else

```

```

36         if B_reg(0)='1' then ACC <= ACC + A_reg; end if;
37         A_reg <= A_reg(6 downto 0) & '0';    -- 左移1位
38         B_reg <= '0' & B_reg(3 downto 1);    -- 右移1位
39         cnt <= cnt+1;
40     end if;
41 end if;
42 end if;
43 end process;
44 product <= ACC;
45 end behav;

```

31.13 考试常见变式

1. 变式 1: 实现串并转换 (SIPO)

串行数据输入，4 个时钟周期后，并行输出完整 4 位数据。

2. 变式 2: 实现并串转换 (PISO)

先 load 并行数据，然后 4 个时钟周期移位输出。

3. 变式 3: 多位移位 (桶形移位器)

一次移多位 (不是每次只移 1 位)。需要更复杂的组合逻辑。

4. 变式 4: 移位寄存器 + 反馈 = LFSR/环形计数器

将串行输出反馈回串行输入。这将在第七部分详细展开。

5. 变式 5: 移位寄存器的 Testbench 设计

如何验证移位寄存器的正确性。

31.14 Testbench

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity shift_reg_tb is
5  end shift_reg_tb;
6
7  architecture test of shift_reg_tb is
8      signal clk, rst_n, si, so : std_logic;
9      signal po : std_logic_vector(3 downto 0);

```

```
10  constant clk_period : time := 20 ns;
11
12  component shift_reg_right is
13      port (clk, rst_n, si : in std_logic;
14            so : out std_logic; po : out std_logic_vector(3 downto 0));
15  end component;
16  begin
17      uut: shift_reg_right port map (clk, rst_n, si, so, po);
18
19      clk_process: process
20      begin
21          clk <= '0'; wait for clk_period/2;
22          clk <= '1'; wait for clk_period/2;
23      end process;
24
25      stimulus: process
26      begin
27          rst_n <= '0'; si <= '0'; wait for 30 ns;
28          rst_n <= '1'; wait for 5 ns;
29
30          -- 输入序列: 1, 0, 1, 1
31          si <= '1'; wait for clk_period;
32          si <= '0'; wait for clk_period;
33          si <= '1'; wait for clk_period;
34          si <= '1'; wait for clk_period;
35
36          -- 观察: po应该依次为 1000, 0100, 1010, 1101
37          wait;
38      end process;
39  end test;
```

Listing 31.5: 移位寄存器 Testbench

移位寄存器秒杀法: 右移: `reg <= si & reg(N-1 downto 1)`

左移: `reg <= reg(N-2 downto 0) & si`

并行装载: `if load='1' then reg <= data_in;`

串行输出: 最高位/最低位 = so

并行输出: reg 的全部位 = po

31.15 变式详解

第三十二章 移位寄存器——全部变式详解

32.1 变式 1：串并转换（SIPO）

完整教学

【功能】串行输入 → 4 拍后并行读取。UART/SPI 接收器的基础。【VHDL】`reg <= si & reg(N-1 downto 1); po <= reg;`

32.2 变式 2：并串转换（PISO）

完整教学

【功能】load 并行数据 → 逐拍串行输出。UART/SPI 发送器的基础。【VHDL】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity piso_shift_reg is
5     generic (N : integer := 8);
6     port (
7         clk      : in  std_logic;
8         rst_n    : in  std_logic;
9         load     : in  std_logic;
10        data_in  : in  std_logic_vector(N-1 downto 0);
11        so       : out std_logic
12    );
13 end piso_shift_reg;
14
15 architecture behavioral of piso_shift_reg is
16     signal reg : std_logic_vector(N-1 downto 0);
17 begin
```

```
18 process(clk, rst_n)
19 begin
20     if rst_n = '0' then
21         reg <= (others => '0');
22     elsif rising_edge(clk) then
23         if load = '1' then
24             reg <= data_in;
25         else
26             reg <= '0' & reg(N-1 downto 1);
27         end if;
28     end if;
29 end process;
30 so <= reg(0);
31 end behavioral;
```

32.3 变式 3：双向移位

完整教学

【VHDL】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity bidir_shift_reg is
5     generic (N : integer := 8);
6     port (
7         clk    : in  std_logic;
8         rst_n  : in  std_logic;
9         dir    : in  std_logic;
10        si_r   : in  std_logic;
11        si_l   : in  std_logic;
12        po     : out std_logic_vector(N-1 downto 0)
13    );
14 end bidir_shift_reg;
15
16 architecture behavioral of bidir_shift_reg is
17     signal reg : std_logic_vector(N-1 downto 0);
18 begin
19     process(clk, rst_n)
```

```
20  begin
21      if rst_n = '0' then
22          reg <= (others => '0');
23      elsif rising_edge(clk) then
24          if dir = '1' then
25              reg <= si_r & reg(N-1 downto 1);
26          else
27              reg <= reg(N-2 downto 0) & si_l;
28          end if;
29      end if;
30  end process;
31  po <= reg;
32  end behavioral;
```

32.4 变式 4：循环移位

完整教学

【功能】数据不丢失——最低位移回最高位。【VHDL】`reg <= reg(0) & reg(N-1 downto 1);`（右循环）【本质】这就是环形计数器的基础！

32.5 变式 5：算术移位（保持符号位）

完整教学

【逻辑右移】高位补 0 → `'0' & reg(N-1 downto 1)` 【算术右移】高位补符号位 → `reg(N-1) & reg(N-1 downto 1)` 【应用】有符号数 ÷2 操作。

第七部分

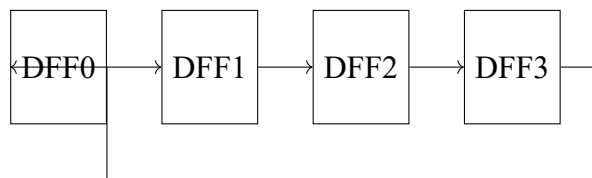
环形计数器体系

第三十三章 移位寄存器构建环形计数器

33.1 从移位寄存器出发

回顾移位寄存器：数据从一端进入，经过 N 拍从另一端出来。

现在思考一个问题：如果把串行输出**反馈**到串行输入呢？



反馈环：Q3 → D0

环形计数器的形成：环形计数器 = 移位寄存器 + Q 输出反馈到串行输入
数据不再从外部输入，而是在 D 触发器链中**循环**流动。

数据在闭合的环中无限循环

33.2 逐拍分析：环形计数器如何工作

假设 4 位环形计数器，初始状态通过复位设为 1000（即 Q3=1, Q2=0, Q1=0, Q0=0）。

注意：这里假设 Q3 是最高位（最左），Q0 是最低位（最右）。

反馈连接：Q0 → D3（最低位反馈回最高位的输入）。

33.2.1 时钟第 0 拍（初始/复位后）

寄存器内容：[Q3 Q2 Q1 Q0] = [1 0 0 0]

33.2.2 时钟第 1 拍上升沿

环形计数器——时钟第 1 拍：

- $D3 = Q0 = 0$ （反馈值）
- $D2 = Q3 = 1$
- $D1 = Q2 = 0$
- $D0 = Q1 = 0$

上升沿后：

- $Q3 \leftarrow D3 = 0$ （从 $Q0$ 反馈回来的 0）
- $Q2 \leftarrow D2 = 1$ （ $Q3$ 原来的 1 右移一位）
- $Q1 \leftarrow D1 = 0$
- $Q0 \leftarrow D0 = 0$

寄存器内容：[0 1 0 0]（1 向右移动了一位！）

33.2.3 时钟第 2 拍上升沿

环形计数器——时钟第 2 拍：

- $D3 = Q0 = 0$
- $D2 = Q3 = 0$
- $D1 = Q2 = 1$
- $D0 = Q1 = 0$

上升沿后：[0 0 1 0]（1 继续右移）

33.2.4 时钟第 3 拍上升沿

环形计数器——时钟第 3 拍：

- $D3 = Q0 = 0$
- $D2 = Q3 = 0$
- $D1 = Q2 = 0$
- $D0 = Q1 = 1$

上升沿后：[0 0 0 1]（1 移到了最右边）

33.2.5 时钟第 4 拍上升沿

环形计数器——时钟第 4 拍：关键时刻！：

- $D3 = Q0 = 1$ （反馈环起作用！ $Q0$ 的 1 反馈回 $D3$ ）
- $D2 = Q3 = 0$
- $D1 = Q2 = 0$
- $D0 = Q1 = 0$

上升沿后：[1 0 0 0]（1 回到最左边——完成一圈循环！）

33.2.6 完整的循环过程

时钟拍	Q3	Q2	Q1	Q0
0（初始）	1	0	0	0
1	0	1	0	0
2	0	0	1	0
3	0	0	0	1
4	1	0	0	0
5	0	1	0	0
...	...	...	...	...

这个“1 在循环移动”的现象就是环形计数器的本质。

33.3 环形计数器 vs 普通计数器

特性	二进制计数器	环形计数器
硬件结构	加法器 + D 触发器	N 个 D 触发器 + 反馈环
状态编码	二进制 (000,001,...)	One-hot (1000,0100,...)
N 个状态需要	$\lceil \log_2 N \rceil$ 个 DFF	N 个 DFF
状态解码	需要组合逻辑	直接看哪个 DFF 的 Q=1
速度	受加法器进位链限制	最快 (只有移位)
最大状态数	2^k	k (k 个 DFF 产生 k 个状态)

环形计数器用更多触发器换取更快的速度和更简单的解码。

33.4 为什么反馈后形成循环状态

反馈把串行输出接回串行输入，形成一个闭合回路。

- 没有反馈：数据从一端进入，从另一端消失（一去不回）
- 有反馈：数据从一端出来后，重新从另一端进入（无限循环）

这就像一条首尾相接的传送带——东西在上面无限循环移动。

33.5 约翰逊计数器（扭环形计数器）

约翰逊计数器是环形计数器的变体：将反相的 Q 输出反馈回输入。

- 环形计数器：Q0 → D3（同相反馈）
- 约翰逊计数器（扭环形）： $\overline{Q_0}$ → D3（反相反馈）

效果：k 个触发器可以产生 2k 个状态（比普通环形计数器多一倍！）
4 位约翰逊计数器的状态序列：

$0000 \rightarrow 1000 \rightarrow 1100 \rightarrow 1110 \rightarrow 1111 \rightarrow 0111 \rightarrow 0011 \rightarrow 0001 \rightarrow 0000$

共 8 个状态（4 个触发器产生 8 个状态）。

33.6 VHDL 实现

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity ring_counter is
5     port (
6         clk    : in  std_logic;
7         rst_n   : in  std_logic;
8         q       : out std_logic_vector(3 downto 0)
9     );
10 end ring_counter;
11
12 architecture behavioral of ring_counter is
13     signal reg : std_logic_vector(3 downto 0);
14 begin
15     process(clk, rst_n)
16     begin
17         if rst_n = '0' then
18             reg <= "1000"; -- 初始状态: 只有最高位为1
19         elsif rising_edge(clk) then
20             reg <= reg(0) & reg(3 downto 1); -- 右移 + 最低位反馈到最高位
21         end if;
22     end process;
23
24     q <= reg;
25 end behavioral;
```

Listing 33.1: 4 位环形计数器

关键语句: `reg <= reg(0) & reg(3 downto 1)`

这是右移 + 反馈的合并:

- `reg(0)` (最低位) → `reg(3)` (反馈到最高位输入)
- `reg(3)` → `reg(2)` (右移)
- `reg(2)` → `reg(1)` (右移)
- `reg(1)` → `reg(0)` (右移)

33.6.1 约翰逊计数器 VHDL

```
1 library ieee;
```

```
2 use ieee.std_logic_1164.all;
3
4 entity johnson_counter is
5     port (
6         clk    : in  std_logic;
7         rst_n   : in  std_logic;
8         q       : out std_logic_vector(3 downto 0)
9     );
10 end johnson_counter;
11
12 architecture behavioral of johnson_counter is
13     signal reg : std_logic_vector(3 downto 0);
14 begin
15     process(clk, rst_n)
16     begin
17         if rst_n = '0' then
18             reg <= (others => '0'); -- 初始全0
19         elsif rising_edge(clk) then
20             reg <= (not reg(0)) & reg(3 downto 1); -- 反相的最低位移入最高
21             位
22         end if;
23     end process;
24
25     q <= reg;
26 end behavioral;
```

Listing 33.2: 4 位约翰逊计数器（扭环形）

33.7 考试常见变式

1. 变式 1：设计 N 位环形计数器

N 位环形计数器产生 N 个状态，初始值 100...0（one-hot 编码）。

2. 变式 2：设计 2N 位约翰逊计数器

N 个触发器产生 2N 个状态。反馈是反相的最低位移入最高位。

3. 变式 3：自启动的环形计数器

如果因干扰进入非法状态（如 0101，两个 1），需要自动回到合法状态。这需要额外的修正逻辑。

4. 变式 4：比较环形计数器与二进制计数器

常考：N 状态各需多少触发器？各有何优缺点？

33.8 Testbench

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity ring_counter_tb is
5 end ring_counter_tb;
6
7 architecture test of ring_counter_tb is
8     signal clk, rst_n : std_logic;
9     signal q : std_logic_vector(3 downto 0);
10    constant clk_period : time := 20 ns;
11
12    component ring_counter is
13        port (clk, rst_n : in std_logic; q : out std_logic_vector(3
14            downto 0));
15    end component;
16
17    begin
18        uut: ring_counter port map (clk, rst_n, q);
19
20        clk_process: process
21        begin
22            clk <= '0'; wait for clk_period/2;
23            clk <= '1'; wait for clk_period/2;
24        end process;
25
26        stimulus: process
27        begin
28            rst_n <= '0'; wait for 30 ns;
29            rst_n <= '1';
30            -- 观察两个完整循环 (8拍)
31            wait for clk_period * 9;
32            wait;
33        end process;
34    end test;
```

Listing 33.3: 环形计数器 Testbench

环形计数器秒杀法：初始：reg = "1000"（只有一个 1）

每个时钟沿：1 向右移动一位（因为反馈环）

核心 VHDL：reg <= reg(0) & reg(N-1 downto 1)

约翰逊计数器：用反相：reg <= (not reg(0)) & reg(N-1 downto 1)

33.9 变式详解

第三十四章 环形计数器——全部变式详解

34.1 变式 1: N 位环形计数器

完整教学

【通用模板】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity ring_counter_n is
5     generic (N : integer := 4);
6     port (
7         clk    : in  std_logic;
8         rst_n   : in  std_logic;
9         q       : out std_logic_vector(N-1 downto 0)
10    );
11 end ring_counter_n;
12
13 architecture behavioral of ring_counter_n is
14     signal reg : std_logic_vector(N-1 downto 0);
15 begin
16     process(clk, rst_n)
17     begin
18         if rst_n = '0' then
19             reg <= (N-1 => '1', others => '0'); -- One-hot初值
20         elsif rising_edge(clk) then
21             reg <= reg(0) & reg(N-1 downto 1); -- 右移+反馈
22         end if;
23     end process;
24
25     q <= reg;
```

```
26 end behavioral;
```

Listing 34.1: N 位环形计数器 (Generic 实现)

【结论】 N 个 DFF → N 个状态。所有状态中恰好一个 1 (One-hot 编码)。

34.2 变式 2: 约翰逊计数器 (2N 个状态)

完整教学

【反馈】 `reg <= (not reg(0)) & reg(N-1 downto 1);`

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity johnson_counter_n is
5      generic (N : integer := 4);
6      port (
7          clk    : in  std_logic;
8          rst_n   : in  std_logic;
9          q       : out std_logic_vector(N-1 downto 0)
10     );
11 end johnson_counter_n;
12
13 architecture behavioral of johnson_counter_n is
14     signal reg : std_logic_vector(N-1 downto 0);
15 begin
16     process(clk, rst_n)
17     begin
18         if rst_n = '0' then
19             reg <= (others => '0');
20         elsif rising_edge(clk) then
21             reg <= (not reg(0)) & reg(N-1 downto 1);
22         end if;
23     end process;
24
25     q <= reg;
26 end behavioral;
```

Listing 34.2: N 位约翰逊计数器 (Generic 实现)

【4 位约翰逊状态序列】 0000 → 1000 → 1100 → 1110 → 1111 → 0111 →

0011 → 0001 → 0000 【结论】N 个 DFF → 2N 个状态（比环形计数器多一倍）。

34.3 变式 3：自启动环形计数器

完整教学

【问题】干扰可能使环形计数器进入非法状态。【方案】检测非法状态（如全 0 或多个 1），强制恢复到合法状态 (1000)。

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity ring_counter_safe is
5     port (
6         clk    : in  std_logic;
7         rst_n   : in  std_logic;
8         q       : out std_logic_vector(3 downto 0)
9     );
10 end ring_counter_safe;
11
12 architecture behavioral of ring_counter_safe is
13     signal reg : std_logic_vector(3 downto 0);
14
15     function count_ones(v : std_logic_vector) return integer is
16         variable cnt : integer := 0;
17     begin
18         for i in v'range loop
19             if v(i) = '1' then cnt := cnt + 1; end if;
20         end loop;
21         return cnt;
22     end function;
23
24 begin
25     process(clk, rst_n)
26     begin
27         if rst_n = '0' then
28             reg <= "1000";
29         elsif rising_edge(clk) then
30             if count_ones(reg) /= 1 then
31                 reg <= "1000"; -- 非法状态，恢复
32             else
```

```
33     reg <= reg(0) & reg(3 downto 1);
34     end if;
35     end if;
36     end process;
37
38     q <= reg;
39 end behavioral;
```

Listing 34.3: 自启动 4 位环形计数器

【关键】 在移位前检查: `if count_ones(reg) /= 1 then reg <= "1000";`

34.4 变式 4：用环形计数器实现时序控制器

完整教学

【应用】 多周期操作器的步骤控制。**【5 步控制器】** 5 位环形计数器 → 每拍恰好一个步骤信号为 1 → 控制对应步骤。

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity timing_controller is
5      port (
6          clk    : in  std_logic;
7          rst_n  : in  std_logic;
8          step   : out std_logic_vector(4 downto 0)
9      );
10 end timing_controller;
11
12 architecture behavioral of timing_controller is
13     signal reg : std_logic_vector(4 downto 0);
14 begin
15     process(clk, rst_n)
16     begin
17         if rst_n = '0' then
18             reg <= "10000";
19         elsif rising_edge(clk) then
20             reg <= reg(0) & reg(4 downto 1);
21         end if;
22     end process;
```



```
23  
24     step <= reg;  
25 end behavioral;
```

Listing 34.4: 5 步时序控制器

【优点】 无需额外的译码逻辑——直接读 Q 值。

第八部分

序列检测器体系

第三十五章 序列检测器——状态机的核心应用

35.1 什么是序列检测器

序列检测器：在串行输入流中检测特定模式的电路。

输入：一串 bit 流（每个时钟周期输入 1bit）输出：当检测到目标序列时，输出 =1

序列检测器的本质：序列检测器一定是一个状态机。

为什么？因为它需要“记住已经匹配了序列的前几位”——这需要记忆。组合逻辑做不到——它不记得历史。

35.2 为什么序列检测器一定是状态机

考虑检测序列 1011：

假设当前输入流是：... 1 0 1 ...

此时你看到了“101”，还剩一个“1”就匹配成功。

这个“已经匹配了 3 位”的信息必须被记住。下一拍如果输入是 1，就检测到了；如果是 0，就需要重新开始匹配。

这个“记住了多少”就是状态。

状态在序列检测器中的含义：状态 = “目前已匹配了目标序列的前几位”

- S0（初始）：匹配了 0 位（等待序列开始）
- S1：匹配了 1 位
- S2：匹配了 2 位
- S3：匹配了 3 位
- S4（检测到）：匹配了 4 位（完整序列!）

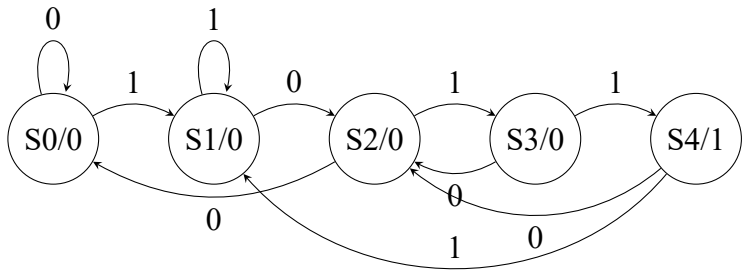
35.3 1011 序列检测器——Moore 型（可重叠检测）

35.3.1 状态定义（5 状态严格 Moore）

检测目标：1011。Moore 型使用 5 个状态，S4 为专用输出状态。

状态	含义（已匹配的前缀）	输出 y
S0	初始状态，未匹配任何位	0
S1	匹配第 1 位“1”	0
S2	匹配前 2 位“10”	0
S3	匹配前 3 位“101”	0
S4	匹配完整序列“1011”	1

35.3.2 状态转移图



Moore 型标注规则：输出标在状态圈内（状态名/输出值），转移弧上只标输入值。S0-S3 输出为 0，S4 输出为 1。

35.3.3 状态转移分析

- **S0：** din=0→S0； din=1→S1
- **S1：** din=0→S2； din=1→S1（“11” 的后一个 1 仍是有效前缀）
- **S2：** din=0→S0； din=1→S3
- **S3：** din=0→S2（“1010” 的后缀“10” 匹配前缀）； din=1→S4
- **S4：** din=0→S2； din=1→S1（**可重叠：**末尾 1 可作为下一组序列起始）

35.3.4 状态转移表

当前状态	输入 din	下一状态	输出 y
S0	0	S0	0
S0	1	S1	0
S1	0	S2	0
S1	1	S1	0
S2	0	S0	0
S2	1	S3	0
S3	0	S2	0
S3	1	S4	0
S4	0	S2	1
S4	1	S1	1

35.3.5 VHDL 实现（两段式，异步高电平复位）

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity seq1011_moore is
5      port(
6          clk : in  std_logic;    -- 系统时钟，上升沿有效
7          rst : in  std_logic;    -- 异步高电平复位
8          din : in  std_logic;    -- 串行数据输入
9          y   : out std_logic     -- 检测输出，高电平表示检测到1011
10     );
11 end entity seq1011_moore;
12
13 architecture behav of seq1011_moore is
14     type state_type is (S0, S1, S2, S3, S4);
15     signal curr_state, next_state : state_type;
16 begin
17     -- 第一段：时序逻辑（状态寄存器，时钟+异步复位）
18     reg_proc: process(clk, rst)
19     begin
20         if rst = '1' then
21             curr_state <= S0;
22         elsif rising_edge(clk) then
23             curr_state <= next_state;
24         end if;
25     end process reg_proc;
26
27     -- 第二段：组合逻辑（状态转移判断）

```

```

28  comb_proc: process(curr_state, din)
29  begin
30      case curr_state is
31          when S0 =>
32              if din = '1' then next_state <= S1;
33              else next_state <= S0; end if;
34          when S1 =>
35              if din = '0' then next_state <= S2;
36              else next_state <= S1; end if;
37          when S2 =>
38              if din = '1' then next_state <= S3;
39              else next_state <= S0; end if;
40          when S3 =>
41              if din = '1' then next_state <= S4;
42              else next_state <= S2; end if;
43          when S4 =>
44              if din = '1' then next_state <= S1;    -- 重叠
45              else next_state <= S2; end if;
46          when others =>
47              next_state <= S0;
48      end case;
49  end process comb_proc;
50
51  -- Moore型输出: 仅由当前状态决定
52  y <= '1' when curr_state = S4 else '0';
53 end architecture behav;

```

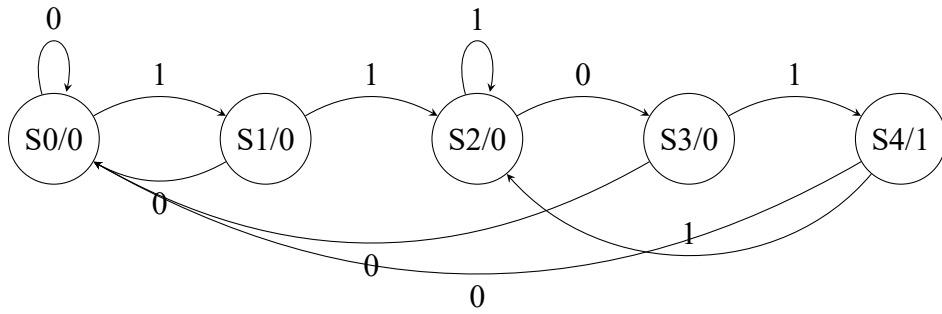
Listing 35.1: Moore 型 1101 序列检测器 (5 状态, 可重叠)

35.4 1101 序列检测器 (Moore 型)

检测目标: 1101。5 状态严格 Moore, S4 为专用输出状态。

状态定义:

- S0: 初始 (匹配 0 位) S1: 已匹配"1" S2: 已匹配"11"
- S3: 已匹配"110" S4: 匹配完整序列"1101", 输出 y=1



状态转移：S0+1→S1, S1+1→S2, S2+0→S3, S3+1→S4（检测到！）。S4+0→S0, S4+1→S2（重叠：末尾的”1”可开始新序列）。

35.5 重叠检测 vs 非重叠检测

这是必考的区别！

重叠检测：检测到序列后，已匹配的后缀可以用于下一次匹配。

例：输入...1011011...（检测 1011，重叠）

第一次检测：1 0 1 1 0 1 1 → 在位置 4 检测到 1011 第二次检测：1 0 1 1 0 1 1 → 位置 4 的”11”中的第二个”1”开始新匹配

非重叠检测：检测到序列后，重新从初始状态开始。

例：输入...1011011...（检测 1011，非重叠）

第一次检测：1 0 1 1 0 1 1 → 在位置 4 检测到 1011 第二次检测：1 0 1 1 0 1 1 ... → 从位置 5 重新开始（忽略位置 4 的后缀）

写作 VHDL 的区别：

重叠检测：在检测到状态，根据输入决定返回哪个状态（可能有重叠前缀）非重叠检测：检测到后，直接回到 S0/S1（看具体情况）

35.6 Moore 型 vs Mealy 型——考试必考核心对比

35.6.1 两种状态机模型的根本区别

Moore vs Mealy 的本质：Moore 型状态机：输出仅取决于当前状态。

Mealy 型状态机：输出取决于当前状态 + 当前输入。

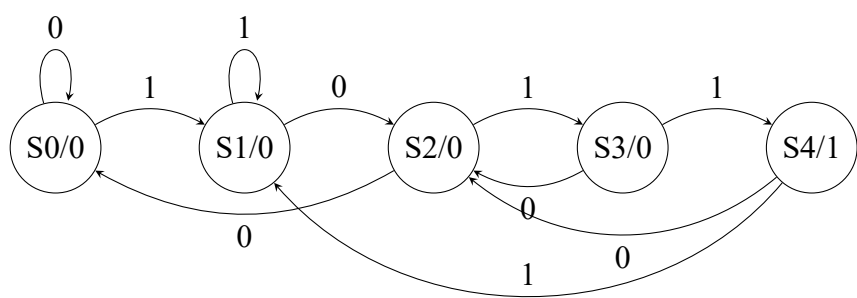
这个区别看似微小，但导致了两者在状态数、输出时序和硬件结构上的显著差异。

35.6.2 Moore 型 1011 序列检测器（5 状态）

【状态定义】严格 Moore 型使用 5 个状态，S4 为专用输出状态。

状态	含义	输出 y
S0	未匹配任何有效前缀（初始/复位）	0
S1	已匹配第 1 位：”1”	0
S2	已匹配前 2 位：”10”	0
S3	已匹配前 3 位：”101”	0
S4	匹配完整序列”1011”	1

【状态转移图】输出标在状态圈内（Moore 型标志）。



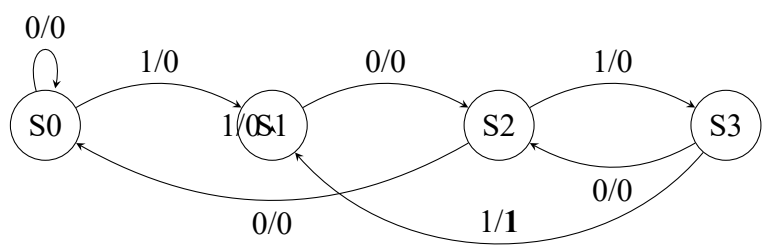
【输出逻辑】严格 Moore 型， $y = 1$ 当且仅当 $curr_state = S4$ ，与当前输入 din 无关。进入 S4 后输出持续整个状态周期（稳定无毛刺）。

35.6.3 Mealy 型 1011 序列检测器（4 状态）

【状态定义】Mealy 型用 4 个状态，无专用输出状态，输出在转移弧上。

状态	含义
S0	未匹配任何有效前缀
S1	已匹配第 1 位：”1”
S2	已匹配前 2 位：”10”
S3	已匹配前 3 位：”101”

【状态转移图（标注格式：输入/输出）】



$S3 \xrightarrow{1/1} S1$: 输入 =1 时 $y=1$ (检测到 1011), 去 $S1$ (可重叠——末尾 1 作新序列起始)。

【关键对比】

- **Moore:** 5 状态 ($S0-S4$), 输出仅取决于当前状态, $y = 1$ when $curr_state = S4$
- **Mealy:** 4 状态 ($S0-S3$), 无 $S4$, 输出在 $S3+din=1$ 的转移弧上直接产生
- Moore 输出晚 1 拍 (等状态稳定), Mealy 输出与输入同步

35.6.4 完整对比表

对比维度	Moore 型	Mealy 型
输出决定因素	仅当前状态	状态 + 输入
状态数 (检测 1011)	5 个 ($S0-S4$)	4 个 ($S0-S3$)
输出时序	进入 $S4$ 后输出 (晚 1 拍)	转移同时输出 (同步)
输出持续时间	整个 $S4$ 周期 (稳定)	仅在条件满足时
对输入毛刺敏感	不敏感 (只看状态)	敏感 (输入参与输出)
状态图标注	状态圈内” 状态/输出”	转移弧上” 输入/输出”
综合结果	3FF+ 输出逻辑	2FF+ 次态/输出组合逻辑
考试推荐	更安全稳定, 优先使用	更紧凑, 按题目要求

35.6.5 VHDL 实现对比 (两段式, 异步高电平复位)

【Moore 型——5 状态, 可重叠】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity seq1011_moore is
5     port(
6         clk : in  std_logic;
7         rst : in  std_logic;    -- 异步高电平复位
8         din : in  std_logic;
9         y   : out std_logic
10    );
11 end entity seq1011_moore;
12
13 architecture behav of seq1011_moore is
14     type state_type is (S0, S1, S2, S3, S4);
15     signal curr_state, next_state : state_type;
16 begin
17     -- 第一段: 时序逻辑 (状态寄存器)
```

```

18  reg_proc: process(clk, rst)
19  begin
20      if rst = '1' then curr_state <= S0;
21      elsif rising_edge(clk) then curr_state <= next_state;
22      end if;
23  end process reg_proc;
24
25  -- 第二段：组合逻辑（状态转移）
26  comb_proc: process(curr_state, din)
27  begin
28      case curr_state is
29          when S0 =>
30              if din = '1' then next_state <= S1;
31              else next_state <= S0; end if;
32          when S1 =>
33              if din = '0' then next_state <= S2;
34              else next_state <= S1; end if;
35          when S2 =>
36              if din = '1' then next_state <= S3;
37              else next_state <= S0; end if;
38          when S3 =>
39              if din = '1' then next_state <= S4;
40              else next_state <= S2; end if;
41          when S4 =>
42              if din = '1' then next_state <= S1;    -- 重叠
43              else next_state <= S2; end if;
44          when others => next_state <= S0;
45      end case;
46  end process comb_proc;
47
48  -- Moore输出：仅由当前状态决定
49  y <= '1' when curr_state = S4 else '0';
50 end architecture behav;

```

Listing 35.2: Moore 型 1011 序列检测器

【Mealy 型——4 状态，可重叠，默认赋值】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity seq1011_mealy is
5      port(
6          clk : in  std_logic;
7          rst : in  std_logic;    -- 异步高电平复位

```

```
8     din : in  std_logic;
9     y   : out std_logic
10 );
11 end entity seq1011_mealy;
12
13 architecture behav of seq1011_mealy is
14     type state_type is (S0, S1, S2, S3);
15     signal curr_state, next_state : state_type;
16 begin
17     -- 第一段：时序逻辑（状态寄存器）
18     reg_proc: process(clk, rst)
19     begin
20         if rst = '1' then curr_state <= S0;
21         elsif rising_edge(clk) then curr_state <= next_state;
22         end if;
23     end process reg_proc;
24
25     -- 第二段：组合逻辑（状态转移 + Mealy输出）
26     comb_proc: process(curr_state, din)
27     begin
28         -- 默认赋值，避免组合逻辑锁存
29         next_state <= S0;
30         y <= '0';
31
32         case curr_state is
33             when S0 =>
34                 if din = '1' then next_state <= S1;
35                 else next_state <= S0; end if;
36             when S1 =>
37                 if din = '0' then next_state <= S2;
38                 else next_state <= S1; end if;
39             when S2 =>
40                 if din = '1' then next_state <= S3;
41                 else next_state <= S0; end if;
42             when S3 =>
43                 if din = '1' then
44                     next_state <= S1;
45                     y <= '1'; -- 检测到1011，输出有效（重叠）
46                 else
47                     next_state <= S2;
48                 end if;
49             when others => next_state <= S0;
50         end case;
51     end process comb_proc;
52 end architecture behav;
```

Listing 35.3: Mealy 型 1011 序列检测器

【关键代码差异】

- **Moore:** 5 状态，输出在进程外: `y <= '1' when curr_state = S4 else '0';`
- **Mealy:** 4 状态，输出在 `comb_proc` 内: `y <= '1';`（默认为 0，检测到时置 1）
- **默认赋值（Mealy 必须）:** `next_state <= S0; y <= '0';` 写在 `case` 之前，避免锁存
- **可重叠/非重叠切换:** 改 S4/S3 的 `din=1` 跳转——回 S0= 非重叠，回 S1= 重叠

35.6.6 输出时序对比

【Moore 型输出晚 1 拍】

时钟拍	1	2	3	4	5
输入 din	1	0	1	1	0
Moore curr_state	S1	S2	S3	S4	S1
Moore 输出 y	0	0	0	1（第 4 拍!）	0
Mealy curr_state	S1	S2	S3	S1	S2
Mealy 输出 y	0	0	0	1（第 4 拍!）	0

注：严格 5 状态 Moore 进入 S4 即输出 $y = 1$ ，在第 4 拍（与 Mealy 同拍）输出。Moore 输出仅取决于当前状态是否为 S4，与输入 `din` 无关，这是 Moore 型与 Mealy 型的根本区别。

35.6.7 考试如何选择

Moore vs Mealy 考试策略:

1. 默认选 **Moore 型（5 状态）**：状态图更清晰，输出稳定，符合教材规范
2. 题目明确要求 **Mealy 型**时才用 **Mealy（4 状态）**
3. 状态图标注规范：
 - Moore：状态圈内写” 状态名/输出值”（如 S0/0, S4/1）
 - Mealy：转移弧上写” 输入值/输出值”（如 1/1）
4. **VHDL 必须用两段式**：时序 `reg_proc` + 组合 `comb_proc`
5. **Mealy 必须有默认赋值**：避免组合逻辑锁存

35.6.8 硬件综合结果对比

硬件模块	Moore 型	Mealy 型
状态寄存器	3 个 D 触发器（编码 5 状态）	2 个 D 触发器（编码 4 状态）
次态逻辑	组合电路（当前状态 +din → 下一状态）	组合电路（当前状态 +din → 下一状态）
输出逻辑	独立组合电路（仅读当前状态）	合并于次态逻辑中（读当前状态 +din）
输出信号 y	S4 时 y=1，与 din 无关	S3 且 din=1 时 y=1，随转移产生
关键路径延迟	寄存器延迟 + 次态逻辑延迟	寄存器延迟 + 次态/输出组合延迟
输出毛刺风险	低（仅状态变化时输出才变）	较高（din 直接参与输出）

结论：Moore 型输出稳定、无毛刺，适合对时序要求严格的场合。Mealy 型状态少、响应快，但输出可能受输入毛刺影响。考试中优先使用 Moore 型。

35.7 考试常见变式

1. 变式 1：检测 1011（重叠）

检测后在 S3 状态，输入 1→检测到，同时此 1 可作为新序列的第一个 1，因此去 S1（不是 S0）。

2. 变式 2：检测 0101

状态：S0（初始）→S1（匹配”0”）→S2（匹配”01”）→S3（匹配”010”）。S3+ 输入 1= 检测到。

3. 变式 3：检测 1111

4 个连续的 1。状态：S0→S1（1 个 1）→S2（2 个 1）→S3（3 个 1）。S3+ 输入 1= 检测到（重叠：保持 S3）。

4. 变式 4：同时检测多个序列

如同时检测 1011 和 1101。需要合并两个状态机。

5. 变式 5：Mealy 型 vs Moore 型检测器

Mealy：输出取决于状态 + 输入（输出可以早一拍） Moore：输出仅取决于状态（输出晚一拍，但更稳定）

考试通常要求 Moore 型。

35.8 Testbench

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity seq1011_moore_tb is
5  end seq1011_moore_tb;
6
7  architecture test of seq1011_moore_tb is
8      signal clk, rst, din, y : std_logic;
9      constant T : time := 20 ns;
10
11     component seq1011_moore is
12         port(clk, rst, din : in std_logic; y : out std_logic);
13     end component;
14 begin
15     uut: seq1011_moore port map(clk, rst, din, y);
16
17     clk <= not clk after T/2;
18
19     stimulus: process
20     begin
21         rst <= '1'; wait for 30 ns;  -- 异步高电平复位
22         rst <= '0'; wait for 5 ns;
23
24         -- 输入序列: 1 0 1 1 0 1 0 1 1 (可重叠验证)
25         din <= '1'; wait for T;  -- S0→S1
26         din <= '0'; wait for T;  -- S1→S2
27         din <= '1'; wait for T;  -- S2→S3
28         din <= '1'; wait for T;  -- S3→S4, y=1 (检测到!)
29         din <= '0'; wait for T;
30         din <= '1'; wait for T;
31         din <= '0'; wait for T;
32         din <= '1'; wait for T;
33         din <= '1'; wait for T;  -- 第9拍再次检测到!
34         wait;
35     end process;
36 end test;

```

Listing 35.4: Moore 型 1011 序列检测器 Testbench (异步高电平复位)

序列检测器秒杀法：第一步：确定状态状态 = 已匹配的序列前缀（每个状态代表匹配了序列的前几位）

第二步：画状态图对每个状态，考虑输入 0 和输入 1 分别去哪里

第三步：写状态转移表

第四步：写 VHDL

- type 定义状态枚举
- case 语句处理每个状态
- 检测到条件：在倒数第二个状态 + 最后一个匹配位

关键技巧：状态 S3 下输入 1→ 检测到。但去哪？

- 重叠检测：分析后几位能否作为新前缀 →S1（”1011” 最后的”1” 可以是新序列的第一个”1”）
- 非重叠检测：直接回 S0

35.9 状态机的其他应用——不止序列检测器

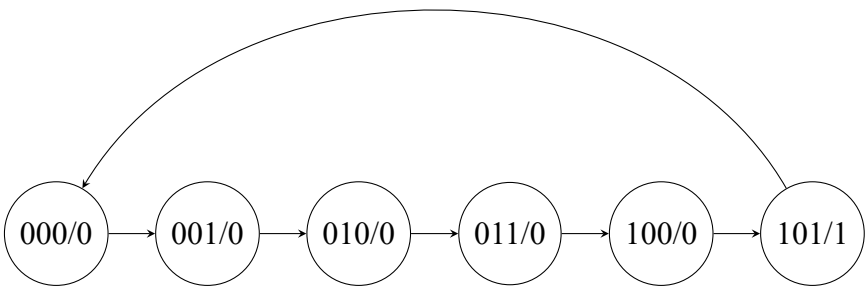
序列检测器是状态机最经典的应用，但状态机的应用远不止于此。实际上，前面学过的很多电路本质上都是状态机。

35.9.1 计数器 = 状态机

用状态机的眼光重新看模 6 计数器：

状态机要素	模 6 计数器对应	硬件实现
当前状态	cnt[2:0]（3 位 D 触发器）	3 个 DFF
次态逻辑	$cnt_{next} = cnt + 1$ （若 $cnt = 5$ 则归 0）	加法器 + 比较器
输出逻辑	count = cnt, carry = 1 when cnt=5	连线 + 比较器

状态转移图（模 6 计数器）：



(状态标注格式: 计数值/carry 输出)

用直观状态机写法重写模 6 计数器:

```

1 architecture Behavioral of counter_mod6_fsm is
2     type state is (s0,s1,s2,s3,s4,s5);
3     signal present_state, next_state: state;
4 begin
5     process(rst, clk)
6     begin
7         if rst='0' then present_state <= s0;
8         elsif clk'event and clk='1' then
9             present_state <= next_state;
10        end if;
11    end process;
12
13    COM: process(present_state)
14    begin
15        case present_state is
16            when s0 => count<="000"; carry<='0'; next_state<=s1;
17            when s1 => count<="001"; carry<='0'; next_state<=s2;
18            when s2 => count<="010"; carry<='0'; next_state<=s3;
19            when s3 => count<="011"; carry<='0'; next_state<=s4;
20            when s4 => count<="100"; carry<='0'; next_state<=s5;
21            when s5 => count<="101"; carry<='1'; next_state<=s0;
22            when others => count<="000"; carry<='0'; next_state<=s0;
23        end case;
24    end process COM;
25 end Behavioral;

```

Listing 35.5: 模 6 计数器——每个状态一行, 输出和下一拍一起写

对比: 上面用的是**枚举 6 个状态**的写法 (6 个 when 分支), 下面是用加法器 + 清零的写法:

```

1 if cnt = 5 then cnt <= 0; else cnt <= cnt + 1; end if;

```

Listing 35.6: 模 6 计数器——传统写法 (等价但更简洁)

两段代码**完全等价**——综合出来的硬件一模一样。但枚举写法让你看清: 计数器就是一个 6 状态的循环状态机。

拓展练习: 用状态机实现模 5 计数器

将上面模 6 的方法直接套用到模 5: 5 个状态 s0-s4。

```

1 type state is (s0,s1,s2,s3,s4);
2 signal present_state, next_state: state;

```



```

3  ...
4  COM: process(present_state)
5  begin
6      case present_state is
7          when s0 => count<="000"; next_state<=s1;
8          when s1 => count<="001"; next_state<=s2;
9          when s2 => count<="010"; next_state<=s3;
10         when s3 => count<="011"; next_state<=s4;
11         when s4 => count<="100"; next_state<=s0;
12         when others => count<="000"; next_state<=s0;
13     end case;
14 end process COM;

```

Listing 35.7: 模 5 计数器——直观状态机写法

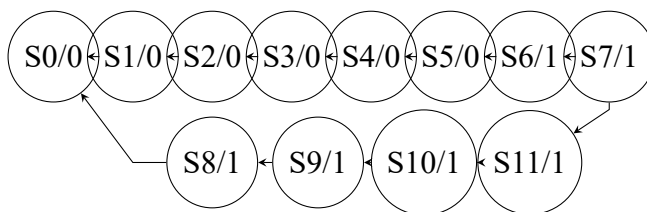
任意模 N 计数器都可以这样写—— N 个状态， N 个 **when** 分支。每个分支写清楚当前状态输出什么值、下一拍去哪里。模 12=12 行，模 24=24 行，以此类推。

35.9.2 分频器 = 状态机

分频器的本质：一个状态在 M 个状态间循环，输出在特定状态翻转。

例题：12 分频器（占空比 50%）。 $M = 12$ ，需要 12 个状态。前 6 个状态输出 0，后 6 个状态输出 1。

状态转移图（12 状态循环）：



（S0-S5 输出 0，S6-S11 输出 1，12 个状态循环。6 拍低 + 6 拍高 = 50% 占空比。）

VHDL——直观状态机写法：

```

1  architecture Behavioral of div12_fsm is
2      type state is (s0,s1,s2,s3,s4,s5,s6,s7,s8,s9,s10,s11);
3      signal present_state, next_state: state;
4  begin
5      process(rst, clk)
6      begin
7          if rst='0' then present_state <= s0;
8          elsif clk'event and clk='1' then
9              present_state <= next_state;
10             end if;
11         end process;

```

```

12
13 COM: process(present_state)
14 begin
15     case present_state is
16         when s0 => clk_out<='0'; next_state<=s1;
17         when s1 => clk_out<='0'; next_state<=s2;
18         when s2 => clk_out<='0'; next_state<=s3;
19         when s3 => clk_out<='0'; next_state<=s4;
20         when s4 => clk_out<='0'; next_state<=s5;
21         when s5 => clk_out<='0'; next_state<=s6;
22         when s6 => clk_out<='1'; next_state<=s7;
23         when s7 => clk_out<='1'; next_state<=s8;
24         when s8 => clk_out<='1'; next_state<=s9;
25         when s9 => clk_out<='1'; next_state<=s10;
26         when s10 => clk_out<='1'; next_state<=s11;
27         when s11 => clk_out<='1'; next_state<=s0;
28         when others => clk_out<='0'; next_state<=s0;
29     end case;
30 end process COM;
31 end Behavioral;

```

Listing 35.8: 12 分频器——每个状态一行，输出和转移一起写

和传统计数器写法对比：

```

1 if cnt=11 then cnt<=0; else cnt<=cnt+1; end if;
2 clk_out <= '0' when cnt < 6 else '1';

```

Listing 35.9: 传统写法（模 12 计数器 + 比较）

同样等价！状态机 12 状态枚举写法更直观——你看状态图就知道输出波形是什么。计数器写法更紧凑——4 位计数器 + 比较器搞定。考试中两种写法都正确，状态机写法更容易画状态转移图拿步骤分。

35.9.3 移位寄存器 = 状态机

移位寄存器的“状态”就是 N 个 D 触发器的内容——一个 N 位向量，总共 2^N 种可能。次态函数非常简单： $Q_{next} = \{si, Q[N-1:1]\}$ （右移）或 $\{Q[N-2:0], si\}$ （左移）。

本质上它是状态机，但次态逻辑太简单了——一行拼接就表达完毕，不需要 case 枚举。

移位寄存器是状态机——但你不会用 case 去写它。

4 位移位寄存器有 $2^4 = 16$ 个状态。理论上可以枚举 16 个状态用 case 写——比如状态“0000”输入 1→下一拍“1000”；状态“1000”输入 0→下一拍“0100”... 以此类推，16

个状态 $\times 2$ 种输入 = 32 个 when 分支。

但没人会这样写——太蠢了。一行 `reg <= si & reg(3 downto 1)` 就搞定的事情，写 32 行 case 毫无意义。

这说明一个重要区别：

- 有些电路的次态逻辑有简洁的数学表达（移位 = 拼接、计数 $= +1$ ） $\rightarrow$ 不需要枚举状态
- 有些电路没有（序列检测的前缀匹配、售货机的金额累计） $\rightarrow$ 必须枚举状态

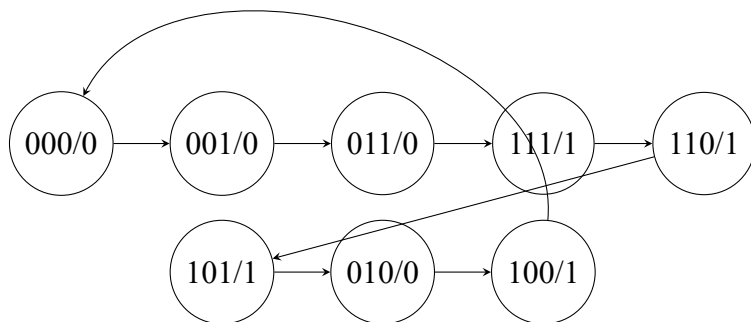
什么时候必须用显式状态机？次态逻辑没有简洁公式时。序列检测器检测 1011——你没法用一个数学表达式描述“已匹配 101 时输入 1 $\rightarrow$ 检测到”，所以必须枚举 S0/S1/S2/S3 四个状态。计数器和移位寄存器的次态有简洁公式（+1 和拼接），用传统写法即可。

环形计数器和 Johnson 计数器是移位寄存器的特例：反馈线不同，同样不需要显式状态机。

35.9.4 序列发生器 = 状态机

序列发生器是无外部输入的状态机——没有 din，次态只取决于当前状态。以序列 00011101 为例：

状态转移图（8 状态，无输入）：



（状态标注格式：Q/输出值。转移弧上无标注——因为没有外部输入。）

注意和序列检测器的对比：

- 检测器弧上有输入值 (0/1)，因为去哪取决于 din
- 发生器弧上没有输入，因为去哪是确定的（CASE 表写死了）

用状态机写法实现：

```

1  architecture Behavioral of seqgen_fsm is
2      type state is (s0,s1,s2,s3,s4,s5,s6,s7);
3      signal present_state, next_state: state;
4  begin
5      -- 状态寄存器
6      process(rst, clk)
7      begin
8          if rst='0' then present_state <= s0;
9          elsif clk'event and clk='1' then
10             present_state <= next_state;
11          end if;
12      end process;
13
14      -- 次态逻辑 + 输出（每个状态一行：输出什么 + 下一拍去哪）
15      COM: process(present_state)
16      begin
17          case present_state is
18              when s0 => q <= '0'; next_state <= s1;
19              when s1 => q <= '0'; next_state <= s2;
20              when s2 => q <= '0'; next_state <= s3;
21              when s3 => q <= '1'; next_state <= s4;
22              when s4 => q <= '1'; next_state <= s5;
23              when s5 => q <= '1'; next_state <= s6;
24              when s6 => q <= '0'; next_state <= s7;
25              when s7 => q <= '1'; next_state <= s0;
26              when others => q <= '0'; next_state <= s0;
27          end case;
28      end process COM;
29  end Behavioral;

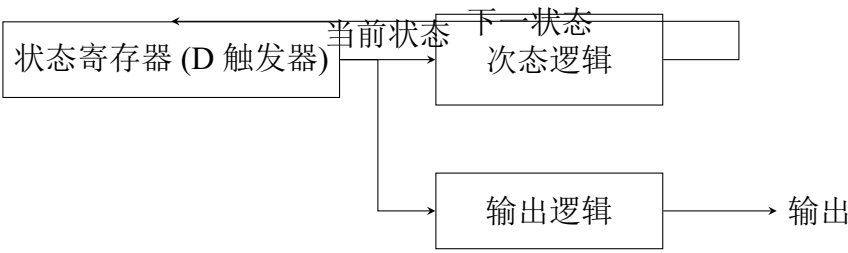
```

Listing 35.10: 序列发生器 00011101——直观状态机写法

这种写法的好处：每个状态一行——when s3 => q <= '1'; next_state <= s4; → 状态 s3 输出 1，下一拍去 s4。8 个状态 8 行，序列 00011101 直接写在代码里，一目了然。和状态转移图完全对应——图上每个圈写什么，代码里每个 when 就写什么。

和移位寄存器 +CASE 表写法对比：两者功能完全相同。状态机枚举写法更直观（8 个状态看得清清楚楚），查表写法更紧凑（不需要枚举状态类型）。

35.9.5 统一视角



所有同步时序电路都遵循这个结构

电路	次态逻辑	输出逻辑	输入
计数器	+1 加法器	计数值/进位	无（或 en/load）
移位寄存器	移位拼接	并行/串行数据	si/din
序列发生器	CASE 查表	Q 最高位	无
序列检测器	前缀匹配判断	detected 信号	din 串行输入

核心结论：

电路	次态逻辑	需要显式状态机？
计数器	$cnt + 1$ （有简洁数学公式）	不需要，一行 $cnt \leq cnt + 1$ 即可
移位寄存器	拼接运算（有简洁公式）	不需要，一行 $reg \leq si \& reg$ 即可
序列发生器	CASE 查表/枚举	可以（枚举更直观）
序列检测器	前缀匹配判断	必须——没有简洁公式

什么时候用显式状态机：次态逻辑没有简洁数学表达式的时候。计数器 +1、移位拼接——一行代码搞定。序列检测的前缀匹配、售货机的金额累计——必须老老实实画状态图、写 case 枚举。不是所有电路都要写成状态机，但所有电路都可以用状态机来理解。

35.10 变式详解

第三十六章 序列检测器——全部变式详解

36.1 变式 1：检测 1011（重叠 vs 非重叠）

完整教学

【核心区别】S3+ 输入 1→ 检测到！

- 非重叠 → 回 S0（重新开始）
- 重叠 → 去 S1（最后的”1”作为新序列的第一个”1”）

【重叠检测版本——完整 VHDL（Moore 型，5 状态）】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity seq1011_moore is
5     port(
6         clk : in  std_logic;
7         rst : in  std_logic;    -- 异步高电平复位
8         din : in  std_logic;
9         y   : out std_logic
10    );
11 end entity seq1011_moore;
12
13 architecture behav of seq1011_moore is
14     type state_type is (S0, S1, S2, S3, S4);
15     signal curr_state, next_state : state_type;
16 begin
17     reg_proc: process(clk, rst)
18     begin
19         if rst = '1' then curr_state <= S0;
20         elsif rising_edge(clk) then curr_state <= next_state;
```

```

21     end if;
22 end process reg_proc;
23
24 comb_proc: process(curr_state, din)
25 begin
26     case curr_state is
27         when S0 =>
28             if din = '1' then next_state <= S1;
29             else next_state <= S0; end if;
30         when S1 =>
31             if din = '0' then next_state <= S2;
32             else next_state <= S1; end if;
33         when S2 =>
34             if din = '1' then next_state <= S3;
35             else next_state <= S0; end if;
36         when S3 =>
37             if din = '1' then next_state <= S4;
38             else next_state <= S2; end if;
39         when S4 =>
40             if din = '1' then next_state <= S1;    -- 重叠! 末尾1→新序
41             else next_state <= S2; end if;
42             -- 列起始
43         when others => next_state <= S0;
44     end case;
45 end process comb_proc;
46
47 y <= '1' when curr_state = S4 else '0';
48 end architecture behav;

```

Listing 36.1: 1011 序列检测器——Moore 型可重叠

【考试判断】看 S4 状态的 din=1 跳转目标。S4+1→S0= 非重叠，S4+1→S1= 可重叠。

36.2 变式 2: 检测 1101

完整教学

【状态】S0→S1→S2→S3→(输入 1= 检测到) 【重叠处理】S3+1→检测到 → 去 S2 (“1101” 中后两位“11” 匹配“11” 前缀) 【关键】S2+1→S2 (“111” 中后两个“11” 为有效前缀)。

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity seq1101_moore is
5      port(
6          clk : in  std_logic;
7          rst : in  std_logic;
8          din : in  std_logic;
9          y   : out std_logic
10     );
11 end entity seq1101_moore;
12
13 architecture behav of seq1101_moore is
14     type state_type is (S0, S1, S2, S3, S4);
15     signal curr_state, next_state : state_type;
16 begin
17     reg_proc: process(clk, rst)
18     begin
19         if rst = '1' then curr_state <= S0;
20         elsif rising_edge(clk) then curr_state <= next_state;
21         end if;
22     end process reg_proc;
23
24     comb_proc: process(curr_state, din)
25     begin
26         case curr_state is
27             when S0 =>
28                 if din = '1' then next_state <= S1;
29                 else next_state <= S0; end if;
30             when S1 =>
31                 if din = '0' then next_state <= S0;
32                 else next_state <= S2; end if;
33             when S2 =>
34                 if din = '0' then next_state <= S3;
35                 else next_state <= S2; end if;    -- din=1保持S2
36             when S3 =>
37                 if din = '1' then next_state <= S4;
38                 else next_state <= S0; end if;
39             when S4 =>
40                 if din = '1' then next_state <= S2;    -- 重叠
41                 else next_state <= S0; end if;
42             when others => next_state <= S0;
43         end case;
```



```

44     end process comb_proc;
45
46     -- Moore输出: 仅取决于状态
47     y <= '1' when curr_state = S4 else '0';
48 end architecture behav;

```

Listing 36.2: 1101 序列检测器——Moore 型可重叠 (5 状态)

36.3 变式 3: 检测 1111 (连续 4 个 1)

完整教学

【状态】S0→S1(1 个 1)→S2(2 个 1)→S3(3 个 1)→(第 4 个 1→检测) 【特点】任意时刻输入 0→回 S0 (连续中断)。【重叠】S3+1→S3 (“1111”后 3 个“111”为有效前缀)。

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity seq1111_moore is
5      port(
6          clk : in  std_logic;
7          rst : in  std_logic;
8          din : in  std_logic;
9          y   : out std_logic
10     );
11 end entity seq1111_moore;
12
13 architecture behav of seq1111_moore is
14     type state_type is (S0, S1, S2, S3, S4);
15     signal curr_state, next_state : state_type;
16 begin
17     reg_proc: process(clk, rst)
18     begin
19         if rst = '1' then curr_state <= S0;
20         elsif rising_edge(clk) then curr_state <= next_state;
21         end if;
22     end process reg_proc;
23
24     comb_proc: process(curr_state, din)

```

```
25  begin
26      case curr_state is
27          when S0 =>
28              if din = '1' then next_state <= S1;
29              else next_state <= S0; end if;
30          when S1 =>
31              if din = '1' then next_state <= S2;
32              else next_state <= S0; end if;
33          when S2 =>
34              if din = '1' then next_state <= S3;
35              else next_state <= S0; end if;
36          when S3 =>
37              if din = '1' then next_state <= S4;
38              else next_state <= S0; end if;
39          when S4 =>
40              if din = '1' then next_state <= S4;    -- 重叠! 保持 S4
41              else next_state <= S0; end if;
42          when others => next_state <= S0;
43      end case;
44  end process comb_proc;
45
46  y <= '1' when curr_state = S4 else '0';
47 end architecture behav;
```

Listing 36.3: 1111 序列检测器——Moore 型可重叠 (5 状态)

36.4 变式 4: 检测 0101

完整教学

【状态】检测 0101 (4 位序列), 状态按 S0-S3 顺序命名, 每个状态含义取决于目标序列:

- S0: 初始 (匹配 0 位)
- S1: 已匹配“0” (序列第一位)
- S2: 已匹配“01” (序列前两位)
- S3: 已匹配“010” (序列前三位, 等待最后一位 1)

【注意】S3+1→检测到；S3+0→S1（重叠：“0100”最后的“0”可作为新序列第一个“0”）。

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity seq0101_moore is
5      port(
6          clk : in  std_logic;
7          rst : in  std_logic;
8          din : in  std_logic;
9          y   : out std_logic
10     );
11 end entity seq0101_moore;
12
13 architecture behav of seq0101_moore is
14     type state_type is (S0, S1, S2, S3, S4);
15     signal curr_state, next_state : state_type;
16 begin
17     reg_proc: process(clk, rst)
18     begin
19         if rst = '1' then curr_state <= S0;
20         elsif rising_edge(clk) then curr_state <= next_state;
21         end if;
22     end process reg_proc;
23
24     comb_proc: process(curr_state, din)
25     begin
26         case curr_state is
27             when S0 =>
28                 if din = '0' then next_state <= S1;
29                 else next_state <= S0; end if;
30             when S1 =>
31                 if din = '1' then next_state <= S2;      -- "01"
32                 else next_state <= S1; end if;            -- "00"保持
33             when S2 =>
34                 if din = '0' then next_state <= S3;      -- "010"
35                 else next_state <= S0; end if;
36             when S3 =>
37                 if din = '1' then next_state <= S4;      -- "0101"
38                 else next_state <= S1; end if;            -- "0100"→S1
39             when S4 =>
40                 if din = '0' then next_state <= S1;      -- 重叠

```

```

41     else next_state <= S0; end if;
42     when others => next_state <= S0;
43 end case;
44 end process comb_proc;
45
46 y <= '1' when curr_state = S4 else '0';
47 end architecture behav;

```

Listing 36.4: 0101 序列检测器——Moore 型可重叠（5 状态）

36.5 变式 5：用移位寄存器 + 比较器替代状态机

完整教学

【方案】 移位寄存器存最近 N 位 → 与目标序列比较。**【完整 VHDL】**

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity seq_detector_shift is
5      port(
6          clk : in  std_logic;
7          rst : in  std_logic;
8          din : in  std_logic;
9          y   : out std_logic
10     );
11 end seq_detector_shift;
12
13 architecture behavioral of seq_detector_shift is
14     signal reg : std_logic_vector(3 downto 0);
15 begin
16     process(clk, rst)
17     begin
18         if rst = '1' then
19             reg <= (others => '0');
20         elsif rising_edge(clk) then
21             reg <= din & reg(3 downto 1);
22         end if;
23     end process;
24
25     y <= '1' when reg = "1011" else '0';

```

```
26 end behavioral;
```

Listing 36.5: 移位寄存器 + 比较器实现 1011 序列检测

【优点】代码极简。不需推导状态转移。**【注意】**自动实现重叠检测。**【考试适用】**除非题目明确要求“用状态机设计”，否则移位寄存器方案是最佳选择。

第九部分

VHDL 硬件综合专题

第三十七章 VHDL 硬件综合专题

37.1 学习目标

不是学 VHDL 语法。而是理解：

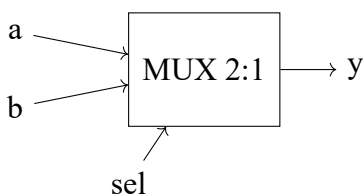
你写的每一行 VHDL，最终在 FPGA 中变成了什么硬件？

37.2 if-else 综合成什么

```
1  if sel = '1' then
2    y <= a;
3  else
4    y <= b;
5  end if;
```

Listing 37.1: if-else 示例

综合结果：MUX（数据选择器）



37.2.1 if-else 级联

```
1  if sel = "00" then
2    y <= d0;
3  elsif sel = "01" then
4    y <= d1;
5  elsif sel = "10" then
6    y <= d2;
7  else
```

```

8   y <= d3;
9   end if;

```

综合结果：优先级编码的选择器（级联 MUX）。sel=11 优先级最低（在最后的 else 中），综合工具可能优化为并行 MUX。

if-else 与硬件：if-else → 优先级逻辑（如优先级编码器、级联 MUX）
 条件从上到下优先级递减。
 如果条件互斥，综合工具常优化为并行 case 结构。

37.3 case 综合成什么

```

1   case sel is
2     when "00" => y <= d0;
3     when "01" => y <= d1;
4     when "10" => y <= d2;
5     when "11" => y <= d3;
6   end case;

```

综合结果：并行 MUX（所有分支并行判断）
 与 if-else 不同，case 的所有分支并行判断，没有优先级。

case 与硬件：case → 并行选择器 / 译码器结构
 所有分支地位平等，综合器生成并行比较逻辑。
 适用于：

- MUX（选择器）
- 译码器
- 状态机的状态转移
- 查找表

37.4 process 综合成什么

```

1   process(clk)
2   begin
3     if rising_edge(clk) then
4       q <= d;

```



```
5   end if;  
6 end process;
```

process 块本身不产生硬件。**process** 只是一种描述方式。
process 内部的逻辑决定综合成什么：

- 有 **clk** 边沿检测 → 产生 D 触发器/寄存器
- 纯组合逻辑（无 **clk**）→ 产生组合逻辑门
- 带异步复位 → D 触发器 + 复位逻辑

process 与硬件：**process** = 硬件描述的容器（本身不是硬件）
process(**CLK**) + **rising_edge** → D 触发器/寄存器组
process(全部输入) + 无 **clk** → 组合逻辑
决定硬件的是 **process** 内部的代码，不是 **process** 本身。

37.5 signal 对应什么硬件

```
1 signal x : std_logic;  
2 signal data : std_logic_vector(7 downto 0);
```

signal = 一根线（组合逻辑）或一个寄存器（时序逻辑）

- **signal x : std_logic** → 1 根物理连线或 1 个 D 触发器
- **signal data : std_logic_vector(7 downto 0)** → 8 根线或 8 个 D 触发器

关键判断：**signal** 是线还是寄存器？

如果 signal 在 process (CLK) 内被赋值	D 触发器/寄存器
如果 signal 在组合逻辑 process 或并行语句中被赋值	连线

37.6 variable 对应什么硬件

```
1 process(clk)  
2   variable tmp : std_logic_vector(3 downto 0);  
3 begin  
4   if rising_edge(clk) then  
5     tmp := a + b;      -- 立即生效  
6     result <= tmp;     -- 将变量值赋给信号  
7   end if;  
8 end process;
```

variable = process 内部的临时存储（通常不产生额外硬件）

- variable 的值在赋值后**立即**更新（:=）
- signal 的值在 process **结束时**才更新（<=）
- variable 通常只是中间计算，综合器将其优化为连线
- 但在某些情况下（如在 process 中被读取前先赋值），综合为寄存器

37.7 常见 VHDL 模式的硬件对应

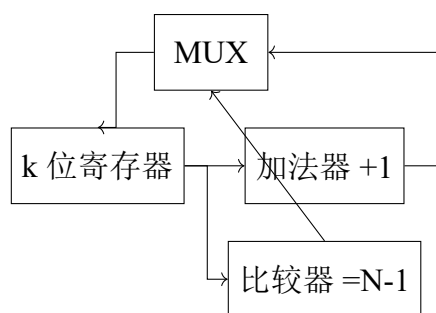
37.7.1 计数器代码 → 什么结构

```

1 process(clk)
2 begin
3     if rising_edge(clk) then
4         if cnt = N-1 then
5             cnt <= 0;
6         else
7             cnt <= cnt + 1;
8         end if;
9     end if;
10 end process;

```

综合结果：



k 个 D 触发器 + 加法器 + 比较器 + MUX

37.7.2 移位寄存器代码 → 什么结构

```

1 reg <= si & reg(N-1 downto 1);  -- 右移

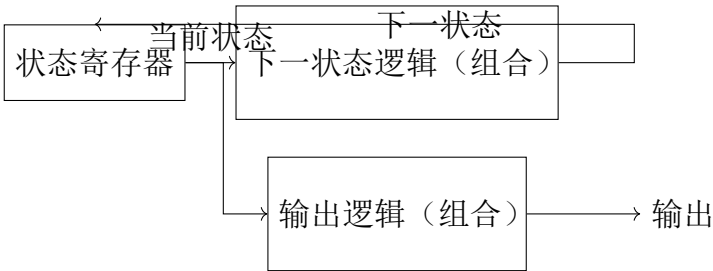
```

综合结果：N 个 D 触发器首尾串联（ $Q_k \rightarrow D_{k+1}$ ），数据每次右移一位。

37.7.3 状态机代码 → 什么结构

```
1 case state is
2   when S0 => if input='1' then state <= S1; end if;
3   when S1 => ...
```

综合结果:



三部分：状态寄存器 + 次态逻辑 + 输出逻辑

37.8 VHDL 编码风格对硬件的影响

VHDL 写法	综合结果	注意事项
if-else 级联	优先级编码	顶层条件优先级最高
case	并行选择	条件必须互斥
rising_edge	上沿触发 D 触发器	产生寄存器
无敏感信号 clk 的 process	组合逻辑	敏感列表必须包含所有输入
signal 赋值 <=	晚更新（process 结束）	适合寄存器
variable 赋值:=	立即更新	适合中间计算
for-generate	并行复制 N 个相同硬件	适合规则结构

37.9 关键综合规则

- 1. 有 **clk** → 产生触发器/寄存器
- 2. 有 **clk + if rising_edge** → 产生正沿触发 D 触发器
- 3. 无 **clk** 的 **process** + **完整敏感列表** → 产生组合逻辑
- 4. **不完整敏感列表** → 可能产生锁存器（latch）——通常应避免
- 5. **case** 不完备（有 **when others**）→ 组合逻辑，不会产生锁存器
- 6. **case** 不完备（无 **when others**）→ 可能产生锁存器——必须加 **when others**

VHDL 综合速查:

VHDL 模式	硬件
if rising_edge(clk) then q <= d;	D 触发器
y <= a when sel='1' else b;	2 选 1 MUX
with sel select y <= ...	MUX（并行）
case state is ...	状态机 + 译码逻辑
reg <= si & reg(N-1 downto 1);	移位寄存器
cnt <= cnt + 1;	加法器 + 寄存器（计数器）
if cnt = N-1 then cnt <= 0;	比较器（清零条件）

37.10 变式详解

第三十八章 VHDL 综合——全部变式详解

38.1 变式 1: if-else 的综合结果

完整教学

【综合规则】

- if-else 级联 → 优先级 MUX 链。上层 if 优先级最高。
- 带 clk 的 if → 产生寄存器 (D 触发器)。
- 不带 clk 的 if → 产生组合逻辑。
- 不完整的 if(缺少 else) → 可能产生锁存器 (Latch)——通常应避免。

【组合逻辑 MUX——完整 VHDL】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity mux_ifelse is
5     port (
6         a    : in  std_logic;
7         b    : in  std_logic;
8         sel  : in  std_logic;
9         y    : out std_logic
10    );
11 end mux_ifelse;
12
13 architecture behavioral of mux_ifelse is
14 begin
15     process(a, b, sel)
16     begin
17         if sel = '1' then
```

```
18     y <= a;
19     else
20         y <= b;
21     end if;
22 end process;
23 end behavioral;
```

Listing 38.1: if-else 综合为组合逻辑 MUX

【时序逻辑（带使能的 D 触发器）——完整 VHDL】

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity dff_en is
5      port (
6          clk : in  std_logic;
7          en  : in  std_logic;
8          d   : in  std_logic;
9          q   : out std_logic
10     );
11 end dff_en;
12
13 architecture behavioral of dff_en is
14 begin
15     process(clk)
16     begin
17         if rising_edge(clk) then
18             if en = '1' then
19                 q <= d;
20             end if;
21         end if;
22     end process;
23 end behavioral;
```

Listing 38.2: if+clk 综合为带使能的 D 触发器

38.2 变式 2: case 的综合结果

完整教学

【综合规则】

- case→ 并行 MUX 或译码器。所有分支并行判断，无优先级。
- case 不完备且无 when others→ 可能产生锁存器。
- case 完备 (有 when others)→ 纯组合逻辑或状态机。

【case 综合示例——完整 VHDL】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity mux_case is
5     port (
6         d0 : in  std_logic;
7         d1 : in  std_logic;
8         d2 : in  std_logic;
9         d3 : in  std_logic;
10        sel : in  std_logic_vector(1 downto 0);
11        y   : out std_logic
12    );
13 end mux_case;
14
15 architecture behavioral of mux_case is
16 begin
17     process(d0, d1, d2, d3, sel)
18     begin
19         case sel is
20             when "00" => y <= d0;
21             when "01" => y <= d1;
22             when "10" => y <= d2;
23             when "11" => y <= d3;
24             when others => y <= '0';
25         end case;
26     end process;
27 end behavioral;
```

Listing 38.3: case 综合为并行 MUX

【if-else vs case 硬件对比】

	if-else	case
综合结果	优先级编码器/级联 MUX	并行 MUX/译码器
优先级	有 (从上到下递减)	无 (并行)
速度	慢 (级联延迟)	快 (并行)
面积	小 (级联结构)	大 (并行结构)

38.3 变式 3: signal vs variable

完整教学

【**signal(<=)**】赋值在 process 结束时更新。适合跨进程通信和寄存器。【**variable(<=)**】赋值立即更新。适合 process 内部中间计算。

【综合结果】

- signal 在时序 process 中 →D 触发器
- signal 在组合 process 中 → 连线
- variable→ 通常不产生额外硬件（被综合器优化为连线）
- 但 variable 在读取前先赋值 → 可能综合为寄存器

【signal vs variable 对比示例——完整 VHDL】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity sig_var_demo is
6   port (
7     clk      : in  std_logic;
8     rst_n    : in  std_logic;
9     a        : in  unsigned(3 downto 0);
10    b        : in  unsigned(3 downto 0);
11    result    : out unsigned(7 downto 0)
12  );
13 end sig_var_demo;
14
15 architecture behavioral of sig_var_demo is
16   signal sig_tmp : unsigned(7 downto 0);  -- signal: process结束后更新
```



```

17 begin
18   process(clk, rst_n)
19     variable var_tmp : unsigned(7 downto 0); -- variable: 立即更新
20   begin
21     if rst_n = '0' then
22       sig_tmp <= (others => '0');
23       result  <= (others => '0');
24     elsif rising_edge(clk) then
25       var_tmp := a * b; -- := 立即赋值
26       sig_tmp <= a * b; -- <= 延迟到 process 结束
27       result  <= var_tmp; -- 使用 variable 的立即值
28     end if;
29   end process;
30 end behavioral;

```

Listing 38.4: signal (<=) 与 variable (:=) 对比

【考试判断】看赋值符号：<=（信号）vs :=（变量）。

38.4 变式 4: process 的综合结果

完整教学

【process 本身不产生硬件】硬件由 process 内部代码决定。

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity process_demo is
5    port (
6      clk    : in  std_logic;
7      rst_n  : in  std_logic;
8      d      : in  std_logic;
9      a      : in  std_logic;
10     b      : in  std_logic;
11     sel    : in  std_logic;
12     q      : out std_logic;
13     y      : out std_logic
14   );
15 end process_demo;
16

```

```
17 architecture behavioral of process_demo is
18     signal q_reg : std_logic;
19 begin
20     -- (1) 时序 process: process(clk)+rising_edge → D触发器
21     process(clk)
22     begin
23         if rising_edge(clk) then
24             q_reg <= d;
25         end if;
26     end process;
27
28     -- (2) 时序 process带异步复位: process(clk,rst) → D触发器+异步复位
29     process(clk, rst_n)
30     begin
31         if rst_n = '0' then
32             q <= '0';
33         elsif rising_edge(clk) then
34             q <= d;
35         end if;
36     end process;
37
38     -- (3) 组合 process: process(all inputs)+无 clk → 组合逻辑
39     process(a, b, sel)
40     begin
41         if sel = '1' then
42             y <= a;
43         else
44             y <= b;
45         end if;
46     end process;
47 end behavioral;
```

Listing 38.5: 三种典型 process 及其综合结果

process 写法	综合结果
process(clk)+rising_edge	D 触发器/寄存器
process(all_inputs)+ 无 clk	组合逻辑
process(clk,rst)+ 异步 rst	D 触发器 + 异步复位
不完整敏感列表	可能锁存器 (应避免)

38.5 变式 5：常见 VHDL 模式 → 硬件对照表

完整教学

VHDL 模式	硬件
if rising_edge(clk) then Q<=D;	1 个 D 触发器
y <= a when sel='1' else b;	2 选 1 MUX
with sel select y<=...	并行 MUX
case state is when...	状态机 + 译码器
reg <= si & reg(N-1 downto 1);	移位寄存器
cnt <= cnt + 1;	加法器 + 寄存器
if cnt=N-1 then cnt<=0;	比较器 (清零条件)
for i in 0 to 7 generate	8 个相同的硬件副本

考试聚焦

VHDL 综合考点总结：

- 有时钟边沿 = 有触发器
- if-else= 优先级，case= 并行
- signal<= 延迟更新，variable:= 立即更新
- process 只是容器，内部代码决定硬件
- for-generate= 复制硬件 N 份

第十部分

考试训练体系

第三十九章 计数器考试变式题

题目 1：模 7 计数器

题目：设计一个模 7 计数器（0-6 循环），写出 VHDL 代码和 Testbench。

分析：

- $N = 7$, $k = \lceil \log_2 7 \rceil = 3$ 位
- 清零条件：cnt = 6(110)
- 状态序列：000→001→010→011→100→101→110→000

VHDL:

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity mod7_counter is
6     port (
7         clk    : in  std_logic;
8         rst_n   : in  std_logic;
9         q       : out std_logic_vector(2 downto 0)
10    );
11 end mod7_counter;
12
13 architecture behavioral of mod7_counter is
14     signal cnt : unsigned(2 downto 0);
15 begin
16     process(clk, rst_n)
17     begin
18         if rst_n = '0' then
19             cnt <= "000";
20         elsif rising_edge(clk) then
21             if cnt = 6 then
22                 cnt <= "000";
23             else
```

```

24     cnt <= cnt + 1;
25     end if;
26     end if;
27 end process;
28
29 q <= std_logic_vector(cnt);
30 end behavioral;

```

Listing 39.1: 模 7 计数器 (0-6 循环)

题目 2: 模 13 计数器

题目: 设计模 13 计数器 (0-12 循环), 写出 VHDL。

分析:

- $N = 13$, $k = \lceil \log_2 13 \rceil = 4$ 位
- 清零: $\text{cnt} = 12(1100)$

VHDL:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity mod13_counter is
6      port (
7          clk    : in  std_logic;
8          rst_n   : in  std_logic;
9          q       : out std_logic_vector(3 downto 0)
10     );
11 end mod13_counter;
12
13 architecture behavioral of mod13_counter is
14     signal cnt : unsigned(3 downto 0);
15 begin
16     process(clk, rst_n)
17     begin
18         if rst_n = '0' then
19             cnt <= "0000";
20         elsif rising_edge(clk) then
21             if cnt = 12 then
22                 cnt <= "0000";
23             else

```

```
24     cnt <= cnt + 1;
25     end if;
26   end if;
27 end process;
28
29 q <= std_logic_vector(cnt);
30 end behavioral;
```

Listing 39.2: 模 13 计数器 (0-12 循环)

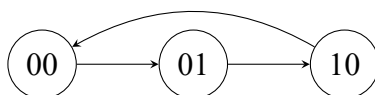
题目 3: 模 3 计数器

题目: 设计模 3 计数器, 画出状态转移图, 写 VHDL。

分析:

- $N = 3$, $k = 2$ 位
- 状态: $00 \rightarrow 01 \rightarrow 10 \rightarrow 00$
- 清零: $\text{cnt}=2(10)$

状态图:



VHDL:

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity mod3_counter is
6    port (
7      clk    : in  std_logic;
8      rst_n  : in  std_logic;
9      q      : out std_logic_vector(1 downto 0)
10   );
11 end mod3_counter;
12
13 architecture behavioral of mod3_counter is
14   signal cnt : unsigned(1 downto 0);
15 begin
16   process(clk, rst_n)
```

```

17  begin
18      if rst_n = '0' then
19          cnt <= "00";
20      elsif rising_edge(clk) then
21          if cnt = 2 then
22              cnt <= "00";
23          else
24              cnt <= cnt + 1;
25          end if;
26      end if;
27  end process;
28
29  q <= std_logic_vector(cnt);
30  end behavioral;

```

Listing 39.3: 模 3 计数器 (0-2 循环)

题目 4: 带使能的模 8 计数器

题目: 设计带使能信号 en 的模 8 计数器。en=1 时计数，en=0 时保持。

VHDL:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity mod8_counter_en is
6      port (
7          clk    : in  std_logic;
8          rst_n  : in  std_logic;
9          en     : in  std_logic;
10         q      : out std_logic_vector(2 downto 0)
11     );
12 end mod8_counter_en;
13
14 architecture behavioral of mod8_counter_en is
15     signal cnt : unsigned(2 downto 0);
16 begin
17     process(clk, rst_n)
18     begin
19         if rst_n = '0' then
20             cnt <= "000";
21         elsif rising_edge(clk) then

```



```
22     if en = '1' then
23         cnt <= cnt + 1;  -- 3位自然溢出=模8
24     end if;
25 end if;
26 end process;
27
28 q <= std_logic_vector(cnt);
29 end behavioral;
```

Listing 39.4: 带使能的模 8 计数器

题目 5: 带同步加载的模 10 计数器

题目: 设计带同步加载 (load) 的模 10 计数器。load=1 时 cnt = data_in; load=0 时正常计数 (0-9)。

VHDL:

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity mod10_counter_load is
6      port (
7          clk      : in  std_logic;
8          rst_n     : in  std_logic;
9          load      : in  std_logic;
10         data_in   : in  std_logic_vector(3 downto 0);
11         q         : out std_logic_vector(3 downto 0)
12     );
13 end mod10_counter_load;
14
15 architecture behavioral of mod10_counter_load is
16     signal cnt : unsigned(3 downto 0);
17 begin
18     process(clk, rst_n)
19     begin
20         if rst_n = '0' then
21             cnt <= "0000";
22         elsif rising_edge(clk) then
23             if load = '1' then
24                 cnt <= unsigned(data_in);
25             elsif cnt = 9 then
26                 cnt <= "0000";
```

```

27     else
28         cnt <= cnt + 1;
29     end if;
30 end if;
31 end process;
32
33 q <= std_logic_vector(cnt);
34 end behavioral;

```

Listing 39.5: 带同步加载的模 10 计数器

题目 6: 递增/递减双向计数器

题目: 设计模 16 双向计数器。up=1 递增，up=0 递减。

VHDL:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity mod16_ud_counter is
6      port (
7          clk      : in  std_logic;
8          rst_n    : in  std_logic;
9          up       : in  std_logic;
10         q        : out std_logic_vector(3 downto 0)
11     );
12 end mod16_ud_counter;
13
14 architecture behavioral of mod16_ud_counter is
15     signal cnt : unsigned(3 downto 0);
16 begin
17     process(clk, rst_n)
18     begin
19         if rst_n = '0' then
20             cnt <= "0000";
21         elsif rising_edge(clk) then
22             if up = '1' then
23                 cnt <= cnt + 1;
24             else
25                 cnt <= cnt - 1;
26             end if;
27         end if;

```

```
28     end process;
29
30     q <= std_logic_vector(cnt);
31 end behavioral;
```

Listing 39.6: 模 16 双向计数器

题目 7: 模 10 BCD 计数器带 7 段显示

题目: 设计模 10 BCD 计数器 + BCD-7 段译码器。输出驱动 7 段数码管显示 0-9。

解:

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity bcd_counter_7seg is
6      port (
7          clk      : in  std_logic;
8          rst_n    : in  std_logic;
9          seg      : out std_logic_vector(6 downto 0)  -- a~g, 低有效
10     );
11 end bcd_counter_7seg;
12
13 architecture behavioral of bcd_counter_7seg is
14     signal cnt : unsigned(3 downto 0);
15 begin
16     -- 模 10 BCD 计数器
17     process(clk, rst_n)
18     begin
19         if rst_n = '0' then
20             cnt <= "0000";
21         elsif rising_edge(clk) then
22             if cnt = 9 then
23                 cnt <= "0000";
24             else
25                 cnt <= cnt + 1;
26             end if;
27         end if;
28     end process;
29
30     -- BCD-7 段译码器 (共阳极: 0 亮 1 灭)
31     process(cnt)
```

```

32  begin
33      case cnt is
34          when "0000" => seg <= "1000000";  -- 0
35          when "0001" => seg <= "1111001";  -- 1
36          when "0010" => seg <= "0100100";  -- 2
37          when "0011" => seg <= "0110000";  -- 3
38          when "0100" => seg <= "0011001";  -- 4
39          when "0101" => seg <= "0010010";  -- 5
40          when "0110" => seg <= "0000010";  -- 6
41          when "0111" => seg <= "1111000";  -- 7
42          when "1000" => seg <= "0000000";  -- 8
43          when "1001" => seg <= "0010000";  -- 9
44          when others => seg <= "1111111";  -- 非法：全灭
45      end case;
46  end process;
47  end behavioral;

```

Listing 39.7: 模 10 BCD 计数器 +7 段译码器

题目 8：模 60 计数器（秒/分计数器）

题目：设计模 60 计数器用于时钟应用。输出秒/分的十位和个位。

解：使用两个 BCD 计数器级联——个位模 10（0-9），十位模 6（0-5）。个位计到 9 时向下一个周期产生进位，十位加 1。当十位 =5 且个位 =9 时，整体回 0。

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity mod60_counter is
6      port (
7          clk      : in  std_logic;
8          rst_n    : in  std_logic;
9          sec_lo   : out std_logic_vector(3 downto 0);  -- 个位 (0-9)
10         sec_hi   : out std_logic_vector(3 downto 0)  -- 十位 (0-5)
11     );
12  end mod60_counter;
13
14  architecture behavioral of mod60_counter is
15      signal cnt_lo : unsigned(3 downto 0);  -- 个位
16      signal cnt_hi : unsigned(3 downto 0);  -- 十位
17  begin
18      process(clk, rst_n)

```

```

19  begin
20      if rst_n = '0' then
21          cnt_lo <= "0000";
22          cnt_hi <= "0000";
23      elsif rising_edge(clk) then
24          if cnt_hi = 5 and cnt_lo = 9 then
25              cnt_lo <= "0000";
26              cnt_hi <= "0000";
27          elsif cnt_lo = 9 then
28              cnt_lo <= "0000";
29              cnt_hi <= cnt_hi + 1;
30          else
31              cnt_lo <= cnt_lo + 1;
32          end if;
33      end if;
34  end process;
35
36  sec_lo <= std_logic_vector(cnt_lo);
37  sec_hi <= std_logic_vector(cnt_hi);
38  end behavioral;

```

Listing 39.8: 模 60 BCD 计数器

题目 9: 模 24 BCD 计数器

题目: 设计模 24 BCD 计数器。0-23 循环。

分析: 十位模 3 (0-2), 个位模 10 (0-9)。清零条件: 23。

解:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity mod24_counter is
6      port (
7          clk      : in  std_logic;
8          rst_n    : in  std_logic;
9          hour_lo  : out std_logic_vector(3 downto 0);  -- 个位 (0-9)
10         hour_hi  : out std_logic_vector(3 downto 0)  -- 十位 (0-2)
11     );
12  end mod24_counter;
13
14  architecture behavioral of mod24_counter is

```

```

15  signal cnt_lo : unsigned(3 downto 0);  -- 个位
16  signal cnt_hi : unsigned(3 downto 0);  -- 十位
17  begin
18  process(clk, rst_n)
19  begin
20      if rst_n = '0' then
21          cnt_lo <= "0000";
22          cnt_hi <= "0000";
23      elsif rising_edge(clk) then
24          if cnt_hi = 2 and cnt_lo = 3 then  -- 23→00
25              cnt_lo <= "0000";
26              cnt_hi <= "0000";
27          elsif cnt_lo = 9 then
28              cnt_lo <= "0000";
29              cnt_hi <= cnt_hi + 1;
30          else
31              cnt_lo <= cnt_lo + 1;
32          end if;
33      end if;
34  end process;
35
36  hour_lo <= std_logic_vector(cnt_lo);
37  hour_hi <= std_logic_vector(cnt_hi);
38  end behavioral;

```

Listing 39.9: 模 24 BCD 计数器

题目 10: 模 100 BCD 计数器

题目: 设计两位 BCD 模 100 计数器 (00-99 循环)。

分析: 个位每满 10 向十位进位。十位满 9+ 个位满 9 (即 99) 时清零。

解:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity mod100_counter is
6  port (
7      clk      : in  std_logic;
8      rst_n    : in  std_logic;
9      cnt_lo   : out std_logic_vector(3 downto 0);  -- 个位 (0-9)
10     cnt_hi   : out std_logic_vector(3 downto 0);  -- 十位 (0-9)

```

```

11 );
12 end mod100_counter;
13
14 architecture behavioral of mod100_counter is
15     signal lo : unsigned(3 downto 0);
16     signal hi : unsigned(3 downto 0);
17 begin
18     process(clk, rst_n)
19     begin
20         if rst_n = '0' then
21             lo <= "0000";
22             hi <= "0000";
23         elsif rising_edge(clk) then
24             if hi = 9 and lo = 9 then -- 99→00
25                 lo <= "0000";
26                 hi <= "0000";
27             elsif lo = 9 then
28                 lo <= "0000";
29                 hi <= hi + 1;
30             else
31                 lo <= lo + 1;
32             end if;
33         end if;
34     end process;
35
36     cnt_lo <= std_logic_vector(lo);
37     cnt_hi <= std_logic_vector(hi);
38 end behavioral;

```

Listing 39.10: 模 100 BCD 计数器

题目 11: 给定计数器时序图, 推断模值

题目: 观察计数器 Q2Q1Q0 的波形: 000→001→010→011→100→000→…。求模值。

解: 出现了 5 个不同状态 (000-100), 所以模值=5。

题目 12: 给定 VHDL 推断模值

题目: 分析以下 VHDL 代码所描述的计数器模值。

```

1 library ieee;
2 use ieee.std_logic_1164.all;

```

```

3  use ieee.numeric_std.all;
4
5  entity mystery_counter is
6      port (
7          clk    : in  std_logic;
8          rst_n   : in  std_logic;
9          q       : out std_logic_vector(3 downto 0)
10     );
11 end mystery_counter;
12
13 architecture behavioral of mystery_counter is
14     signal cnt : unsigned(3 downto 0);
15 begin
16     process(clk, rst_n)
17     begin
18         if rst_n = '0' then
19             cnt <= "0000";
20         elsif rising_edge(clk) then
21             if cnt = "1010" then
22                 cnt <= "0000";
23             else
24                 cnt <= cnt + 1;
25             end if;
26         end if;
27     end process;
28
29     q <= std_logic_vector(cnt);
30 end behavioral;

```

Listing 39.11: 推断模值——完整代码

求模值 N。

解：清零条件是 cnt=10(1010)，所以 N=10+1=11？不对！清零条件 =cnt=1010=10，此时 cnt 还未计数到 11 就被清零了。模值 = 清零条件值 +1=10+1=11？再确认：cnt 从 0 计到 10 清零，所以 N=11。

关键公式：模值 = 清零比较值 + 1

题目 13：分析计数器最大工作频率

题目：4 位行波进位加法器的进位链延迟为每个 FA=2ns，D 触发器 $t_{setup} = 1\text{ns}$ ， $t_{cko} = 1\text{ns}$ 。求计数器的最大工作频率。

分析：关键路径延迟 = $t_{cko} + \text{进位链延迟} (4 \times 2 = 8\text{ns}) + t_{setup} = 1 + 8 + 1 = 10\text{ns}$ $f_{max} = 1/10\text{ns} = 100\text{MHz}$

题目 14：多个计数器级联的分析

题目：模 8 计数器 + 模 5 计数器级联。总模值是多少？

解：模 8 每 8 拍产生一次进位，模 5 每收到进位 +1。总模值 = $8 \times 5 = 40$ 。

通用公式：级联总模值 = $M_1 \times M_2 \times \cdots \times M_k$

题目 15：BCD 计数器的非法状态恢复

题目：模 10 BCD 计数器可能因干扰进入非法状态 (1010-1111)。设计自恢复逻辑。

解：

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity bcd_counter_safe is
6     port (
7         clk    : in  std_logic;
8         rst_n   : in  std_logic;
9         q       : out std_logic_vector(3 downto 0)
10    );
11 end bcd_counter_safe;
12
13 architecture behavioral of bcd_counter_safe is
14     signal cnt : unsigned(3 downto 0);
15 begin
16     process(clk, rst_n)
17     begin
18         if rst_n = '0' then
19             cnt <= "0000";
20         elsif rising_edge(clk) then
21             if cnt >= 10 then      -- 非法状态 (1010-1111)
22                 cnt <= "0000";
23             elsif cnt = 9 then
24                 cnt <= "0000";
25             else
26                 cnt <= cnt + 1;
27             end if;
28         end if;
29     end process;
30
31     q <= std_logic_vector(cnt);
32 end behavioral;
```

Listing 39.12: 自恢复 BCD 计数器（非法状态检测）

题目 16-20：综合应用题

题目 16：数字钟核心

设计时分秒计数器（模 60 秒 + 模 60 分 + 模 24 时）。

解：三个计数器级联——秒 (模 60)→分 (模 60)→时 (模 24)。

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity digital_clock is
6      port (
7          clk      : in  std_logic;
8          rst_n    : in  std_logic;
9          sec_lo   : out std_logic_vector(3 downto 0);
10         sec_hi   : out std_logic_vector(3 downto 0);
11         min_lo   : out std_logic_vector(3 downto 0);
12         min_hi   : out std_logic_vector(3 downto 0);
13         hour_lo  : out std_logic_vector(3 downto 0);
14         hour_hi  : out std_logic_vector(3 downto 0)
15     );
16 end digital_clock;
17
18 architecture behavioral of digital_clock is
19     signal s_lo, s_hi : unsigned(3 downto 0); -- 秒
20     signal m_lo, m_hi : unsigned(3 downto 0); -- 分
21     signal h_lo, h_hi : unsigned(3 downto 0); -- 时
22     signal s_carry : std_logic; -- 秒→分进位
23     signal m_carry : std_logic; -- 分→时进位
24 begin
25     process(clk, rst_n)
26     begin
27         if rst_n = '0' then
28             s_lo <= "0000"; s_hi <= "0000";
29             m_lo <= "0000"; m_hi <= "0000";
30             h_lo <= "0000"; h_hi <= "0000";
31             s_carry <= '0'; m_carry <= '0';
32         elsif rising_edge(clk) then
33             -- 秒计数器（模 60）

```

```
34     if s_hi = 5 and s_lo = 9 then
35         s_lo <= "0000"; s_hi <= "0000";
36         s_carry <= '1';
37     elsif s_lo = 9 then
38         s_lo <= "0000"; s_hi <= s_hi + 1;
39         s_carry <= '0';
40     else
41         s_lo <= s_lo + 1;
42         s_carry <= '0';
43     end if;
44     -- 分计数器（模60，秒进位触发）
45     if s_carry = '1' then
46         if m_hi = 5 and m_lo = 9 then
47             m_lo <= "0000"; m_hi <= "0000";
48             m_carry <= '1';
49         elsif m_lo = 9 then
50             m_lo <= "0000"; m_hi <= m_hi + 1;
51             m_carry <= '0';
52         else
53             m_lo <= m_lo + 1;
54             m_carry <= '0';
55         end if;
56     else
57         m_carry <= '0';
58     end if;
59     -- 时计数器（模24，分进位触发）
60     if m_carry = '1' then
61         if h_hi = 2 and h_lo = 3 then
62             h_lo <= "0000"; h_hi <= "0000";
63         elsif h_lo = 9 then
64             h_lo <= "0000"; h_hi <= h_hi + 1;
65         else
66             h_lo <= h_lo + 1;
67         end if;
68     end if;
69 end if;
70 end process;
71
72 sec_lo <= std_logic_vector(s_lo);
73 sec_hi <= std_logic_vector(s_hi);
74 min_lo <= std_logic_vector(m_lo);
75 min_hi <= std_logic_vector(m_hi);
76 hour_lo <= std_logic_vector(h_lo);
77 hour_hi <= std_logic_vector(h_hi);
78 end behavioral;
```

Listing 39.13: 数字钟核心——时分秒计数器

题目 17: 带暂停的计数器

Pause=1 时暂停计数, pause=0 时恢复。

解:

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity counter_pause is
6      port (
7          clk      : in  std_logic;
8          rst_n    : in  std_logic;
9          pause     : in  std_logic;
10         q        : out std_logic_vector(3 downto 0)
11     );
12 end counter_pause;
13
14 architecture behavioral of counter_pause is
15     signal cnt : unsigned(3 downto 0);
16 begin
17     process(clk, rst_n)
18     begin
19         if rst_n = '0' then
20             cnt <= "0000";
21         elsif rising_edge(clk) then
22             if pause = '0' then
23                 cnt <= cnt + 1;  -- 4位自然溢出=模16
24             end if;
25         end if;
26     end process;
27
28     q <= std_logic_vector(cnt);
29 end behavioral;
```

Listing 39.14: 带暂停的模 16 计数器

题目 18: 带预置位的倒计时计数器

从预置值倒计时到 0, 到 0 后重新加载预置值。

解:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity countdown_preset is
6      port (
7          clk      : in  std_logic;
8          rst_n    : in  std_logic;
9          preset   : in  std_logic_vector(3 downto 0);
10         q        : out std_logic_vector(3 downto 0);
11         zero     : out std_logic  -- 倒计时到0标志
12     );
13 end countdown_preset;
14
15 architecture behavioral of countdown_preset is
16     signal cnt : unsigned(3 downto 0);
17 begin
18     process(clk, rst_n)
19     begin
20         if rst_n = '0' then
21             cnt <= unsigned(preset);
22         elsif rising_edge(clk) then
23             if cnt = 0 then
24                 cnt <= unsigned(preset);  -- 到0后重新加载
25             else
26                 cnt <= cnt - 1;
27             end if;
28         end if;
29     end process;
30
31     q <= std_logic_vector(cnt);
32     zero <= '1' when cnt = 0 else '0';
33 end behavioral;

```

Listing 39.15: 带预置位的倒计时计数器

题目 19: 两相正交计数器

输出两个相位差 90 度的方波（格雷码计数器的一种应用）。

解: 使用 2 位格雷码计数器产生正交输出。序列: 00→01→11→10→00。

```

1  library ieee;

```

```

2  use ieee.std_logic_1164.all;
3
4  entity quadrature_counter is
5      port (
6          clk      : in  std_logic;
7          rst_n    : in  std_logic;
8          phase_a   : out std_logic;  -- 相位A
9          phase_b   : out std_logic  -- 相位B (滞后90度)
10     );
11 end quadrature_counter;
12
13 architecture behavioral of quadrature_counter is
14     signal gray : std_logic_vector(1 downto 0);
15 begin
16     process(clk, rst_n)
17     begin
18         if rst_n = '0' then
19             gray <= "00";
20         elsif rising_edge(clk) then
21             case gray is
22                 when "00" => gray <= "01";
23                 when "01" => gray <= "11";
24                 when "11" => gray <= "10";
25                 when "10" => gray <= "00";
26                 when others => gray <= "00";
27             end case;
28         end if;
29     end process;
30
31     phase_a <= gray(0);
32     phase_b <= gray(1);
33 end behavioral;

```

Listing 39.16: 两相正交计数器（格雷码）

题目 20: 用计数器实现序列检测

计数器状态作为地址，ROM 存放序列，输出 + 检测逻辑。

解：计数器遍历状态作为 ROM 地址，ROM 预先存放待检测序列的各 bit，逐拍与输入比较。

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;

```

```
4
5 entity counter_seq_detector is
6   port (
7     clk      : in  std_logic;
8     rst_n    : in  std_logic;
9     data_in   : in  std_logic;
10    detected  : out std_logic
11  );
12 end counter_seq_detector;
13
14 architecture behavioral of counter_seq_detector is
15   signal addr : unsigned(1 downto 0); -- 0-3, 遍历4个序列位
16   type rom_type is array (0 to 3) of std_logic;
17   -- ROM存放目标序列"1011" (地址0+bit3, 1+bit2, 2+bit1, 3+bit0)
18   constant pattern : rom_type := ('1', '0', '1', '1');
19   signal match : std_logic_vector(3 downto 0); -- 逐位匹配标志
20 begin
21   -- 地址计数器: 0+1+2+3+0循环
22   process(clk, rst_n)
23   begin
24     if rst_n = '0' then
25       addr <= "00";
26     elsif rising_edge(clk) then
27       if addr = 3 then
28         addr <= "00";
29       else
30         addr <= addr + 1;
31       end if;
32     end if;
33   end process;
34
35   -- 比对逻辑
36   process(clk, rst_n)
37   begin
38     if rst_n = '0' then
39       match <= (others => '0');
40     elsif rising_edge(clk) then
41       if data_in = pattern(to_integer(addr)) then
42         match(to_integer(addr)) <= '1';
43       else
44         match(to_integer(addr)) <= '0';
45       end if;
46     end if;
47   end process;
48
```

```
49  -- 全部4位匹配→检测到
50  detected <= '1' when match = "1111" else '0';
51  end behavioral;
```

Listing 39.17: 计数器 +ROM 实现序列检测

计数器类题目的统一解法：第一步：确定模值 N 第二步： $k = \lceil \log_2 N \rceil$ ，确定位宽 第三步：清零条件 = $N-1$ 的二进制 第四步：写 VHDL 模板（if cnt= $N-1$ then cnt<=0 else cnt<=cnt+1） 第五步：如果有级联，低位进位接高位时钟使能
关键公式：

- 模值 = 清零比较值 + 1
- 总模值（级联）= $M_1 \times M_2$
- 2^N 模值不需要显式清零

第四十章 分频器考试变式题

题目 1：2 分频器（直接取 Q0）

题目：用最简单的方法实现 2 分频器。

分析：计数器最低位 Q0 天然 2 分频。只需一个 D 触发器每拍翻转。

VHDL:

```
1 process(clk, rst_n)
2 begin
3     if rst_n = '0' then q <= '0';
4     elsif rising_edge(clk) then q <= not q; end if;
5 end process;
```

输出频率: $f_{out} = f_{clk}/2$

题目 2：4 分频器（直接取 Q1）

题目：用计数器实现 4 分频器。

分析：2 位计数器，Q1=4 分频。直接取 Q1 作为输出。

VHDL:

```
1 signal cnt : std_logic_vector(1 downto 0);
2 process(clk) begin
3     if rising_edge(clk) then cnt <= cnt + 1; end if;
4 end process;
5 clk_div4 <= cnt(1); -- Q1天然4分频
```

题目 3：10 分频器（占空比 50%）

题目：设计 10 分频器，占空比 50%。

分析：

- 模 10 计数器（0-9）

- cnt 0-4 输出 0, cnt 5-9 输出 1
- 每个周期 5 拍低 +5 拍高

VHDL:

```

1 signal cnt : std_logic_vector(3 downto 0);
2 process(clk, rst_n)
3 begin
4     if rst_n = '0' then cnt <= "0000";
5     elsif rising_edge(clk) then
6         if cnt = "1001" then cnt <= "0000";
7         else cnt <= cnt + 1; end if;
8     end if;
9 end process;
10 clk_div10 <= '0' when cnt < "0101" else '1'; -- 0-4低, 5-9高

```

题目 4：3 分频器（占空比非 50%）

题目：设计 3 分频器。

分析：

- 模 3 计数器（0→1→2→0）
- 简单方案：cnt=0,1 输出 0, cnt=2 输出 1（1/3 占空比）
- 50% 占空比方案：需要上升沿 + 下降沿双计数器

题目 5：5 分频器

题目：设计 5 分频器。

分析：模 5 计数器，占空比 3/5 或使用双沿方法实现 50% 占空比。

题目 6：100 分频器

题目：用 50MHz 时钟产生 500kHz 信号，设计分频器。

分析： $M = 50\text{MHz}/500\text{kHz} = 100$ 。模 100 计数器。

VHDL:

```

1 constant M : integer := 100;
2 signal cnt : integer range 0 to M-1;

```

```

3 process(clk, rst_n)
4 begin
5     if rst_n = '0' then cnt <= 0;
6     elsif rising_edge(clk) then
7         if cnt = M-1 then cnt <= 0; else cnt <= cnt + 1; end if;
8     end if;
9 end process;
10 clk_out <= '0' when cnt < M/2 else '1';

```

题目 7：秒脉冲发生器

题目：用 100MHz 时钟产生 1Hz 秒脉冲。

分析： $M = 100,000,000$ 。需要 27 位计数器 ($2^{26} < 10^8 < 2^{27}$)。

题目 8：多级分频器

题目：用 2 分频 +5 分频级联实现 10 分频。

分析： $M_{total} = M_1 \times M_2 = 2 \times 5 = 10$ 。

题目 9：可调分频比的计数器

题目：设计分频比可动态设置的分频器 (DIV 从 1 到 16 可调)。

VHDL:

```

1 signal cnt : std_logic_vector(3 downto 0);
2 process(clk) begin
3     if rising_edge(clk) then
4         if cnt = DIV - 1 then cnt <= "0000";
5         else cnt <= cnt + 1; end if;
6     end if;
7 end process;

```

题目 10：占空比可调的分频器

题目：设计 10 分频器，占空比可调 (高电平持续周期数 DUTY 可配置)。

VHDL:

```

1 clk_out <= '0' when cnt < DUTY else '1';

```

题目 11：正交分频器（I/Q 输出）

题目：设计分频器同时输出 I 和 Q 两路信号，Q 比 I 延迟 90 度。

分析：需要 4 分频才能产生 90 度相位差。I 从 cnt[1] 取，Q 从特定比较值取。

题目 12：50% 占空比的奇数分频器

题目：设计占空比 50% 的 5 分频器。

分析：使用正沿 2 分频 + 负沿 2 分频组合的方法。

题目 13：给定分频器输出波形，推断分频比

题目：输出信号每 8 个输入时钟周期完成一个完整周期，求分频比。

解： $M = 8$ （8 分频）。

题目 14：分析 Qn 分频输出频率

题目：时钟 100MHz，求 4 位计数器 Q0、Q1、Q2、Q3 的输出频率。

解：

- Q0: $100/2 = 50\text{MHz}$
- Q1: $100/4 = 25\text{MHz}$
- Q2: $100/8 = 12.5\text{MHz}$
- Q3: $100/16 = 6.25\text{MHz}$

题目 15：分频器级联频率计算

题目：100MHz $\rightarrow$ 5 分频 $\rightarrow$ 7 分频，最终输出频率？

解： $f_{out} = 100\text{MHz}/(5 \times 7) = 100/35 \approx 2.857\text{MHz}$

分频器类题目的统一解法：

1. 计算分频比： $M = f_{in}/f_{out}$
2. 设计模 M 计数器
3. 偶数分频： $\text{cnt} < M/2$ 输出 0，否则输出 1
4. 奇数分频：用双沿技术实现 50% 占空比
5. 2^N 分频：直接取 Q_n 输出（最简单）
6. 级联：总 $M =$ 各级 M 的乘积

第四十一章 移位寄存器考试变式题

题目 1：串并转换（SIPO）

题目：设计一个 4 位串入并出移位寄存器。串行输入数据，4 个时钟周期后并行输出完整 4 位数据。

VHDL:

```
1 signal reg : std_logic_vector(3 downto 0);
2 process(clk) begin
3     if rising_edge(clk) then
4         reg <= si & reg(3 downto 1);  -- 串行右移
5     end if;
6 end process;
7 po <= reg;
```

逐拍分析（输入 1011）:

拍	输入	[Q3 Q2 Q1 Q0]
1	1	[1 0 0 0]
2	0	[0 1 0 0]
3	1	[1 0 1 0]
4	1	[1 1 0 1]

4 拍后 po=1101（数据反转了，因为右移）。如想保持顺序，用左移。

题目 2：并串转换（PISO）

题目：设计 4 位并入串出移位寄存器。先 load 并行数据，然后移位输出。

VHDL:

```
1 signal reg : std_logic_vector(3 downto 0);
2 process(clk) begin
3     if rising_edge(clk) then
4         if load = '1' then
```

```

5      reg <= data_in;
6      else
7          reg <= '0' & reg(3 downto 1);  -- 右移，高位补0
8      end if;
9  end if;
10 end process;
11 so <= reg(0);

```

题目 3：双向移位寄存器

题目：设计 4 位双向移位寄存器。dir=1 右移，dir=0 左移。

```

1  if dir = '1' then
2      reg <= si_right & reg(3 downto 1);
3  else
4      reg <= reg(2 downto 0) & si_left;
5  end if;

```

题目 4：带并行装载的通用移位寄存器（74LS194 风格）

题目：设计类似 74LS194 的 4 位通用移位寄存器。支持：保持、右移、左移、并行装载四种模式。

VHDL:

```

1  case mode is
2      when "00" => null;                -- 保持
3      when "01" => reg <= si_r & reg(3 downto 1);  -- 右移
4      when "10" => reg <= reg(2 downto 0) & si_l;  -- 左移
5      when "11" => reg <= data_in;            -- 装载
6  end case;

```

题目 5：桶形移位器

题目：设计 8 位桶形移位器，一次可移 0-7 位（移位量由 3 位输入控制）。

分析：桶形移位器是纯组合逻辑（不需要时钟），本质上是多路选择器网络。

题目 6：移位寄存器的延迟线应用

题目：用移位寄存器实现 4 拍延迟线（输入信号延迟 4 个时钟周期后输出）。

解：4 位移位寄存器，so 输出 = 4 拍前的 si 输入。

题目 7：序列检测用移位寄存器方案

题目：用移位寄存器 + 比较器检测序列 1011（替代状态机方案）。

分析：4 位移位寄存器存储最近 4 位输入。当 reg="1011" 时输出 1。

```
1 reg <= data_in & reg(3 downto 1);  
2 detected <= '1' when reg = "1011" else '0';
```

题目 8：8 位移位寄存器

题目：设计 8 位串入并出移位寄存器。

答案：位宽从 4 扩展到 8，其他逻辑完全相同。

题目 9：循环移位寄存器

题目：设计循环移位寄存器（最低位移入最高位，不丢失数据）。

VHDL:

```
1 reg <= reg(0) & reg(7 downto 1); -- 循环右移
```

题目 10：算术移位寄存器

题目：设计算术右移寄存器（保持符号位不变）。

```
1 reg <= reg(7) & reg(7 downto 1); -- 算术右移：符号位不变
```

题目 11：逻辑移位 vs 算术移位

题目：比较逻辑左移和算术左移的区别。

分析：左移时逻辑移位和算术移位等价（都补 0）。右移时：逻辑右移补 0，算术右移补符号位。

题目 12：移位寄存器实现乘除法

题目：用移位寄存器实现 $\times 2$ 和 $\div 2$ 运算。

分析：

- 左移 1 位 = $\times 2$ ($\text{reg} \leftarrow \text{reg}(6 \text{ downto } 0) \& '0'$)
- 右移 1 位 = $\div 2$ ($\text{reg} \leftarrow '0' \& \text{reg}(7 \text{ downto } 1)$)

题目 13：给定移位寄存器初值和输入序列，推断最终状态

题目：4 位右移寄存器初值 0000，输入序列 1011。4 拍后寄存器内容？

解（逐拍）：

- 初始：[0000]
- 拍 1(si=1)：[1000]
- 拍 2(si=0)：[0100]
- 拍 3(si=1)：[1010]
- 拍 4(si=1)：[1101]

4 拍后：[1101]

题目 14：移位寄存器 Testbench 设计

题目：为 8 位移位寄存器设计完整的 Testbench，验证串入并出功能。

题目 15：移位寄存器构建伪随机序列（LFSR）

题目：用 4 位移位寄存器 + 反馈构建最大长度 LFSR。

分析：反馈多项式 $X^4 + X^3 + 1$ 。反馈 = Q3 XOR Q2。产生序列长度 = $2^4 - 1 = 15$ （全 0 状态除外）。

移位寄存器类题目的统一解法：

1. 右移: `reg <= si & reg(N-1 downto 1)`
2. 左移: `reg <= reg(N-2 downto 0) & si`
3. 循环: `reg <= reg(0) & reg(N-1 downto 1)`
4. 并行装载: `if load='1' then reg <= data_in;`
5. 串出: `reg(0)` (右移) 或 `reg(N-1)` (左移)
6. 并出: 整个 `reg`

第四十二章 环形计数器考试变式题

题目 1：4 位环形计数器

题目:设计 4 位环形计数器。初始状态 1000,经历 4 个状态循环:1000→0100→0010→0001→1000。

VHDL:

```
1 signal reg : std_logic_vector(3 downto 0);
2 process(clk, rst_n)
3 begin
4     if rst_n = '0' then reg <= "1000";
5     elsif rising_edge(clk) then
6         reg <= reg(0) & reg(3 downto 1);
7     end if;
8 end process;
```

逐拍验证:

拍	[Q3 Q2 Q1 Q0]
0	[1 0 0 0]
1	[0 1 0 0]
2	[0 0 1 0]
3	[0 0 0 1]
4	[1 0 0 0] □ 循环

题目 2：8 位环形计数器

题目: 设计 8 位环形计数器。初始 10000000。8 个状态循环。

VHDL:

```
1 signal reg : std_logic_vector(7 downto 0);
2 reg <= reg(0) & reg(7 downto 1); -- 8位环形右移
```

题目 3：4 位约翰逊计数器（扭环形）

题目：设计 4 位约翰逊计数器。初始 0000。产生 8 个状态。

VHDL:

```
1 reg <= (not reg(0)) & reg(3 downto 1);
2 -- 状态序列: 0000+1000+1100+1110+1111+0111+0011+0001+0000
```

状态分析:

拍	[Q3 Q2 Q1 Q0]
0	[0 0 0 0]
1	[1 0 0 0]
2	[1 1 0 0]
3	[1 1 1 0]
4	[1 1 1 1]
5	[0 1 1 1]
6	[0 0 1 1]
7	[0 0 0 1]
8	[0 0 0 0] □

关键规律：约翰逊计数器（N 个 DFF）= 2N 个状态。

题目 4：带自启动的环形计数器

题目：环形计数器可能因干扰进入非法状态（如两个 1 同时为 1）。设计自启动逻辑。

分析：检测非法状态并自动恢复到合法状态。

VHDL:

```
1 if reg = "0000" or (reg(3)='1' and reg(2)='1') or ... then
2   reg <= "1000"; -- 恢复到合法状态
3 else
4   reg <= reg(0) & reg(3 downto 1);
5 end if;
```

题目 5：环形计数器 vs 二进制计数器比较

题目：比较 8 状态环形计数器和 8 状态二进制计数器。

	环形计数器	二进制计数器
DFF 数量	8 个	3 个 ($\lceil \log_2 8 \rceil$)
状态编码	One-hot	Binary
解码复杂度	0 (直接读)	需要译码器
最大速度	1 级 DFF 延迟	进位链延迟

题目 6：左移环形计数器

题目：设计左移环形计数器（1 从右向左移动）。

VHDL:

```
1 reg <= reg(2 downto 0) & reg(3);  -- 左移循环
2 -- 序列: 0001+0010+0100+1000+0001
```

题目 7：自校正约翰逊计数器

题目：设计自校正的 4 位约翰逊计数器，能自动从非法状态恢复到合法状态。

题目 8：用环形计数器做序列发生器

题目：用 4 位环形计数器的 Q 输出作为序列输出。产生什么序列？

解：Q3 输出序列：1, 0, 0, 0, 1, 0, 0, 0, ...（周期 4）各 Q 输出产生不同相位的同一序列。

题目 9：环形计数器的状态数推导

题目：N 位移位寄存器可构成多少种不同状态的环形计数器？

解：One-hot 编码，N 个状态。每个状态只有一个 1。

题目 10：约翰逊计数器的状态数推导

题目：N 位移位寄存器可构成多少种不同状态的约翰逊计数器？

解：2N 个状态。

题目 11：给定波形推断计数器类型

题目：4 位寄存器输出波形为：1000→0100→0010→0001→1000…。判断类型和状态数。

解：4 位环形计数器，4 个状态。

题目 12：给定初值和反馈逻辑推断状态序列

题目：5 位移位寄存器初值 11000，反馈 = $Q_0 \text{ XOR } Q_1$ 。推断前 8 个状态。

分析：这是 LFSR 的行为，需要逐拍计算反馈值并移位。

题目 13：环形计数器的 one-hot 特性优势

题目：为什么 One-hot 编码的环形计数器适用于高速设计？

分析：

- 状态解码不需要任何组合逻辑（直接看哪个 DFF=1）
- 关键路径只有 1 级 DFF 延迟（没有加法器进位链）
- 适合超高速状态机
- 代价：状态数 = DFF 数（资源效率低）

题目 14：用环形计数器实现时序控制器

题目：用 5 位环形计数器实现一个 5 步时序控制器，每步对应一个控制信号。

解：Q4-Q0 分别作为步骤 1-5 的控制信号。每个时钟周期只有一个信号有效。

题目 15：扭环形计数器做分频器

题目：4 位扭环形计数器（8 状态）。各 Q 输出的频率是多少？

解：每个 Q 完成一个完整周期需要 8 拍。

$$f_Q = f_{clk}/8$$

环形计数器类题目的统一解法：

1. **结构：**移位寄存器 + Q 输出反馈到输入
2. **普通环形：**反馈同相 $\rightarrow N$ 状态 (N 个 DFF)
3. **扭环形 (约翰逊)：**反馈反相 $\rightarrow 2N$ 状态
4. **核心 VHDL：**`reg <= reg(0) & reg(N-1 downto 1)` (环形右移)
5. **自启动：**检测非法状态，强制恢复到 100...0
6. **优点：**超快 (无进位链)，输出直接可用 (无需译码)
7. **缺点：**状态数 = DFF 数 (资源效率低)

第四十三章 序列检测器考试变式题

题目 1：检测 1011（重叠检测）

题目：设计 Mealy 型序列检测器，检测 1011（重叠检测）。画出状态图，写 VHDL。

状态定义：

- S0：未匹配
- S1：已匹配“1”
- S10：已匹配“10”
- S101：已匹配“101”

关键状态转移：

- S101 + 输入 1 → 检测到（detected=1），去 S1（最后 1 位作为新序列的第一个 1——重叠）

题目 2：检测 1011（非重叠检测）

题目：设计 Moore 型序列检测器，检测 1011（非重叠）。检测到后重新开始。

关键区别：S101 + 输入 1 → 检测到，去 S0（重新开始——非重叠）。

题目 3：检测 1101（重叠）

题目：设计序列检测器，检测 1101（重叠）。画状态图。

状态定义：

- S0→S1→S11→S110（已匹配“110”）
- S110 + 1 = 检测到，去 S1（重叠：最后的“1”作为新序列的第一个“1”）
- 注意：S11 + 输入 1 = 仍留在 S11（“111”中后两个“11”匹配了“11”前缀）

题目 4：检测 0101（重叠）

题目：设计序列检测器，检测 0101。

状态：S0→S01→S010→S0101→检测

题目 5：检测 1111（4 个连续 1）

题目：设计检测连续 4 个 1 的序列检测器。

状态：

- S0→S1(1)→S11(2)→S111(3)→(第 4 个 1→检测)
- 任何时候输入 0，回到 S0
- S111 + 1 → detected=1，去 S111（重叠：最后 3 个 1 可以作为新序列的前 3 个 1）

题目 6：检测 1001

题目：设计检测 1001 的序列检测器。

状态：S0→S1→S10→S100→S1001(检测)

关键：S10+0→S100。S100+1→检测。S100+0→S0（”1000”不匹配任何前缀）。

题目 7：检测序列长度可配置

题目：设计可检测任意 4 位序列的可编程序列检测器（目标序列 TARGET 可配置）。

VHDL:

```
1 signal reg : std_logic_vector(3 downto 0);  
2 reg <= data_in & reg(3 downto 1);  
3 detected <= '1' when reg = TARGET else '0';
```

（这是用移位寄存器方案实现的可编程检测器）

题目 8：同时检测 1011 和 1101

题目：设计电路同时检测 1011 和 1101。

分析：合并两个状态机（状态共享）。状态包括所有可能的前缀。

题目 9：010 检测器（3 位序列）

题目：设计检测 010 的序列检测器。画状态图，写 VHDL。

状态：S0→S01(已匹配 01)→S010(检测到)

状态转移：

- S0 + 0 → S01 （”0” 已经匹配了 01 的第一个 bit）
- S01 + 1 → S010 （检测到!）
- S01 + 0 → S01 （”00”——最后的 0 仍然是有效的第一个 bit）
- S010 + 0 → S01 （检测后，最后的 0 作为新序列的第一个 bit——重叠）

题目 10：连续 N 个相同 bit 检测器

题目：设计检测连续 N 个 0（或连续 N 个 1）的检测器。

分析：用一个计数器统计连续相同 bit 的个数。

题目 11：Moore vs Mealy 检测器比较

题目：比较 Moore 型和 Mealy 型序列检测器检测 1011 的差异。

	Moore 型	Mealy 型
输出决定因素	仅当前状态	当前状态 + 输入
检测延迟	需进入检测状态（晚 1 拍）	同一拍输出
状态数	多 1 个（需要”检测到”状态）	少 1 个
输出稳定性	整个周期稳定	输入变化时可能变

题目 12：序列检测器 + 计数器统计检测次数

题目：扩展 1011 检测器，增加计数功能：统计检测到多少次 1011。

VHDL:

```
1 signal detect_count : integer range 0 to 255;
2 process(clk) begin
3     if rising_edge(clk) then
4         if detected = '1' then
5             detect_count <= detect_count + 1;
6         end if;
7     end if;
8 end process;
```

题目 13：带超时检测的序列检测器

题目：如果连续输入超过 16 个时钟周期未检测到序列，输出 timeout 信号。

分析：增加一个超时计数器。每次状态变化时清零，超过阈值时 timeout=1。

题目 14：双向序列检测器

题目：检测 1011 和 1101 中任意一个（两者共享前缀“1”）。

状态图合并： $S_0 \rightarrow S_1(1) \rightarrow$ 分支： $S_{10}(10)$ 或 $S_{11}(11)$ $S_{10} \rightarrow S_{101}(101) \rightarrow (1 \rightarrow$ 检测 1011) $S_{11} \rightarrow S_{110}(110) \rightarrow (1 \rightarrow$ 检测 1101)

题目 15：格雷码序列检测

题目：输入是并行 3 位格雷码，检测格雷码序列“000→001→011→010→110”的出现。

分析：这是并行输入的序列检测。状态机根据并行输入值转移。

题目 16：含“无关项”的序列检测

题目：检测 1x1x（第一位和第三位为 1，其他任意）。例：1010 匹配，1111 也匹配。

分析：用移位寄存器存储最近 4 位，只比较第 3 位和第 1 位是否为 1。

题目 17：给定状态图，写 VHDL

题目：给定状态转移图，写出对应的 VHDL 状态机代码。

秒杀方法：

1. type 定义状态枚举（每个状态一个枚举值）
2. process 敏感列表 = clk, rst_n
2. case 语句逐个处理每个状态的转移
3. 输出逻辑根据状态（Moore）或状态 + 输入（Mealy）赋值

题目 18：给定输入序列和状态机，求输出

题目：1011 检测器（重叠），输入 = 1011011。问输出序列。

解（逐拍分析）：

拍	1	2	3	4	5	6	7
输入	1	0	1	1	0	1	1
状态	S1	S10	S101	S1	S10	S101	S1
输出	0	0	0	1	0	0	1

输出序列：0 0 0 1 0 0 1（第 4 拍和第 7 拍检测到）

题目 19：状态编码优化

题目：1011 检测器有 4 个状态。分别用二进制编码和 One-hot 编码实现。比较资源消耗。

分析：

- 二进制编码：2 个 DFF ($2^2 = 4$ 状态)，需要组合逻辑解码
- One-hot 编码：4 个 DFF，无需解码逻辑
- 速度 vs 面积的经典权衡

题目 20：复杂序列检测——检测 101 后跟 11

题目：检测序列”101”后紧跟”11”（即检测 10111）。

状态：S0→S1→S10→S101→S1011→S10111(检测)

序列检测器类题目的统一解法——核心步骤：第一步：理解目标序列。确定要检测的序列是什么。

第二步：确定状态。每个状态 = 已匹配的序列前缀。

- S0 = 匹配 0 位
- S 前缀名 = 匹配了前缀名的全部内容
- 最后一个前缀 = 差 1 位就匹配完整序列

第三步：画状态转移图。对每个状态，考虑输入 0 和 1 分别去哪。

第四步：写状态转移表。整理为表格形式。

第五步：写 VHDL。

```
1 type state_type is (...);
2 signal state : state_type;
3 process(clk, rst_n) begin
4     if rst_n='0' then state <= S0;
```

```

5   elsif rising_edge(clk) then
6       case state is
7           when S0 => if din='1' then state <= S1; end if;
8           when S1 => if din='0' then state <= S10; end if;
9           ...
10        end case;
11    end if;
12 end process;

```

序列检测器类题目的统一解法——核心步骤（续）：

第六步：区分重叠/非重叠。

- **重叠检测**：检测到后，分析当前输入的后几位能否作为下一次匹配的前缀
- **非重叠检测**：检测到后直接回 S0

常考序列状态数速查：

- 检测 1011：4 个状态（S0, S1, S10, S101）
- 检测 1101：4 个状态（S0, S1, S11, S110）
- 检测 0101：4 个状态（S0, S01, S010, S0101）
- 检测 1111：4 个状态（S0, S1, S11, S111）
- 检测 010：3 个状态（S0, S01, S010）
- 检测 101：3 个状态（S0, S1, S10）

43.1 全书总结

恭喜你读到这里。

现在回顾一下，你掌握了：

1. **7 个组合逻辑电路**：编码器、译码器、BCD 译码器、MUX、比较器、加法器、级联 MUX
2. **D 触发器的底层模型**：1 bit 存储，时钟沿驱动， $Q(n+1) = D(n)$
3. **计数器的统一框架**：所有模值计数器 = 同一个模板改 N-1
4. **分频器的本质**：计数器的一个应用， Q_n 天然分频

5. 序列发生器的多种方案：计数器 + 映射、状态机、移位寄存器
6. 移位寄存器的数据流动：逐拍分析，数据如水流过管道
7. 环形计数器的循环：反馈形成闭环，1 在环中无限循环
8. 序列检测器的状态机：状态 = 已匹配前缀，输入决定下一状态
9. VHDL 与硬件的对应：每行代码对应什么硬件
10. 100+ 道变式题的解题能力

核心信念：

不是背代码。
是理解电路。
理解它为什么存在。
理解它怎么工作。
理解状态如何在时钟推动下变化。
要相信，世间万般变体设计，皆是本源规则注定的衍生结果！

祝好。